

University of Stuttgart
Institute of Industrial Automation and
Software Engineering

Design & Implementation of an

Edge native Multi-modal Operational Risk Inference Architecture for Robust Autonomous Systems

MT-4006

Submitted at the University of Stuttgart by
Shashank Birur Manjunatha

Information Technology

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: M.Sc. Joachim Grimstad

24.05.2026



Abstract

Autonomous robots are increasingly deployed in environments they share with people, where the hazards they encounter depend on the operational situation rather than on the system in isolation. New situations arise during operation at a rate that pre-deployment analysis cannot anticipate, and the risk registry that governs the deployment, authored once at design time and revised on a fixed schedule, does not remain representative of the conditions the deployed system actually encounters. Maintaining a current account of those conditions is the substance of the post-market monitoring obligation that Article 72 of the EU AI Act imposes on providers of high-risk AI systems, and no existing tool directly addresses it.

This thesis proposes an edge-native multimodal architecture for operational risk inference, producing structured safety artifacts in the FMECA format that safety engineers already use. The architecture comprises two stages connected by a structured scene representation. The perception stage constructs the representation from the onboard camera stream. The reasoning stage, implemented around a small language model, queries a knowledge graph of design-time hazard entries for those relevant to the current scene. When the retrieved entries cover the hazards present in the scene, they serve as the basis for the FMECA entries generated for the scene. When they do not, the model retrieves additional context from a secondary store of incident reports and academic literature, generates entries from that material, and writes them back to the knowledge graph as candidate entries for review by a human verifier.

The architecture is implemented on a single Nvidia Jetson AGX Orin for a quadruped robot delivering a payload across a shared laboratory, and is evaluated offline on a recorded sensor stream across several small language model configurations and inference backends. The evaluation considers four properties that a safety-relevant component must satisfy together: the ability to produce a parseable output, end-to-end latency, agreement with a reference authored by a safety engineer, and reproducibility of the output across repeated runs on identical input. The evaluation demonstrates that the architecture shows promise and can serve as a viable basis for generating auditable FMECA artifacts from the recorded operation of the robot, providing operational evidence to the design-time risk analysis for subsequent revisions.

die Zusammenfassung

Autonome Roboter werden zunehmend in Umgebungen eingesetzt, die sie sich mit Menschen teilen, wobei die Gefahren, denen sie ausgesetzt sind, eher von der jeweiligen Betriebssituation als vom System an sich abhängen. Während des Betriebs entstehen neue Situationen in einer Geschwindigkeit, die im Rahmen einer Analyse vor dem Einsatz nicht vorhersehbar ist, und das Risikoregister, das den Einsatz regelt – das einmalig in der Entwurfsphase erstellt und nach einem festen Zeitplan überarbeitet wird –, spiegelt nicht mehr die Bedingungen wider, denen das eingesetzte System tatsächlich ausgesetzt ist. Die laufende Erfassung dieser Bedingungen ist der Kern der Überwachungspflicht nach dem Inverkehrbringen, die Artikel 72 des EU-KI-Gesetzes den Anbietern von KI-Systemen mit hohem Risiko auferlegt, und kein bestehendes Tool ist direkt darauf ausgerichtet.

Diese Arbeit schlägt eine Edge-native multimodale Architektur für die Ableitung von Betriebsrisiken vor, die strukturierte Sicherheitsartefakte im FMECA-Format erzeugt, das von Sicherheitsingenieuren bereits verwendet wird. Die Architektur umfasst zwei Stufen, die durch eine strukturierte Szenenrepräsentation miteinander verbunden sind. Die Wahrnehmungsstufe erstellt die Repräsentation aus dem Stream der Bordkamera. Die Schlussfolgerungsstufe, die auf einem kleinen Sprachmodell basiert, fragt einen Wissensgraphen mit Gefahren-Einträgen aus der Konstruktionsphase nach den für die aktuelle Szene relevanten Einträgen ab. Wenn die abgerufenen Einträge die in der Szene vorhandenen Gefahren abdecken, dienen sie als Grundlage für die für die Szene generierten FMECA-Einträge. Ist dies nicht der Fall, ruft das Modell zusätzlichen Kontext aus einem sekundären Speicher mit Unfallberichten und wissenschaftlicher Literatur ab, generiert Einträge aus diesem Material und schreibt sie als Kandidateneinträge zur Überprüfung durch einen menschlichen Prüfer zurück in den Wissensgraphen.

Die Architektur wird auf einem einzelnen Nvidia Jetson AGX Orin für einen vierbeinigen Roboter implementiert, der eine Nutzlast durch ein gemeinsames Labor transportiert, und wird offline anhand eines aufgezeichneten Sensor-Datenstroms über mehrere Konfigurationen kleiner Sprachmodelle und Inferenz-Backends hinweg evaluiert. Die Bewertung berücksichtigt vier Eigenschaften, die eine sicherheitsrelevante Komponente gemeinsam erfüllen muss: die Fähigkeit, eine analysierbare Ausgabe zu erzeugen, die End-to-End-Latenz, die Übereinstimmung mit einer von einem Sicherheitsingenieur erstellten Referenz sowie die Reproduzierbarkeit der Ausgabe bei wiederholten Durchläufen mit identischer Eingabe. Die Bewertung zeigt, dass die Architektur vielversprechend ist und als tragfähige Grundlage für die Generierung überprüfbarer FMECA-Artefakte aus dem aufgezeichneten Betrieb des Roboters dienen kann, wodurch operative Belege für die Risikoanalyse in der Entwurfsphase für nachfolgende Überarbeitungen bereitgestellt werden.

Preface and Acknowledgments

This research work was undertaken by me during the fifth and final semester of the Master of Science in Information Technology (INFOTECH) under the Institute of Industrial Automation and Software Engineering (IAS) at the University of Stuttgart.

I would like to thank my supervisor, Joachim Grimstad, for his support throughout this thesis, for his suggestions and guidance, and for giving me the freedom to explore, experiment, and discover the realm of AI assurance for autonomous robotic systems.

My sincere gratitude goes to Prof. Andrey Morozov, first for the wonderful course on Risk Assessment for Robotic Systems, which stands as one of the finest learning experiences of my time at the University of Stuttgart. It was while sitting in those lectures that I felt drawn to the world of safety engineering, and the questions raised there have since become the foundation of this work. Second, and most importantly, for offering me the opportunity to take up this novel topic, and for providing the means and the inspiration necessary for its successful completion.

To my parents and grandparents, thank you for believing in me and for standing by my side as I pursue my dreams so far from home. Your faith has carried me across distances that are difficult to put into words.

To my dear friends Ruthu, Praveen, Suhruth, Vignesh, and Nandini, thank you for keeping me motivated and focused throughout this journey. To the others not named here, you know who you are. I am lucky to have had you all along for this.

Finally, I would like to thank Vaishnavi, my best friend of eight years, for the motivation, the inspiration, and for putting up with my tantrums, for listening to every one of my sagas, and for sticking around so long. This work, in its quiet way, is for you muu.

Table of Contents

Abstract i

Zusammenfassung ii

Preface and Acknowledgements iii

List of Figures vii

List of Tables viii

1 Introduction 1

1.1 Motivation . . . 1
1.2 Aim and Objectives . . . 2
1.3 Approach . . . 2
1.4 Scope and Limitations . . . 3
1.5 Thesis Outline . . . 3

2 Literature Review 4

2.1 Safety Analysis - Design-Time to Run-Time . . . 4
2.2 Safety in Human-Machine Collaboration . . . 6
2.3 Scene Representations for Language-Model Reasoners . . . 8
2.4 Knowledge-Augmented LLM Reasoning for Safety . . . 10
2.5 Edge Deployment of Language and Vision-Language Models . . . 12

3 Concept 15

3.1 Perception . . . 15
3.2 Cognition . . . 17

4 Methodology 19

4.1 Sense . . . 20
4.2 Speculate . . . 26

5 Evaluation and Results 35

5.1 Test Scenario . . . 35
5.2 Models Under Test . . . 36
5.3 Speculation Pipeline Configuration . . . 37
5.4 GPU Scheduling Modes . . . 38
5.5 Scoring Instruments . . . 38

5.6 Viability . . . 39
5.7 Timeliness . . . 41
5.8 Correctness . . . 45
5.8.1 Scene Description Accuracy . . . 45
5.8.2 FMECA Quality . . . 48
5.9 Determinism . . . 50
5.9.1 Scene Description Consistency . . . 51
5.9.2 FMECA Ordinal Agreement . . . 52
5.9.3 Pipeline Path Agreement . . . 54
5.10 Effects of Writeback . . . 55
5.10.1 Latency . . . 56
5.10.2 Correctness . . . 57

6 Discussion and Conclusion **59**

7 Future Scope **62**
7.1 Task-Specific Fine-Tuning for Hazard Identification . . . 62
7.2 Perception and a Tighter Loop . . . 62
7.3 Offloading Normative Retrieval to a Near-Edge Service . . . 63
7.4 Automating Writeback Verification with a Larger Fine-Tuned Model . . . 63
7.5 Deployment on Jetson Thor . . . 63

Bibliography **64**

Appendix **72**
A ROS2 bag consumed by SENSE . . . 72
B G-Eval Rubrics . . . 72
B.1 Scene Description Accuracy . . . 72
B.2 FMECA Faithfulness . . . 74
B.3 FMECA Ground-Truth Alignment . . . 74
C Producer-Consumer Rate Measurement . . . 75
D Lock-Free GPU Scheduling Stall . . . 75
E Parallel Consumer Experiment . . . 76
F Qwen3-4B Thinking Model Failure Analysis . . . 77
G Mechanism of Run-to-Run Variability in Language Model Inference . . . 78
G.1 Autoregressive decoding . . . 78
G.2 Logits, softmax, and sampling . . . 78

G.3 Matrix multiplication and accumulation 79
G.4 Floating-point non-associativity 79
G.5 Where the accumulation order is chosen 79
G.6 From token drift to ordinal disagreement 80
H Writeback Per-Cell Latency on Already-Sufficient Scenes 80
I Seed process FMEA 81
J Pipeline Prompt Templates 83
J.1 Scene Description 83
J.2 Sufficiency Check 83
J.3 FMECA Generation 84
K Ground-Truth Scene Descriptions and FMECA Entries 85

List of Figures

1 Concept overview of the framework. 15
2 Image-grid representation from IG-VLM. 16
3 Overview of Agent-G from Singh et al. [80]. 18
4 Framework overview 19
5 Sequence diagram of one scene window through SENSE 21
6 Class diagram of the SENSE stage 22
7 Egocentric H-pattern overlaid on a representative scene window. 24
8 The five stages of the SPECULATE pipeline 26
9 Sequence diagram of one scene through SPECULATE. 27
10 Class diagram of the SPECULATE stage 28
11 Build and runtime flow of the FMEA knowledge graph. 30
12 Extract-transform-load pipeline for the normative document store. 31
13 Representative keyframes from the test recording. 36
14 Completion rate per model under the two scheduling regimes. 40
15 SmolLM3-3B failure analysis: reasoning trace consumption across nine scenes. . 40
16 Mean pipeline latency for the models served through TensorRT-LLM 43
17 Distribution of per-scene pipeline latency under concurrent GPU scheduling. . . . 43
18 Distribution of per-scene pipeline latency under sequential GPU scheduling. 43
19 Backlog queue wait time for Qwen2.5-3B-Instruct under concurrent scheduling . . 44
20 Mean pipeline latency for the non-TensorRT-LLM configurations 45
21 Scene Description Accuracy G-Eval scores per scene 46
22 FMECA Faithfulness per scene 48
23 FMECA Ground-Truth Alignment per scene 49
24 Inter-run scene description similarity across the three repeated runs for each model. 51
25 Krippendorff's Alpha on ordinal values across the three repeated runs for each model 52
26 Krippendorff's Alpha TensorRT-LLM vs vLLM 54
27 Pipeline path assignments across the three repeated serialized runs for each model. 54
28 Latency, correctness, and determinism tradeoffs 59
29 Per-batch backlog queue wait time for two concurrent runs of Qwen2.5-3B-Instruct 76
30 Per-batch inference time, single worker and two concurrent workers. 77
31 Qwen3-4B (Thinking) Failure 78

List of Tables

1 Severity levels. 33
2 Probability levels. 33
3 Criticality matrix 33
4 Evaluated Model Configurations 36
5 Jetson software-stack constraints behind each quantization-and-backend pairing. . 37
6 Pipeline stage configuration 38
7 Measured decode throughput on the Jetson AGX Orin 64GB. 42
8 BERTScore F1 summary statistics across the nine evaluation scenes. 46
9 End-to-end latency per transition category 56
10 Stage-level latency decomposition on the six N→S scenes, averaged across runs 57
11 Correctness metrics under baseline and writeback-enabled configurations 58
12 Roles of the recorded ROS2 topics inside the SENSE stage. 72
13 Scoring rubric passed to the judge for Scene Description Accuracy. 73
14 Producer-consumer throughput, Qwen2.5-3B-Instruct (instrumented run). 75
15 Failure mode comparison between thinking-enabled models. 78
16 Per-repeat end-to-end latency on the two S→S scenes under sequential scheduling. 81
17 Queue-wait and inference time on the two S→S scenes under concurrent scheduling 81
18 Excerpt of the hand-authored process FMEA covering the navigation entries. . . . 82
19 Runtime token substitutions. 83
20 FMECA JSON schema. 84
21 Ground-truth scene descriptions for the nine evaluation scenes. 85
22 Ground-truth FMECA entries for the nine evaluation scenes, structured fields. . . . 86
23 Severity and probability rationales for the ground-truth FMECA entries. 86

1 Introduction

Autonomous robotic systems have been deployed across a range of domains, including agriculture, personal care, road and air logistics [1]. These systems co-exist with humans and must adapt to dynamic, complex environments. If a system and its environment were entirely understood a priori, any attainable objective could have been accomplished through pre-defined control logic. But uncertainties are inevitable in real-world implementations. They arise from environmental variability that cannot be predicted, long-term system degradation, sensor limitations that yield incomplete or noisy measurements, unanticipated disturbances, and the unpredictability of socio-technical interaction. Managing this uncertainty requires some degree of autonomy [2]. For such systems, operational risk is in turn shaped less by component reliability than by the operational context the system is in and how well it is prepared to handle that situation [3]. Autonomous robot safety, therefore, remains a challenge, because design-time guarantees cannot be established for the scenarios such a system encounters in its operating environment [4].

1.1 Motivation

Safety assurance for industrial machinery has rested on a stable analytical procedure for several decades. A hazard inventory is constructed at design time, controls are specified to contain each hazard within tolerable residual risk, and the resulting system is evaluated. Established safety assurance standards, such as IEC 61508, which covers the functional safety of electrical/electronic systems, and ISO 13849, which covers the safety of machinery control systems, do not account for the risks posed by unknown unknowns [4]. Where the operating environment cannot be defined in full at design time, the residual uncertainty is absorbed into worst-case assumptions and the system is designed defensively against them, producing an assurable design at the cost of performance in normal operation [5]. The procedure targets the ways a system can fail. It does not reach hazards that arise with no component failure at all, where the system performs exactly as designed yet meets an operating scenario its designers did not foresee. The conditions a system is built to operate within can be bounded at design time as its Operational Design Domain (ODD), yet the situations that arise inside that domain cannot be enumerated in advance.

For autonomous robots deployed in shared, dynamic environments, the assumptions on which the design-time procedure rests do not always hold. The AI Incident Database [6], a public catalog maintained by the Responsible AI Collaborative, currently logs several hundred reported incidents involving autonomous physical systems, comprising delivery robots, security robots, autonomous vehicles, and industrial cobots, and the catalog is continuously growing with entries reported by deployers, regulators, and the press. The hazards documented across those entries are overwhelmingly situational, comprising collisions with bystanders the deployment had not cataloged, vehicle-pedestrian strikes under environmental conditions the test fleet had not encountered, and incidents arising from human-robot proximity in retail and public spaces. The MIT AI Risk Repository [7], a systematic review aggregating risks from seventy-four published AI risk frameworks, currently records more than seventeen hundred distinct hazards, and reports that individual frameworks typically cover only a subset of identified risk subdomains, evidence that the hazard space for AI-driven autonomy is open. The hazards an autonomous deployment encounters are properties of the situation in which the system finds itself rather than of the system in isolation, and any hazard inventory drawn for its ODD before deployment is structurally incomplete.

This inventory, the risk registry that governs the deployment, is built once at design time and updated on a schedule that bears no relation to how often the deployed system actually encounters new situations. Scenario-driven and behavior-driven development share this limitation, since the scenarios they define are fixed before deployment and describe expected behavior rather than the operational threats the system meets in the field. Even a modest fleet could generate operational evidence faster than the safety engineer reviewing it could process, and the imbalance could scale with the size of the deployment. The result is a registry that ages against the fleet rather than with it.

The architectural response to this, advanced in the runtime safety literature, is to relocate the risk

assessment activity itself from a one-off design-time exercise to a continuous runtime process, Dynamic Risk Assessment, in which the system reads the situation it occupies and estimates the residual risk it carries [5]. The relocation however does not eliminate design-time assurance. It restricts the design-time assurance argument to the runtime monitor itself, which becomes the artifact whose safety must be established before deployment.

This relocation is not only a proposal of the safety literature but now, a regulatory requirement. The EU AI Act requires providers of high-risk AI systems to operate a post-market monitoring loop [8]. The motivation for this work is to give the safety engineer a tool that helps close that loop. The tool runs alongside the deployment, drafts FMECA-style risk artifacts from what the autonomous robot observes during operation, and expands the risk registry with the entries the engineer promotes after review. Those entries are grounded in the operating environment the robot encounters rather than its ODD, and the scene behind each entry is preserved as test material against which future iterations of the system can be trained and validated.

1.2 Aim and Objectives

This thesis sets out to design and prototype an edge-native software architecture for runtime operational risk inference in autonomous systems, fusing onboard perception, curated domain knowledge, and recorded operational experience into structured, auditable risk artifacts.

Five objectives carry this aim into the work that follows.

1. To define a software architecture that is edge-first, modular, and scalable, capable of performing operational risk inference, adaptable across a range of autonomous platforms, and able to evolve with the operational experience accumulated during deployment.
2. To reason over the operating environment by combining what the autonomous system observes during operation with domain knowledge made available to it, so that the resulting risk assessment reflects both the present situation and the body of safety knowledge relevant to it.
3. To produce grounded risk artifacts that conform to an established format, accompanied by a mechanism that exposes the reasoning underlying each artifact for assessment of its plausibility.
4. To deploy the architecture on representative edge hardware and characterize its behavior against the resource and performance constraints of that hardware.
5. To evaluate and benchmark different reasoning and perception techniques within the architecture, along the dimensions that a safety-relevant system must satisfy.

1.3 Approach

A robot deployed in a dynamic environment does not, by itself, know how to analyze the risks it might encounter. It may be trained to avoid hazards, but that is not the same as analyzing those risks as they arise and enumerating them. Its designers will have considered a few situations it might encounter at design time, in ODD, and those analyses can serve as a starting point. As it operates, the robot reads the environment and asks whether that situation is one that the prior analyses already account for. Where it is, the robot draws its assessment from the precedent it already holds. Where it is not, it draws on a language model to inform itself from a wider body of safety material, assesses the situation based on that, and submits the new case for review, so that, once accepted, it becomes precedent for the next encounter and future improvements. The framework proposed in this work gives this loop a concrete form, operating in two stages connected by a single structured handoff.

The first stage, SENSE, is the robot reading the situation in front of it. It watches the robot's onboard camera and builds a structured account of the situation, the scene, reporting which objects are present, where they are relative to the robot, and how they are moving. To each object it attaches the safety-relevant vocabulary a domain expert has recorded for that class of object, so that the reasoning stage that follows is not left to infer that vocabulary on its own.

The second stage, SPECULATE, is the robot asking what that situation could lead to and whether it has encountered a similar situation before. It turns the scene into risk entries in the FMECA format that safety engineers already use. It first queries the risk registry, the body of hazard analyses prepared at design time, for entries relevant to the scene. A check then asks whether those entries cover the hazards present in the scene. If they do, the assessment is drawn from that precedent.If they do not, the stage consults a secondary store of incident reports and academic safety literature, and generates new entries grounded in that wider material. Entries generated on this second path are written back to the registry as candidates, held for a human reviewer to accept before they become precedent for future encounters.

1.4 Scope and Limitations

This work is scoped to a quadruped robot delivering a payload across a shared laboratory, defined in full in Section 5.1, and the perception parameters, the priors table, and the FMEA knowledge graph that ship with the prototype reflect that deployment scenario. The framework itself is platform-neutral, but the egocentric perception parameters are tuned for a quadruped's camera placement; a different platform will require its own tuning. The deployment hardware is a single Nvidia Jetson AGX Orin 64GB Developer Kit, and evaluation findings reported here are bound to the device. The promotion of writeback entries from pending status to indexed precedent is held as a manual step, in line with the argument that runtime monitors do not remove human responsibility for safety but relocate it from establishing safety directly to assuring that the monitor itself will not underestimate the risks of the environment in which it is deployed.

1.5 Thesis Outline

The remainder of this thesis is organized as follows. Section 2 reviews the literature on Dynamic Risk Assessment, scene representations for language-model reasoners, knowledge-augmented reasoning for safety-critical domains, and the edge deployment of language and vision-language models, and identifies the gaps the present work addresses. Section 3 develops the conceptual framework behind the architecture. Section 4 sets out the methodology, tracing one scene end to end through SENSE and SPECULATE. Section 5 reports the evaluation across viability, latency, correctness, determinism, and the effect of the writeback loop. Section 6 discusses the findings in the context of the regulatory framework and the wider runtime safety literature. Section 7 sets out the future scope.

2 Literature Review

This chapter reviews the literature on the safety of autonomous robots that share space with people, and on the perception and reasoning components on which an operation-time safety assessment of such robots can be built.

2.1 Safety Analysis - Design-Time to Run-Time

Safety analysis of autonomous systems is governed by a set of international standards that define hazard and risk and by a family of analytic techniques developed to apply those definitions to engineered systems. This subsection reviews the standards and techniques, their application to robotic platforms in published case studies, and the more recent proposals for runtime assessment.

ISO/IEC Guide 51 [9], the foundational standard governing safety terminology across all ISO safety standards, defines a hazard as a potential source of harm, encompassing physical injury or damage to people's health, damage to property, or damage to the environment. Risk is correspondingly defined as the combination of the probability of occurrence of harm and the severity of that harm [9]. These definitions are inherited by ISO 12100 [10], which applies them to machinery safety, and by ISO 13482 [11], which extends them to personal care robots operating in proximity to untrained users.

Fault Tree Analysis, Event Tree Analysis, and HAZOP are among the analytic techniques traditionally used to investigate potential accidents before they occur. These techniques are grounded in sequential accident models, which represent accidents as chains of discrete failure events propagating through system components [12]. While effective for relatively simple electromechanical systems, this chain-of-events paradigm has been shown to inadequately capture the unsafe interactions between components that characterize accidents in complex, software-intensive systems [12].

Departing from the chain-of-events paradigm, the System Theoretic Accident Model and Processes (STAMP) reconceptualizes accidents as the result of inadequate control, rather than as mere sequences of component failures. Building on this, System-Theoretic Process Analysis (STPA) is a method that can be applied at any stage of the system life cycle [13]. STPA consists of two primary steps. The first identifies potential inadequacies in system control that could lead to hazardous states, and the second determines how each potentially hazardous control action can manifest [12]. Unlike traditional techniques that focus on component-level failure chains, STPA accounts for scenarios spanning the entire accident process, emphasizing the importance of system-level interactions and controls [14]. Despite its strengths, STPA requires analysts to predefine the control structure and constraints during system design, which can limit its adaptability to highly dynamic or rapidly evolving systems [15].

FMEA is a bottom-up approach to hazard analysis that systematically enumerates failure modes at the component or function level and traces their effects upward through the system [16]. Its variant, Process Failure Mode and Effects Analysis (PFMEA), shifts the unit of analysis from components to process steps, examining each step for potential failures and their downstream effects [17]. While PFMEA's process-step structure was developed for manufacturing, its applicability has been extended to domains where sequential operational workflows must be examined for failure modes. In healthcare, DeRosier et al. [18] adapted FMEA into Healthcare Failure Mode and Effect Analysis (HFMEA), a five-step prospective risk assessment method that maps clinical processes into sequential steps and applies a hazard scoring matrix and decision tree to prioritize failure modes. HFMEA has since been applied to medication administration [19]. In the robotics domain, Ng et al. [20] proposed a Robot-Inclusive FMEA (RIFMEA) framework to identify failures in built environments that compromise service robots' workflows, assess their effects and causes, and provide recommended actions to alleviate these problems. Their work shifted the analytical focus from the robot's internal components to the environment's impact on robot operations, though the analysis remained a design-time evaluation of static building features rather

than the robot's dynamic task execution. Schmitt et al. [21] performed the same transfer to the machine-learning development pipeline, treating the lifecycle stages of data collection, preprocessing, training, validation, and deployment as the process steps of a PFMEA and providing a pre-populated template through which an analyst enumerates failure modes, causes, effects, and mitigations for each stage. What these adaptations share is the recognition that PFMEA's process-step structure is not inherently tied to manufacturing; any sequential operational workflow can serve as the analytical backbone.

Several studies have applied these techniques to robotic systems, illustrating both their strengths and the limitations of design-time hazard analysis. Guiochet [22] deployed HAZOP-UML on three robotic projects spanning healthcare and manufacturing, namely the MIRAS assistive rollator for elderly care, and the PHRIENDS and SAPHARI industrial coworkers. From the use case and sequence diagrams of each system, sixteen, twenty-one, and twenty-eight hazards were recovered for MIRAS, PHRIENDS, and SAPHARI respectively. The spread of these counts illustrates a defining property of HAZOP-UML. The size of the hazard inventory is a direct function of what the analyst has chosen to model, and gaps in the UML representation propagate as gaps in the analysis.

Alemzadeh et al. [23] applied STPA to the RAVEN-II open-source telerobotic surgical platform and identified unsafe control actions in the loops linking the master console, the control software, and the robotic arms acting on the patient. The hazardous interactions surfaced, including inappropriate motor commands and unsafe state transitions, are precisely those that, the authors argue, traditional FTA and FMEA techniques tend to miss because such methods focus on the failure of individual components rather than on the interactions between them. The same study went further and developed a software fault-injection framework to exercise the STPA-derived scenarios on the running robot, drawing an explicit bridge between the design-time hazard catalog and the runtime behavior of the deployed platform.

The RIFMEA framework introduced earlier [20] sits alongside the two studies above but reverses their analytical focus, extending FMEA from the robot itself to the built environment in which the robot operates and shifting attention from internal component failures to the static features of buildings, such as furniture layout, surface finishes, and clearance, that compromise robot workflows. Although the authors validate the framework by deploying a telepresence robot across multiple campus locations, the analytical artifact remains a design-time risk priority number ranking of building elements rather than a runtime assessment of robot behavior in those environments.

These three studies trace a common pattern. Each takes as its analytical input a model of the system constructed during design, whether a UML diagram, a hierarchical control structure, or a building floor plan, and produces a hazard inventory, comprising hazards and safety recommendations from HAZOP-UML, unsafe control actions and causal scenarios from STPA, and risk priority numbers from RIFMEA. Even where the work is complemented by runtime fault injection or empirical robot deployment, the catalog itself remains tied to the design-time model and is positioned as input to downstream safety mechanisms rather than as a runtime view of the deployed system.

Guiochet, Machin, and Waeselynck [24], in their survey of safety-critical advanced robots, attribute the inadequacy of classical dependability techniques to properties specific to such systems, namely the continuous physical interaction with humans and the non-deterministic decisional layers, and conclude that those techniques have to be adapted or revised to remain useful.

One adaptation pursued in response is Dynamic Risk Assessment. Wolf et al. [4] observe that conventional safety approaches restrict the operating capabilities of the robot to prevent hazardous situations, and that this can lead to a performance decrease in cases where the autonomy could theoretically handle the situation safely but providing design-time guarantees is impossible. Müller et al. [3] frame the problem in terms of situational risk, arguing that the risk of an autonomous mobile robot is no longer predominantly defined by the reliability of its components but by the scenario it is in and how well the system is prepared to handle it. Reich and Trapp [5] describe Dynamic Risk Assessment as a technique that shifts the risk assessment activity to runtime and enables the system itself to assess the risk of the current situation. Risk-relevant features of the situation are monitored and used as input to the assessment, and the resulting residual risk es-

timate is fed into a decision-making component that selects the applicable risk control measure closest to the situation-specific acceptable risk level, with adaptation schemes triggered to enact the selected measure.

Shifting the assessment to runtime does not, however, retire the design-time engineer. Reich and Trapp [5] note that an essential part of trustworthy DRA monitoring is the design-time assurance of the monitor itself, that is, how safety engineers argue and support the claim that the monitor is itself safe. Shifting the assessment to runtime therefore does not remove the need for design-time work but relocates it, from establishing safety directly to assuring that the runtime monitor will not underestimate the risk in the operating environment for which it was designed.

The same shift is reflected in the regulatory framework for AI systems in the European Union. Regulation (EU) 2024/1689, the Artificial Intelligence Act, classifies autonomous robots that perform safety-critical functions as high-risk AI systems, and imposes a set of obligations on the providers of such systems that extend across the full lifecycle of the deployed system [8]. Two of these obligations are directly relevant to how risk is assessed and managed during operation. Article 9 requires providers of high-risk AI systems to establish, implement, document, and maintain a risk management system, and defines that system as a continuous iterative process planned and run throughout the entire lifecycle of the high-risk AI system, requiring regular systematic review and updating [8]. Article 72 specifies the post-market monitoring obligation. It requires providers to establish a post-market monitoring system proportionate to the nature of the AI technologies and the risks of the high-risk AI system, and to actively and systematically collect, document, and analyze relevant data on the performance of the system throughout its lifetime, allowing the provider to evaluate the continuous compliance of the system with the requirements set out in the regulation [8].

2.2 Safety in Human-Machine Collaboration

Safety in human-machine collaboration is studied across stationary industrial, service-robot, and mobile-platform settings. This subsection reviews the standards bounding permissible interaction in these settings, the taxonomies of contact hazards, the empirical injury data linking robot kinematics to harm, and the proposals to adjust protective measures at operation time.

ISO 13482 [11] sets safety requirements for mobile servant, physical assistant, and personal carrier robots, all earthbound and traveling at no more than twenty kilometers per hour. It gives the hazards that the robot might encounter and states that the number and types of hazards are directly related to the nature of the robot application, the complexity of the installation, and the level of human-robot interaction incorporated. The standard requires that a risk assessment be conducted for the particular application being considered to determine the protective measures needed. Salvini, Paez-Granados, and Billard [25] argue that ISO 13482:2014 lacks requirements to protect pedestrians and bystanders, since its hazard inventory assumes private settings such as households and offices rather than public spaces in which crowds, proxemics, and occupant misbehaviour shape the hazard profile.

Salvini et al. [26] subsequently supplied a catalog of these public-space hazards, classifying the physical hazards arising from contact between mobile robots and pedestrians into six types. In a collision, the robot and another body hit each other with force. In a crush, the robot presses a person, or a part of the body, against a fixed structure or rolls over a limb. In a push or swipe, the robot rubs sideways against a person and, depending on the surface that makes contact, can cut, pinch, or burn. In a drag, clothing catches on protruding parts, hooks, or gaps in the robot exterior. In a touch, prolonged contact in a bottleneck or queueing situation exposes the person to pinch, cut, burn, or shock. Each of the six types is further characterized along two axes. The first is the intentionality of the contact, separating unintentional contact caused by accidents or malfunctions from intentional contact such as a person reaching out to request the robot's services. The second is the initiator of the contact, separating robot-to-human, human-to-robot, and cooperatively initiated contact. The authors note that the severity of any given hazard depends on the fragility of the person involved - with children, the elderly, and pregnant women at particular risk - and on the geometry and material of the robot exterior at the point of contact. Where Salvini et al. classify by

contact type, Haddadin [27] classifies by contact constraint, distinguishing unconstrained impact, partially constrained impact, constrained impact, clamping in the robot structure, and secondary impact, on the grounds that the injury produced by a given contact depends not only on the body area touched but on whether the body part can recoil from the contact or is trapped against a second surface.

ISO/TS 15066 catalogs four methods of collaborative operation [28]. In a safety-rated monitored stop, the robot ceases motion in the collaborative workspace before the operator enters and may operate non-collaboratively only when the operator is absent. In hand guiding, the operator transmits motion commands through a guiding device after the robot has achieved a safety-rated monitored stop. In speed and separation monitoring, the robot and the operator may move concurrently provided that the robot maintains at least the protective separation distance and initiates a protective stop should that distance be violated. In power and force limiting, physical contact may occur intentionally or unintentionally and risk reduction is achieved by keeping the robot below threshold values determined during the risk assessment. The document flags the skull, forehead, larynx, eyes, ears, and face as zones where contact must be prevented. The third mode is parameterized by the protective separation distance, defined as the sum of six contributions, namely the operator's change in location during the interval the robot needs to react and stop, the robot's change in location during its reaction time, its stopping distance, an intrusion-distance allowance drawn from ISO 13855, and the position uncertainties of the operator and the robot. Robla-Gómez et al. [29] review safe human-robot collaboration in industrial environments with a focus on impact mitigation, distinguishing mechanical-compliance approaches such as viscoelastic coverings, lightweight actuators, and lightweight robot structures from active approaches based on residual-signal collision detection and on RGB-D and time-of-flight sensing, and conclude that no single approach has yet displaced the others.

The four modes are bounded at design time, and a parallel line of work has examined whether their parameters can be adjusted online. Liu et al. [30] propose a dynamic modification of speed and separation monitoring in which risk-related indicators are pre-classified, weighted with thresholds adjustable across scenarios, and updated continuously to trade off safety and production efficiency. Similarly, Katranis et al. [31] monitor the distance between human body parts and the cobot, the cobot's Cartesian velocity, and the orientation of the human head in the collaborative workspace, transform each into a hazard indicator through a non-linear heuristic function, and aggregate the indicators into a total hazard level evaluated on an industrial dataset. Both target stationary industrial cobots and treat the operating mode of the collaborative workspace as fixed by the standards, modulating only the parameters within it, namely the separation distance threshold and the speed envelope, and leaving the choice of mode itself to the design-time hazard analysis.

Mobile platforms are governed by a separate family of standards. ANSI/RIA R15.08-1 [32] sets the design requirements for industrial mobile robots and ANSI/RIA R15.08-2 [33] the application requirements for industrial mobile robot systems and fleets, while ISO 3691-4 [34] specifies the functional-safety requirements for driverless industrial trucks, namely the personnel-detection performance level, braking, speed control, and stability that a moving platform must satisfy where a stationary cobot would instead be bound by ISO 10218. Paez-Granados and Billard [35] provide the empirical injury basis on which the speed limits in these standards rest, reporting from crash tests with adult and child dummies struck by a sixty-kilogram mobile service robot at one, one and a half, and three point one meters per second that fourteen percent serious head, twenty percent neck, and fifty percent chest injury risk are reached in children at the highest speed, with thirty-three percent tibia fracture risk in adults at the same speed. Field evidence on the same question is provided by Sanders, Şener, and Chen [36], who analyzed the seventy-seven robot-related accidents recorded in the United States Occupational Safety and Health Administration Severe Injury Reports between 2015 and 2022 and observed that mobile robots produced a distinct injury distribution from stationary manipulators, with leg and foot fractures dominating the mobile-robot incidents and finger amputations and torso injuries dominating the stationary-robot incidents. Pandey, Parasuraman, and Doshi [37] address the same divide at runtime, proposing a Generalized Safety Index that scores each detected human by relative distance, speed, and orientation, on the explicit grounds that the safety measures developed for manipulators do not transfer to a mobile robot moving among multiple humans.

2.3 Scene Representations for Language-Model Reasoners

The literature on scene representations for language-model reasoners falls broadly into two lines of work. One line absorbs perception, language understanding, and in some cases action generation, into a single vision-language or vision-language-action model. The other line keeps perception and reasoning as separate networks connected by structured representations of the scene.

The first line itself contains two sub-classes that differ in their output modality. The vision-language-action (VLA) sub-class produces action tokens directly from camera frames and natural-language instructions. Kim et al. [38] introduced OpenVLA, a seven-billion-parameter VLA that consumes camera frames as DINOv2 and SigLIP patch tokens fused into a Llama-2 backbone and emits discretised end-effector actions from the same head, trained on roughly nine hundred and seventy thousand real-robot trajectories from the Open X-Embodiment mix and reported to outperform the fifty-five-billion-parameter RT-2-X by sixteen-and-a-half absolute points across twenty-nine generalization tasks.

The vision-language-model (VLM) sub-class, by contrast, produces text output and is used as a standalone perception-and-description layer that returns answers about a scene rather than executable actions. Liu et al. [39] introduced LLaVA-1.5, an open-source instruction-tuned VLM that pairs a CLIP-ViT-L visual encoder with a Vicuna language backbone and answers natural-language questions about images, reporting state-of-the-art performance across eleven visual-question-answering benchmarks. Chen et al. [40] reported the empirical limit of this model class, showing that LLaVA-class VLMs hallucinate more on multi-object queries and that hallucination rates rise as the number of objects in the query grows, which constrains the use of a single VLM to single-object questions. Across both sub-classes the scene representation is implicit, namely a stream of visual tokens with no spatial scaffold, no affordance ontology, and no textual description that an external reasoner could inspect.

Within the second line, language is the interchange format between the perception network and the reasoning network. Ahn et al. [41] factored robot planning into a language-model proposer and a learned affordance scorer, with the affordance acting as the feasibility filter that determines which actions are available to the planner at any given state. Huang et al. [42] closed the loop with Inner Monologue by routing environment feedback to the language model as text, demonstrating that language is the interchange format between perception and reasoning. Liu et al. [43] adopted a systems-first approach to integrating open-knowledge models for robotics, reporting that the critical role lies in the nuanced details of combining open-knowledge systems such as vision-language models with robotic modules rather than in additional model training, with the best-tested deployment reaching fifty-eight point five percent open-ended pick-and-drop success and approximately one point eight times the prior state of the art on the same task.

The perception side of the second line has been shaped by recent advances in open-vocabulary object detection, depth sensing, and multi-object tracking. Perception for autonomous-robot deployments was historically built on detectors trained against a fixed inventory of object classes, with the inventory authored at design time and the detector retrained whenever the operating environment introduced classes outside it [44]. The transition to open-vocabulary detection, in which a single detector is conditioned on text prompts at inference time and recognizes classes the engineer never enumerated, has reshaped what the perception layer can deliver to a downstream language-model consumer whose own vocabulary is virtually unbounded. Cheng et al. [45] introduced YOLO-World, which ports open-vocabulary detection into the YOLO speed regime and reports thirty-five point four AP at fifty-two frames per second on a V100 GPU on the LVIS benchmark, narrowing the gap between vocabulary breadth and inference speed that earlier open-set detectors built on transformer backbones had imposed. YOLO-E by Wang et al. [46] unified text-prompted, visual-prompted, and prompt-free open-vocabulary detection-and-segmentation in one network, reporting on the LVIS benchmark a three point five AP gain over the YOLO-Worldv2 small variant at one point four times the inference speed and one third the training cost of the same baseline.

Rustler, Volprecht, and Hoffmann [47] compared four consumer stereoscopic depth-sensing cameras on planar surfaces and household objects and reported sub-centimetre absolute depth error

below one meter for the Intel RealSense D435 and D455, with the D455 sustaining absolute bias under two centimetres at two meters. Zhang et al. [48] introduced ByteTrack, in which low-confidence detections that single-shot detectors typically discard are associated to existing tracklets through their geometry alone, recovering objects that a stride detector intermittently misses. Aharon, Orfaig, and Bobrovsky [49] extended the same line with BoT-SORT, adding camera-motion compensation through an estimated affine transform on predicted bounding boxes, an enlarged Kalman filter state vector, and an IoU-and-ReID association rule.

Pech-Pacheco et al. [50] established the variance of the Laplacian as a practical sharpness measure through a comparative study on diatom microscopy, and the same statistic remains the standard blur metric in image-processing pipelines. Korolkov [51] composed Laplacian-variance sharpness and LAB mean-luminance brightness, weighted zero point seven and zero point three after z-score normalization, into a single training-free score and used it to select one representative keyframe per scene. Liang et al. [52] reported with KeyVideoLLM approximately sixty-fold compression of frame counts feeding a downstream language model with negligible accuracy loss on video question answering. Kim et al. [53] arranged sampled frames into a single image grid and provided the grid as one image to a vision-language model, surpassing purpose-built video language models on nine of ten benchmarks and demonstrating that explicit temporal modules can be replaced by frame selection arranged into a spatial grid for a single multi-image inference. Scene graphs are the intermediate representation through which spatial perception meets symbolic reasoning. A small number of well-chosen frames per scene is sufficient for a language-model consumer that does not require an end-to-end action policy.

Armeni et al. [54] introduced the hierarchical 3D scene graph spanning building, room, object, and camera as nodes connected by spatial and containment relations as edges, and demonstrated that the same representation supported semantic, geometric, and topological queries. Rosinol et al. [55] introduced Kimera, an open-source visual-inertial SLAM library producing a globally consistent pose graph alongside a real-time metric-semantic mesh in which each face carries a fused semantic label, and demonstrated real-time operation on the EuRoC dataset. Rosinol et al. [56] extended Kimera into the first 3D Dynamic Scene Graph constructed online from a single visual-inertial stream, with five layers spanning the metric-semantic mesh, objects and agents, places and structures, rooms, and the building. Hughes, Chang, and Carlone [57] re-engineered the same hierarchy as Hydra, the first system to optimize such a graph online in response to loop closures. The scene-graph paradigm has been extended to open-vocabulary, foundation-model-driven settings. Gu et al. [58] introduced ConceptGraphs, which consumes a posed RGB-D stream and turns each frame into class-agnostic SAM segments encoded with CLIP features. Multi-view association merges per-frame segments into persistent 3D nodes by summing nearest-neighbour geometric overlap and CLIP cosine similarity, and a two-stage captioner runs LLaVA on each view before GPT-4 summarizes the per-view captions into a final node label. Inter-object edges are first proposed by 3D-bounding-box overlap pruned to a minimum spanning tree and then labeled by GPT-4, yielding an open-vocabulary 3D scene graph that answers natural-language object queries, multi-modal text-plus-image queries, and traversability requests for downstream planning. Spatial scaffolding compresses object position into a discrete vocabulary that a language model can express in text.

Wang et al. [59] introduced VisionGPT for assistive visual navigation, in which an H-pattern grid divides the camera frame into a wide central column split horizontally into ground and frontal zones together with two vertical strips representing the user's flanks. The rationale of the H-pattern is that a language model requires a fixed, discretised representation of object positions relative to the body rather than a continuous geometric description that the model would have to reason about in metric space. Chen et al. [60] and Cheng et al. [61] provided the empirical evidence behind that rationale, with SpatialVLM constructing a synthetic 3D-grounded spatial-VQA dataset and reporting that vision-language models without explicit spatial scaffolding produced invalid quantitative answers and lower accuracy on metric-distance and metric-size questions even on scenes the same models otherwise described correctly. Yang et al. [62] take the same principle further with Set-of-Mark prompting, in which off-the-shelf segmentation produces image regions that are stamped with alphanumeric marks so that the vision-language model grounds objects by emitting the mark identifiers rather than continuous coordinates, and report that GPT-4V's referring-expression comprehension on RefCOCOg rises from twenty-five point seven to eighty-six point

four when given marks in place of pixel coordinates. Nasiriany et al. [63] extend the substitution from grounding to actionable decision-making with PIVOT, annotating each image with a finite set of numbered candidate actions, localisations, or trajectories that the vision-language model selects from rather than emitting continuous outputs, with the candidate set iteratively refined across passes until a precise choice is reached, and demonstrate zero-shot robotic navigation and localisation without finetuning.

2.4 Knowledge-Augmented LLM Reasoning for Safety

The investigation of language models as candidate engines for safety-engineering tasks has motivated a parallel investigation of methods to ground their output in external sources of domain knowledge. This subsection reviews these two lines of work and the safety-oriented systems that combine them.

Large language models have been examined as candidate engines for tasks that previously demanded a human expert, with safety engineering among the domains in which their suitability has been investigated. Diemert and Weber [64] tested whether GPT-3, accessed through ChatGPT, could assist in hazard analysis on a simple water heater system and reported that the model produced analyses of moderate usefulness, with the role of the language model framed as supporting rather than replacing the human analyst. Qi et al. [65] applied ChatGPT to STPA on an Automatic Emergency Brake system and on Electricity Demand Side Management, and reported that without human intervention the model was unreliable, that varying the input semantic complexity or applying domain-agnostic prompt guidelines produced no statistically significant differences, and that with careful design the analyses could nonetheless outperform human experts on specific subtasks. Comparable evaluations have been carried out for FMEA and HAZOP. El Hassani et al. [66] integrate large language models into the FMEA workflow with a human in the loop, combining data collection, pre-processing, and reliability assessment into a framework that they evaluate on an industrial automotive case study built on roughly two hundred and twenty-seven thousand customer reviews and report ninety-one percent substantial agreement with two domain experts together with ninety-eight to ninety-nine percent part-finder accuracy and ninety-seven percent sentiment accuracy. Lee et al. [67] address the harder question of whether the language model can complete a HAZOP worksheet without human intervention, evaluating GPT-4o, GPT-4o-mini, LLAMA 3.2, and Gemini 2.0 against a reference worksheet and reporting textual similarity F1 above eighty-six percent for all four models alongside a much weaker semantic-validity rate of nineteen to thirty-seven percent on the scenarios generated, with a heavy bias toward procedural rather than engineered safeguards.

Lewis et al. [68] introduced retrieval-augmented generation as a hybrid of parametric and non-parametric memory, in which a sequence-to-sequence transformer is conditioned on passages retrieved by a neural retriever from an external index. The framework was motivated by the observation that knowledge-intensive tasks expose the limited ability of parametric models to access and manipulate stored facts, and the original paper flagged providing provenance and updating world knowledge as open problems. Both properties are precisely the ones that matter when the consumer of the generated text is a safety case rather than a casual reader. Ovadia et al. [69] compared retrieval against fine-tuning directly across a range of knowledge-intensive tasks and reported that retrieval consistently outperforms fine-tuning, both for knowledge encountered during pre-training and for entirely new knowledge. The same finding has been reported in the safety-critical setting itself. Iyenghar [70] compared an LLM-only baseline against a retrieval-augmented configuration on a dataset of 7800 hazard scenarios constructed from ISO 12100 and ISO 13849-1, and reported that retrieval raised the accuracy of hazard classification and required Performance Level prediction from 36.4 to 72.8 percent.

Retrieval alone does not eliminate hallucination in high-stakes domains. Magesh et al. [71] conducted the first preregistered evaluation of three commercial retrieval-augmented legal research products and reported that the systems hallucinated between seventeen and thirty-three percent of the time, despite vendor claims of hallucination-free output. Cuconasu et al. [72] characterize this effect at the retrieval level, showing that the highest-scoring passages which are topically re-

lated but do not contain the answer are the most damaging distractors, while genuinely off-topic random passages can be tolerated or even improve accuracy, indicating that the generator is most easily misled by content that resembles the correct evidence.

Pan et al. [73] survey the unification of large language models and knowledge graphs and argue that the two are complementary, with language models offering generalizability and broad language processing but acting as black boxes that suffer from hallucination and lack factual grounding, and knowledge graphs explicitly storing structured factual knowledge while remaining incomplete and difficult to construct. Xia, Jazdi, and Weyrich [74] take this step for safety analysis and propose an LLM multi-agent system that interfaces with an FMEA knowledge database through retrieval-augmented generation. A task classifier labels the active spreadsheet cell as a failure-effect, failure-cause, or control-mechanism request, an embedding LLM retrieves the most semantically similar entries from the database, and an analysis agent followed by a synthesis agent produces a ranked list of candidate completions.

Bahr et al. [75] begin from a set-theoretic standardisation of FMEA in which the analysis is captured by a set of failure modes, mappings from each failure mode to its effects, causes, and measures, an assignment of each failure mode to a process step, and ordinal severity, occurrence, and detection ratings whose product is the risk priority number. From this they derive a graph schema in which FailureMode, FailureEffect, FailureCause, FailureMeasure, and ProcessStep nodes are linked by typed relations such as resultsInFailureEffect, isDueToFailureCause, isImprovedByFailureMeasure, and occursAtProcessStep. A depth-first traversal serializes the subgraph rooted at each FailureMode into a single text chunk, the chunk is embedded, and a VectorEmbedding node is linked back to the FailureMode it summarizes. Here, at runtime the user inquiry is first converted to a Cypher query and evaluated symbolically against the graph, and only when the symbolic match fails does the system fall back to vector search over the embedded subgraphs, with the cosine distance returned alongside the result so that the operator has an estimate of the epistemic distance between the query and the nearest stored entry. The authors report a context-precision improvement on FMEA queries from 0.29 in a baseline vector RAG to 0.82 once the graph query, vector fallback, and DFS-based chunking are combined, and complement this measurement with a user study showing gains in correctness, usability, and relevance over an Excel baseline.

Knowledge graphs cover the typed entities and relations a domain has coded, but much safety-relevant knowledge resides in unstructured normative material, namely incident reports from public failure databases, regulatory bulletins, and academic literature on accident analysis, which is typically distributed as PDF. The Docling pipeline of Auer et al. [76] provides a unified framework to recover text, tables, and reading order from each document into a structured representation suitable for downstream chunking.

Once the document has been recovered into this structured representation, fidelity is lost at the chunking step, where the stream is partitioned into the units that will be embedded and indexed. The common practice is fixed-size token splitting with overlap, which fragments context whenever a definition or framing introduced earlier in the document does not enter the chunk that will later be retrieved. Günther et al. [77] propose late chunking, in which a long-context embedding model is run once over the whole document and each chunk's vector is formed as the mean of the token representations within the chunk's span, so that the chunk inherits context from the rest of the document through the model's attention before the pooling boundary is drawn. Merola and Singh [78] compare late chunking against contextual retrieval, the alternative strategy that prepends an LLM-generated, chunk-specific summary to each chunk before embedding, and report a trade-off, with contextual retrieval preserving semantic coherence more effectively at the cost of LLM calls during indexing, while late chunking offers higher efficiency at the cost of relevance and completeness.

A retrieval-augmented system must also decide when retrieval is needed at all. Always retrieving wastes inference cost and exposes the generator to irrelevant context, while never retrieving forfeits the grounding that motivates the architecture. Jeong et al. [79] train a small classifier to predict the complexity of an incoming query and route it among no retrieval, single-step retrieval, and multi-step retrieval accordingly, reporting that this routing matches always-iterative baselines at substantially lower cost on open-domain question-answering. Singh et al. [80] survey the

broader class of agentic retrieval-augmented systems, in which an autonomous agent decides at inference time which retrieval modality to invoke, reflects on the result, and refines its action, and propose a taxonomy of these systems organized by architecture, including single-agent, multi-agent, hierarchical, corrective, adaptive, graph-based, and document-workflow frameworks. Within this taxonomy, the router architecture, also called single-agent agentic RAG, centralises retrieval decisions in one agent that receives the query, analyses it, and selects among candidate knowledge sources such as a structured database, a semantic vector index, or a web search, before passing the retrieved content to a language model for synthesis. The corrective architecture decomposes the workflow into five specialised agents, namely a context-retrieval agent that fetches documents from a vector store, a relevance-evaluation agent that classifies the retrieved documents as relevant, ambiguous, or irrelevant, a query-refinement agent that rewrites the query when the result is judged insufficient, an external-knowledge-retrieval agent that fetches additional documents from an alternative source, and a response-synthesis agent that produces the final answer, with the relevance-evaluation step gating whether the corrective fetch is invoked. The Agent-G architecture, introduced for graph-augmented retrieval, employs a retriever bank that pairs a graph knowledge base with an unstructured document store under one agent, with a critic module that validates the relevance and quality of the retrieved content and flags low-confidence results for re-retrieval before the language model generates a response.

Recent work has begun to apply this combination of language-model reasoning and retrieved domain knowledge to safety problems. Alenjareghi et al. [81] construct a knowledge graph from ISO/TS 15066 comprising 63 Title nodes, 3,050 Content nodes, and 11,155 Index nodes connected by 110,133 edges, and use it as the retrieval source for an LLM that performs failure mode and effects analysis on disassembly cells. Mahmoud et al. [82] extend the combination to compliant manipulation, processing egocentric frames from a Unitree G1 humanoid through a Molmo-7B vision-language model whose JSON output is embedded and matched against a curated database of validated impedance and velocity profiles, with each retrieved profile checked against ISO/TS 15066 and ISO 13855 before deployment. Sotomi et al. [83] pair a vision-language model with a language model on a mobile robot to detect environmental anomalies and report hazardous situations, and report 91.2 percent accuracy in a thirty-user study on an edge AI platform. Naderi et al. [84] extend the same combination to construction-site inspection, with a SLAM-driven mobile robot whose vision-language model produces scene descriptions, a retriever grounds them in OSHA and site policies, a second vision-language model assesses safety, and a language model writes the report. Zhang et al. [85] propose SafeEmbodAI, a safety framework for mobile robots in embodied AI systems with secure prompting, state management, and safety validation modules, and report a significant improvement over baseline in mixed-obstacle attack scenarios. Sinha et al. [86] place this pattern on a firmer engineering footing, decomposing the runtime monitor into a fast binary anomaly classifier on language-model embeddings followed by a slower generative-LLM fallback wrapped in a model predictive controller that maintains feasibility while the slow stage is reasoning, and demonstrate the design on quadrotors and autonomous vehicles.

2.5 Edge Deployment of Language and Vision-Language Models

This subsection reviews the literature on edge deployment of language and vision-language models, the compression techniques and benchmarks proposed for it, and the multimodal model families targeted at embedded use.

Each token generated during decode requires loading every weight of the model from off-chip memory back to the compute units before the next token can be produced, so the per-token time cost is determined by how fast the device can move bytes from memory to the compute path rather than by how fast it can multiply them. Three measurements together establish this bound for transformer inference. Pope et al. [87] partitioned transformer inference into a prefill phase, in which the input tokens are processed in parallel, and a decode phase, in which tokens are generated sequentially under memory pressure from weight and key-value-cache loading, and derived the parallelism strategies that minimize per-token latency on TPU v4 slices. Yuan et al. [88] sharpened the picture with a layer-by-layer roofline tool that locates each operator on the bandwidth-versus-compute plane, reporting that prefill operators are compute-bound while

decode operators sit on the memory-bound side of the roofline. Recasens et al. [89] closed the batching loophole by measuring inference on the NVIDIA H100, a contemporary data-center GPU, at large batch sizes and reporting that memory bandwidth remains the bottleneck even when many requests are batched together, against the standing expectation that batching would amortise weight loading across the batch. On the Jetson AGX Orin, the platform documentation [90] reports a theoretical peak memory bandwidth of two hundred and four point eight gigabytes per second across the LPDDR5 pool shared between the GPU, the CPU complex, and the deep-learning accelerators, and the ceiling that figure sets binds the per-token decode rate in a way no kernel optimization lifts.

Compression of the model is one response to this bandwidth ceiling. Zheng et al. [91] surveyed the techniques that fit language models onto edge devices and organized them into three categories, namely offline pre-deployment model design through quantization, pruning, and knowledge distillation, online runtime inference optimizations that exploit hardware-specific kernels and memory-management strategies, and on-device applications. Their central observation is that the computational complexity of language models has grown rapidly while edge-device AI capability has advanced more modestly, and the gap between the two is widening rather than closing. They report that effective on-device deployment depends on combining multiple compression techniques rather than relying on any one in isolation, with weight-only quantization paired with structured pruning suited to scenarios that face strict memory constraints under moderate compute, and weight-activation co-quantization paired with aggressive unstructured pruning where maximum compression is required even at the cost of some accuracy.

Four-bit weight-only post-training quantization became viable at hundred-billion-parameter scale through GPTQ [92], in which Frantar et al. fitted each layer's four-bit weights through a single calibration pass that minimizes that layer's output error rather than retraining the model, and reported language-modeling quality on the OPT-175B checkpoint within one perplexity point of the half-precision baseline. Lin et al. [93] refined the four-bit regime with AWQ on the observation that a small fraction of weight channels carry a disproportionate share of the model's predictive signal, identified through activation statistics rather than weight magnitudes, and demonstrated that scaling those salient channels prior to a uniform low-bit quantization preserves accuracy more faithfully than naive uniform quantization while keeping all weights in the same low-bit format. Zandieh et al. [94] extended quantization beyond model weights to the key-value cache that decode reads and writes during generation, randomly rotating cache vectors so that per-coordinate scalar quantizers reach near-optimal distortion and absorbing the residual through a one-bit Quantised Johnson-Lindenstrauss transform, and reported in TurboQuant a more than five-fold reduction in key-value cache footprint on long-context workloads at three-and-a-half bits per channel without retraining. Each scheme moves the bandwidth-bound roofline down by reducing bytes loaded per token rather than lifting the bandwidth ceiling itself.

Arya and Simmhan [95] benchmarked language-model inference on the NVIDIA Jetson AGX Orin 64GB Developer Kit across four models that span a substantial parameter range, namely Microsoft Phi-2 at 2.7B parameters, Meta Llama 3.1 at 8B, Mistral-Small at 23.6B, and DeepSeek-R1-Qwen at 32.8B, all served through PyTorch and the Hugging Face Transformers library. Their throughput metric is the aggregate token throughput across a batch, defined as the sum of input and output tokens across all prompts in the batch divided by the batch latency, and is therefore not directly comparable to the per-stream decode rate a single user observes. They report Llama 3.1 aggregate throughput rising from one hundred and eighty-four to five hundred and fifty-eight tokens per second as the batch size grows from thirty-two to one hundred and twenty-eight at a sequence length of ninety-six in FP16, with the same metric collapsing from two hundred and seventy-one to one hundred and seven tokens per second when the sequence length is increased from one hundred and twenty-eight to one thousand and twenty-four. Power consumption tracks toward the device's sixty-watt peak envelope at the largest batches under the MAXN power profile, with per-query energy ranging from sub-kilojoule levels for the smaller models to around ten kilojoules for the largest configuration. The configuration-level finding most directly relevant to a small-language-model deployment is that INT8 quantization reduces power consumption by up to forty-four percent relative to FP16 on Llama 3.1-8B and by roughly fifty percent on Mistral-Small, while FP16 retains the lowest total energy when integrated across the full batch. Its overall conclusion is that the Jetson AGX Orin is viable for language-model serving, with the residual challenges

sitting in latency and energy usage rather than in model capability.

These constraints also apply to vision-language models. Chu et al. [96] delivered MobileVLM V2 with one-point-four-billion and two-point-seven-billion-parameter language backbones as a mobile vision-language model targeted at resource-constrained devices, demonstrating that the open-vocabulary visual-question-answering capability of larger VLMs could be reproduced at a parameter count compatible with embedded inference.

The vision-language-action family compresses perception and action into a single forward pass. Brohan et al. [97] established the framing with RT-2 on the PaLM-E twelve-billion and PaLI-X fifty-five-billion backbones, with both variants running on cloud TPU pods rather than on the robot itself. Vishwanathan, Subramanian, and Raghunathan [98] profiled the seven-billion-parameter MolmoAct VLA on the Jetson AGX Orin and the next-generation Jetson Thor and reported that the autoregressive action-generation phase consumed roughly seventy-five percent of end-to-end latency, that the resulting per-step latency on both platforms ran two to three hundred times the ten-hertz target needed for dexterous manipulation, and that Thor's five-fold compute advantage over Orin yielded only a one-point-four-fold reduction in end-to-end latency, the signature of a memory-bandwidth-bound workload.

Ahn et al. [41] factored robot planning into a language-model proposer and a learned affordance scorer in SayCan, with the affordance acting as the ontology that determines which actions are physically available to the planner at each state, and reported an eighty-four percent planning success rate and seventy-four percent execution success on a mobile manipulator in their mock-kitchen evaluation. Liang et al. [99] demonstrated with Code as Policies that a structured perception API together with a small set of control primitives is sufficient input for a language model to emit executable Python code that the robot then runs, removing the need for additional task-specific policy training on the tasks evaluated. Huang et al. [42] introduced earlier, closed the loop with Inner Monologue by routing environment feedback (success and failure signals together with scene descriptions) to the language model as text after each action, demonstrating that long-horizon task completion improves through iterative replanning rather than retraining. Chen, Han, and Li [100] took the same architecture to the edge with FASTNav, LoRA-fine-tuning a small language model for multi-point navigation on a robot deployed locally, demonstrating that the structured-intermediate pattern transfers to an NVIDIA Jetson Orin NX edge device.

3 Concept

When a person encounters a new situation, they rarely think about it from first principles. Recollection follows perception almost immediately. Associating the current situation with similar past experiences brings expectations about what might happen and what response suits the moment. If the expectations fit, the response can proceed with little effort. If not, the gap must be closed. The person can act, get it wrong, and learn. Alternatively, the person can consult a trusted source, absorb the advice, and proceed accordingly. Either way, the internal model becomes better prepared for next time.

Cognitive psychology describes the underlying mechanism as schema-driven prediction. Ortiz-Tudela et al. [101] state the position directly. Anticipating future states relies largely on memory, with mnemonic traces serving as the building blocks of prediction. The same study shows that the residual mismatch between expectation and outcome, the prediction error, is what drives the encoding of new information; the failures of the experiential route are therefore not wasted but are themselves the mechanism by which the internal model is brought up to date. Past experience furnishes the priors on which prediction depends, and prediction in turn shapes what the present moment commits back to memory.

The cycle described above, has a structural counterpart in safety engineering for autonomous robots, where risk is increasingly treated as a property of the operational context rather than solely as a property of its components. Muller et al. [3] and Wolf et al. [4] argue that the risk of an autonomous mobile robot is no longer predominantly defned by component reliability but by the scenario it is in and how well the system is prepared to handle it.

The framework presented in this thesis is designed around that same loop. Perception delivers a structured account of the scene, and a library of prior failure analyses serves as the internalized experience that a language model, acting as the agent of recall, associates with the present situation. Where the association is confident, the system concludes directly. Where it is not, the system consults a secondary store of normative material to deepen its account of the consequences, what the situation could lead to. Figure 1 sketches the framework at a high level.

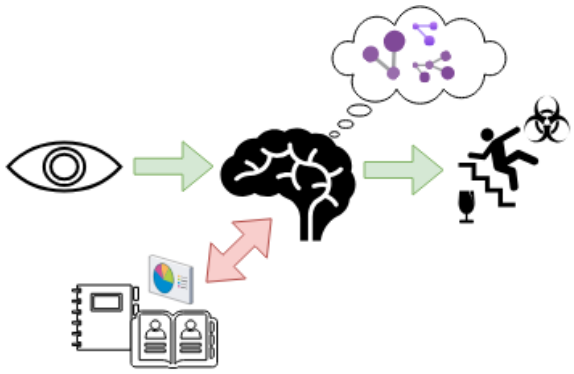


Figure 1: Concept overview of the framework.

3.1 Perception

The perception layer is responsible for producing a structured account of the scene. A scene must report which objects are present, where they are relative to the robot, and how they are moving. Understanding the scene in these terms is a prerequisite for anticipating what could go wrong.

The first attempt put a single vision-language model on the camera stream directly. It produced coherent high-level descriptions but could not reliably follow lengthy, specific prompts.

IG-VLM [53] showed that a few well-chosen frames are enough for a vision-language model to make sense of a video, with the frames arranged into a single image grid in place of explicit

temporal modeling, as in Figure 2. However, when tested, IG-VLM's uniform sampling did not pick the frames that carried the risk in the scene, a limitation that needed to be addressed. Other keyframe selection strategies were examined. The first to be tried was a change score against recent neighbours, which picked frames that carried new content rather than every Nth frame. Change alone admitted motion-blurred frames, since blur also registers as change, so a quality score combining sharpness [50] and luminance [51] was added as a second gate, and a frame qualified as a keyframe only when it cleared both. Temporal ordering was not required here for the framework as perception receives a stream of frames continuously, mimicking live perception, so the grid itself was not used and the keyframes were passed singly to the next step. The compute saving from running detection on a curated handful of frames rather than the full stream is the perception layer's instance of the keyframe compression for downstream language-model consumption that Liang et al. [52] report with KeyVideoLLM.

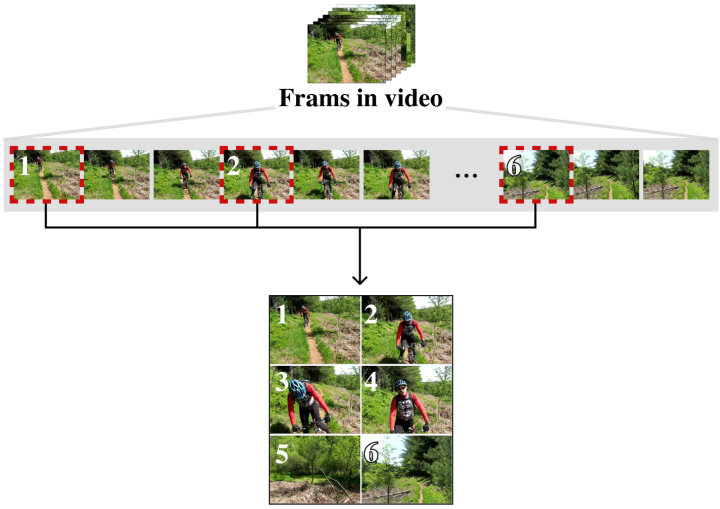


Figure 2: Image-grid representation from IG-VLM.

With keyframes in place, the vision-language model was given clearer, sparser inputs. At this point its granularity limits became visible. The model could not pick out every potential hazardous object in the scene, an outcome consistent with the multi-object hallucination limit Chen et al. [40] report for LLaVA-class models [39]. The framework therefore moved off vision-language perception entirely and onto a detect-then-reason architecture in which an object detector hands its output to the cognition layer through natural language, following the structural cues from SayCan [41] and Inner Monologue [42].

YOLO-E [46] was chosen for three reasons, namely prompt-free and promptable variants in the same checkpoint, a deployment vocabulary that can be revised through re-export, and a TensorRT engine that can be exported for optimised runs on CUDA cores. The keyframe idea was carried back into detection. The detector runs at a fixed stride over the buffered frames rather than on every frame, so the compute saved by keyframe selection is realised at the detection step. The tracker BoT-SORT [49] associates detections across the stride and compensates for the camera's own motion, which is what allows identities to persist on a moving platform. The chain that began by removing perception from the language model ends by giving the cognition layer something the vision-language model could not have supplied on its own, namely persistent object identity through the window, a fused motion state, and an exact numeric velocity estimate per tracked object.

At the close of each window, the keyframe detections are fused into one record, and each detection is tagged with the egocentric zone it falls in, drawn from the four-zone H-grid introduced in VisionGPT [59]. Raw coordinates would require the language model to deduce each object's location from numbers. Computing the zone in a deterministic step therefore replaces the numeric

coordinates with a single zone tag, removes the spatial reasoning from the language model, and saves both tokens and inference time. The record together with the class-level priors then transformed into a lightweight scene graph for the window, in the sense of Armeni et al. [54] and Pan et al. [73] on the complementarity of structured graphs with language-model reasoning, but deliberately lighter than the dense multi-frame scene graph lineage of Rosinol et al. [55] and Gu et al. [58].

The priors are written at deployment time, one entry per class in the detection vocabulary, carrying the hazard categories the class is associated with, the alternate names it may be referred to by, and the classes it typically appears alongside in the operating environment. The hand-authored approach is inspired by Liu et al. [43], who report that deploying open-knowledge models succeeds through integration detail rather than further training.

What detection produces is broad by design. Only a subset of that breadth is relevant to the risk the robot faces at a given moment, and the framework treats relevance as a property of the object's spatial relation to the robot rather than solely its class identity. Objects in the robot's path of travel matter more than those on its flanks, and objects on its flanks matter more than those far ahead of it, and the H-grid carries this priority through to the scene reported to cognition. The geometry of the partition depends on the platform, since a camera at a quadruped's height does not see the ground the same way a chest-height hand-held camera does, and what counts as the ground zone or the front zone is therefore a deployment parameter rather than a fixed property of the framework.

3.2 Cognition

The cognition layer consumes the scene graph perception emits and produces FMECA risk entries for the scene. It tries first to ground each entry in the deployment's FMEA knowledge graph, falling back on a wider body of normative material only when the graph's precedent does not cover the hazards present.

The cognition layer was kept text-only. A multimodal model consuming pixels alongside its prompt would have folded perception and speculation back together at the cost of auditing the framework's conclusions stage by stage. The first attempt was a small text-only language model reading the structured detections from perception and returning a short description of the scene. The descriptions came back coherent but ungrounded. The model had no anchor against which to claim what could go wrong.

Looking for a way to anchor the descriptions, retrieval-augmented generation surfaced as the natural path. Retrieval was preferred over fine-tuning as literature suggested, it is the stronger path on factoid recall and on the safety-critical regime [69, 70]. The natural source of priors was its own design-time artifacts, items that define ODD, and HARA entries. Items where the designers would have thought about things that could go wrong during operation serves as initial memory for the framework. Bahr et al. [75] had show how to lay an FMEA out as a knowledge graph and embed each failure-mode subgraph for retrieval. The framework took their schema, depth-first-traversal embedding scheme. A scene description that the language model provided served as query for vector similarity search. It was observed that the retrieved entries did not always cover the hazards the scene presented. The language model would fill the gap with text that was not grounded in any retrieved evidence.

The choice was either to enrich the FMEA further, or to bring in other, broader sources of grounding. The framework took the second path. Incident reports from public databases, standards on robot safety, and academic literature on robot-related injuries and human-robot collaboration risk were ingested as a secondary store of normative material alongside the FMEA knowledge graph. The PDFs were brought in through the Docling pipeline of Auer et al. [76] for layout-aware extraction, and chunked under the late-chunking strategy of Günther et al. [77] to ensure the context is not lost during chunking.

The cognition layer thus reads against two stores, the structured FMEA knowledge graph and the

unstructured normative store, an arrangement consistent with Pan et al. [73] on the complementarity of graphs and language models for grounded reasoning. The dual-store pattern with a critic between the stores and generation was inspired by Agent-G as surveyed by Singh [80], shown in Figure 3.

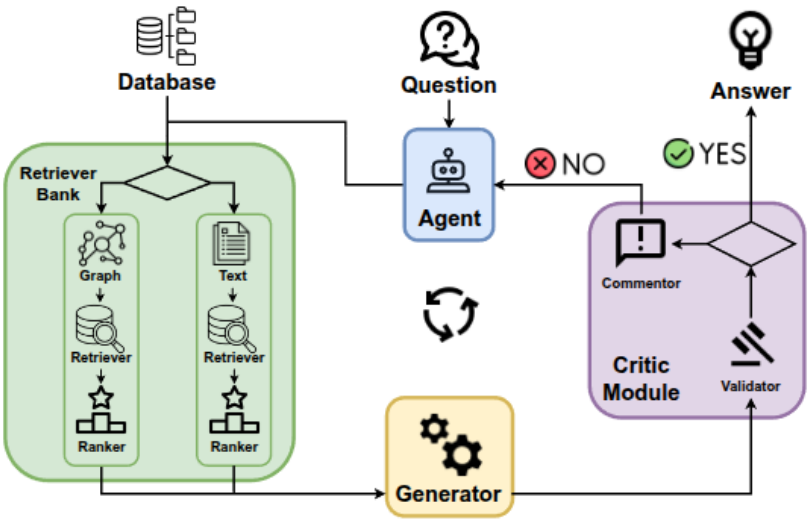


Figure 3: Overview of Agent-G from Singh et al. [80].

The routing between the two stores is fixed. The FMEA knowledge graph is consulted first. When its precedent covers the hazards the scene presents, the entry is generated against it on the sufficient path. When a class-coverage check finds that the retrieved entries leave threat classes uncovered, the scene falls through to the normative store on the novel path. An earlier version of the framework asked the language model to make this routing call, but the call added latency on every scene. Sinha et al. [86] had demonstrated that a fast deterministic check can stand in for the language model on most cases, with the language model reserved as a slower fallback for the cases the fast check flags, and the framework adopted the same pattern by placing the cheap class-coverage gate ahead of the model.

The generation stage writes a structured FMECA entry for the scene, anchored in the specific objects perception reported so the entry stays grounded in what was observed. Structured output is preferred over free prose because small language models are unreliable on free-form safety analyses [65, 67]. The framework follows El Hassani et al. [66] and Sotomi et al. [83] in pairing the model's authorship with a human-in-loop review. A fixed template was used to structure the prompts and evaluation. The severity and probability scales the model rates against are adapted from MIL-STD-882E and reframed for runtime operation.

A criticality category is then derived from the rated severity and probability. The framework uses a criticality matrix and FMECA rather than the FMEA, Risk Priority Number, since the RPN, defined as the product of severity, occurrence, and detection, has the well-documented weakness that different combinations of the three factors produce the same number and leave failure modes of very different operational character indistinguishable in the ranking.

Entries generated by speculating over the normative store are written back into the knowledge graph and held for human review before they enter retrieval. The writeback targets the graph rather than the normative store because the FMEA path returns faster than the normative path, and growing the graph over deployment time reduces the share of future scenes that need the fall-through. Without the review, the model would associate against its own past output without an intermediate gate, reintroducing the hallucination retrieval was adopted to bound. The placement of the review is consistent with the relocation of design-time responsibility to runtime-monitor assurance described by Reich and Trapp [5].

4 Methodology

This section gives the implementation details of the proposed framework. The framework has two stages, namely SENSE which implements the perception layer and SPECULATE which implements the cognition layer, and the description that follows traces the path of a single scene through both. Figure 4 sketches the framework end to end.

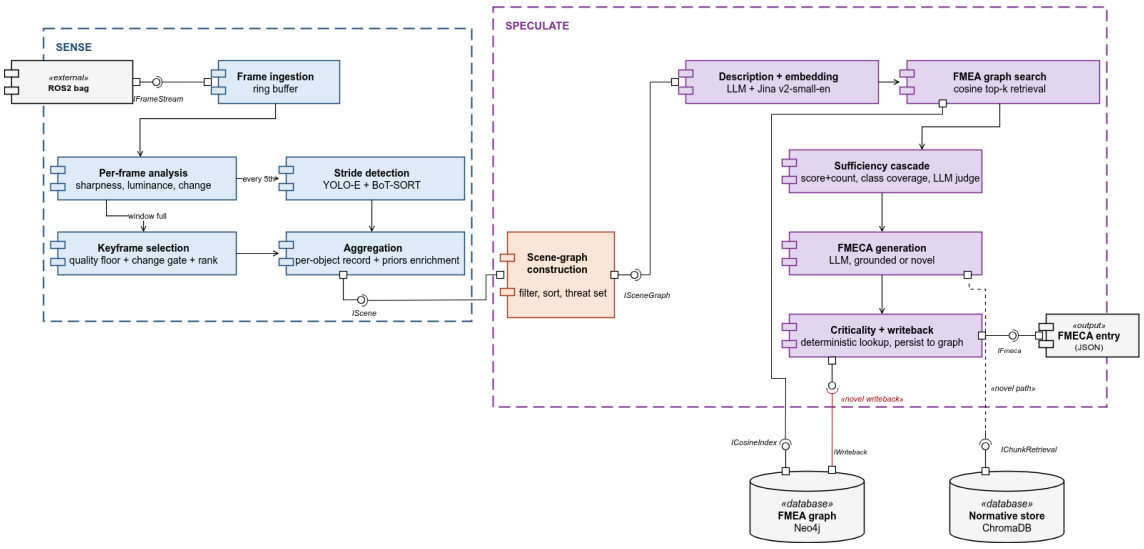


Figure 4: Framework overview

SENSE reads frames from a ROS bag and scores each one for sharpness, brightness, and how much it changes against its neighbours, and on every fifth frame also runs an object detector and tracker so that motion histories build up across the window. When 30 frames are accumulated (a scene), SENSE picks the frames oboove a quality and change threshold as key frames, aggregates the detections on them into a per-object record with a class-level priors of risk tags and retrieval hints, and emits the result as a scene summary. SPECULATE consumes that summary as a scene graph and looks up precedent in a FMEA knowledge graph. When the retrieved entries cover the hazards in the scene, it grounds its FMECA risk entries on those, along the sufficient path. When they do not, it resorts to a store of incident reports and academic literature along the novel path, and writes the resulting entries back to the knowledge graph as candidate precedent for later scenes. The two subsections below trace one scene end to end, first through SENSE and then through SPECULATE.

The FMEA registry, the design-time enumeration of the failures the robot can undergo across its mission, runs as a knowledge graph on a local Neo4j graph database instance, with each curated FMEA row stored as a FailureMode node linked to its ProcessStep, FailureEffect, FailureCause, and FailureMeasure neighbours. A vector index over a companion VectorEmbedding node holds the 512-dimensional Jina v2-small-en embedding for the text of each FailureMode, so retrieval at runtime is a single Cypher call that returns the top three FailureMode rows by cosine similarity together with the full neighbor set for each match. On the novel path, the same embedding is used to query the normative document store running on chroma DB, also deployed locally. Write-back adds a subgraph per generated entry with a status flag, so that generated rows stay distinguishable from curated ones and are gated for review before they enter the curated set.

The normative store covers three kinds of source that the curated graph does not. Published standards on robotics and laboratory safety, academic literature on hazard analysis methods and prior FMEA studies, and incident reports collected from public databases and synthetically generated ones. The PDFs are processed offline by an ETL pipeline and loaded, ready for querying.

4.1 Sense

The perception layer of the framework is realised in the SENSE stage of the pipeline. A scene is the unit of input on which SENSE operates. It corresponds to a fixed-size window of frames captured by the robot's onboard camera over a short interval of operation. The SENSE stage processes each window from end to end and emits a single structured json summarizing what the robot observed during the window, which downstream reasoning consumes one scene at a time.

SENSE consumes a ROS2 bag recorded from an Intel RealSense camera mounted on the robot. The bag interleaves five timestamped streams: color images, depth images already aligned to the color frame by the camera driver at acquisition, the dynamic and static transform trees, and odometry. Topic names and message types are given in Appendix A. Rather than performing explicit message synchronization, SENSE treats each color image as the temporal anchor and pairs it with the most recently received depth, transform, and odometry message. This is sufficient because the camera driver aligns and timestamps depth against color at acquisition, so the paired depth is at most one frame behind the color image; the resulting offset is small and bounded, and accepting it keeps ingestion simple and free of stalls. From each color message SENSE assembles a Frame record bundling the color image, the paired depth and odometry, and the accumulated transform tree. Playback is paced to the wall-clock interval between successive color timestamps so that the pipeline observes frames at their recorded rate.

Each Frame is then written into a fixed-size ring buffer of capacity 30 that holds the current scene window, which is about 2 seconds, with camera running at 15Hz. Every frame is scored for legibility by two measures computed on a grayscale, downscaled copy of the frame, namely the variance of its Laplacian as a sharpness measure and the mean of its pixel intensities as a luminance measure, following Korolkov's argument [51] that sharpness and luminance together determine whether a frame is legible enough to ground downstream reasoning. These two measures are recorded per frame; the keyframe selector is what later combines them into the quality it ranks on, weighting sharpness at 0.7 and luminance at 0.3. The change score captures how much new information the frame contributes, formed as a weighted average of two mean absolute pixel-intensity differences: 0.8 against the immediately preceding frame and 0.2 against the first frame of the stream, which serves as a fixed scene anchor. The short-term difference stays small, since consecutive frames barely change, while the long-term difference keeps growing the longer the run goes on. Weighting the long-term term at only 0.2 stops its ever-increasing size from dominating the score, so the change score continues to reflect genuine frame-to-frame novelty rather than just how far the run has drifted from scene's starting frame. On every fifth frame, the same per-frame pass also invokes the open-vocabulary YOLO-E detector and the BoT-SORT tracker that runs on top of it. The detector returns a class label, a bounding box, and a confidence score for each object it recognizes in the color image, with the nineteen classes the laboratory deployment is concerned with, namely its agents, furniture, portable items, cabling, and structural fixtures, fixed at export time. The tracker assigns each detection a persistent identifier across stride passes and accumulates a short history of its 3D position; by the time the window closes, every object that persisted over enough stride passes and moved more than the sensor noise floor carries a velocity estimate, expressed in the world frame.

Once the buffer fills, a keyframe selector picks the representative frames by scoring each on the quality and change scores already described. Frames below a minimum quality floor are discarded outright, so that motion-blurred or poorly exposed frames cannot be selected on the strength of their change score alone. Frames that clear the quality floor are then gated on a minimum change threshold, and every frame that clears both gates is returned as a keyframe for the scene. If no frame clears the change gate, the selector keeps the single highest-change-scoring frame among the quality-eligible candidates; and if no frame clears the quality floor at all, it falls back to the single highest-quality frame. Either way, every window yields at least one keyframe. The detections attached to those keyframes are aggregated into a per-object record carrying the object's semantic class, its detection confidence, the bounding box and depth at which it was observed, the egocentric zone of the robot it occupies, and the motion state and direction inferred for it across the window. Each frame is then enriched with a class-level priors, meta data, hand-authored that holds the risk tags, retrieval hints, and cross-class associations.

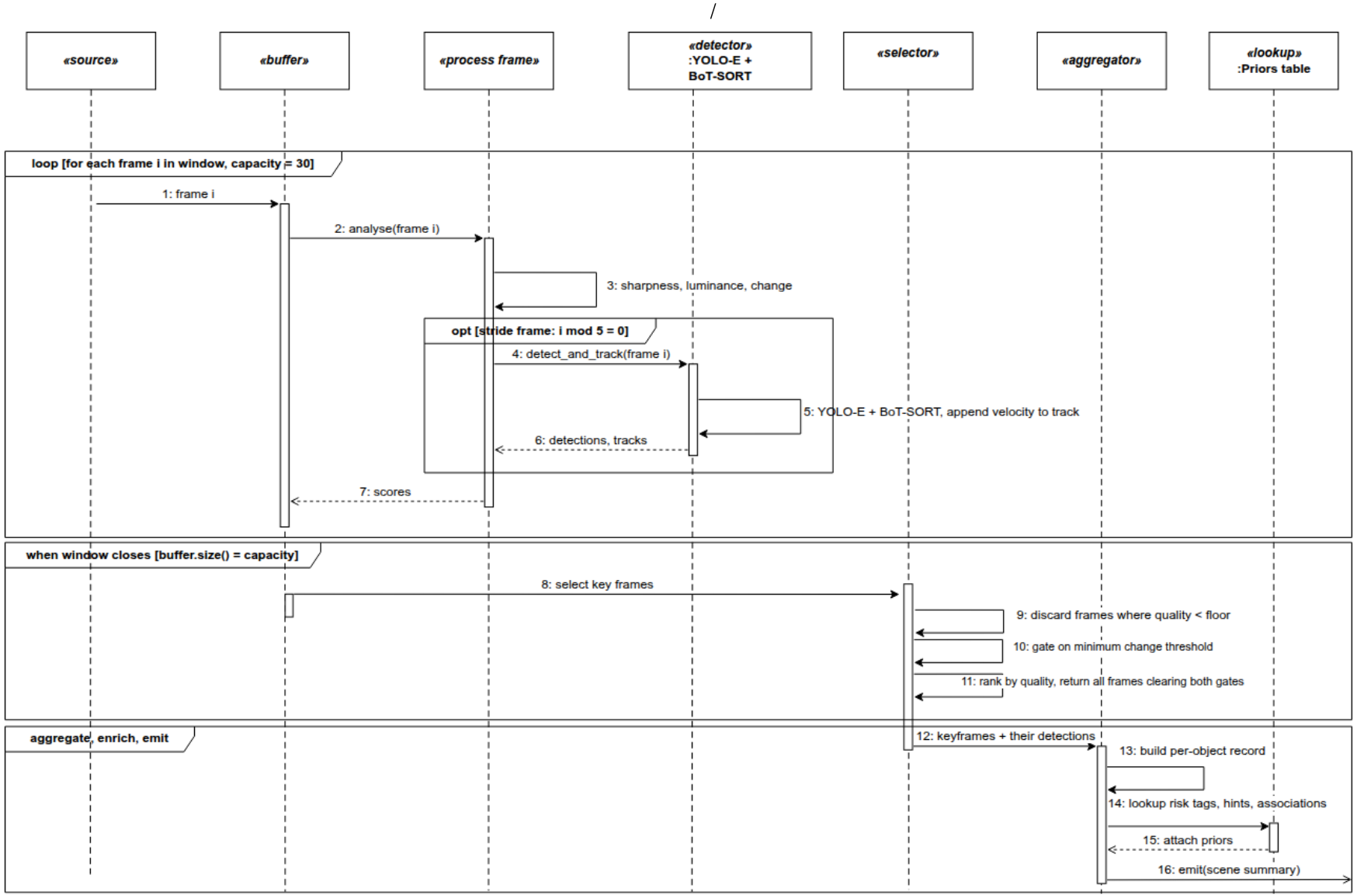


Figure 5: Sequence diagram of one scene window through SENSE

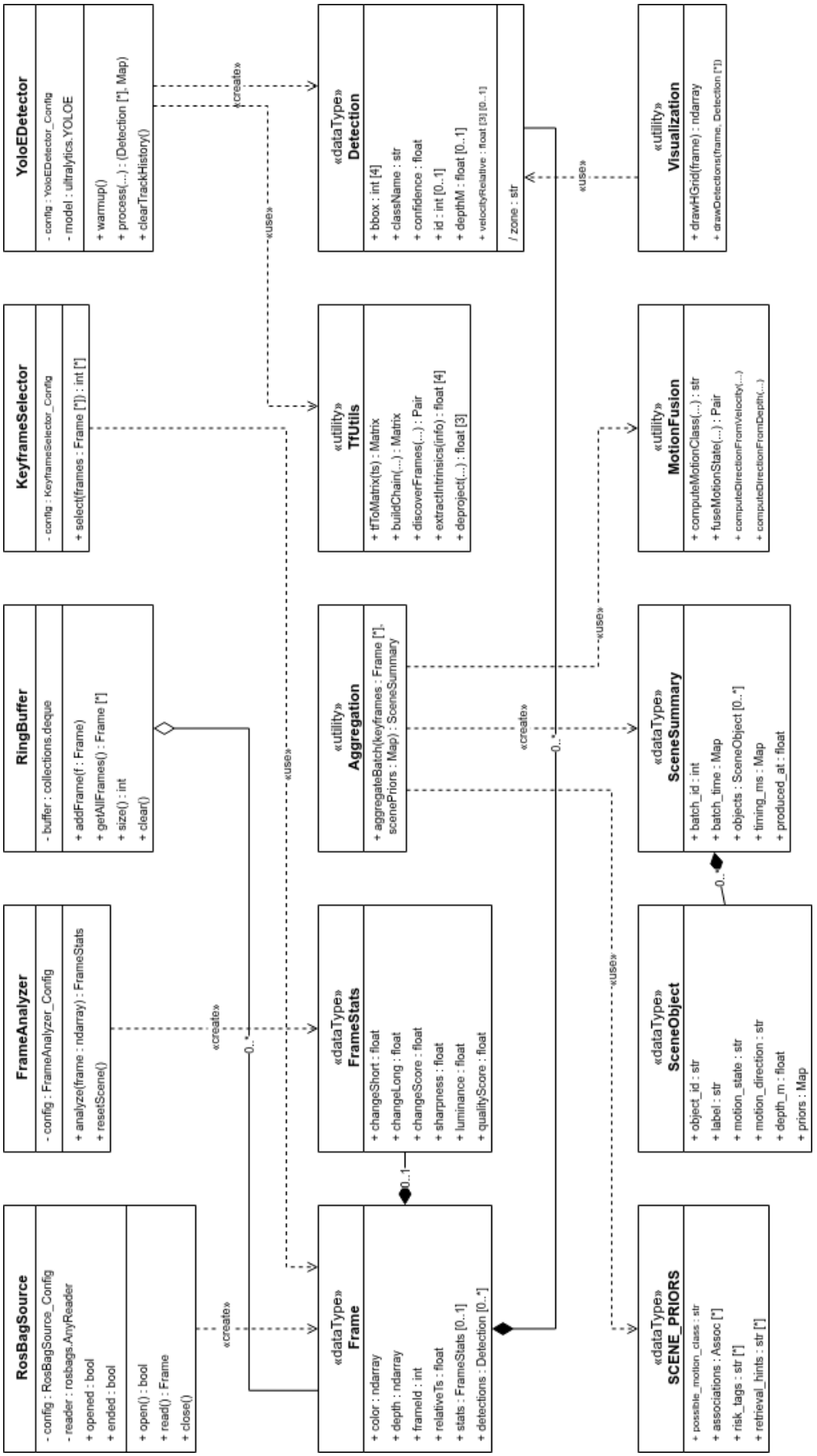


Figure 6: Class diagram of the SENSE stage

The priors scaffold the language model vocabulary that a safety engineer would use. They also help with retrieval, performed by vector similarity between the scene text and stored entries. The retrieval hints supply the domain terms that lift the match. The priors also carry knowledge that a domain expert has written down by hand, such as cross-class associations that record which classes typically operate near one another. The model's role is to speculate over these grounded inputs to generate hazard hypotheses. The completed record constitutes the scene summary that SENSE emits per scene. The buffer is then cleared, the BoT-SORT track history is reset, and the next scene begins.

SENSE runs as a single synchronous worker. The worker pulls each frame from the bag, dispatches it through the processors in turn, and hands the finished scene summary onward. Figure 5 traces those calls for one window, and Figure 6 gives the static counterpart, mapping each lifeline to the class that implements it. RosBagSource decodes the recorded topics into Frames, and FrameAnalyzer scores each one for legibility and change. YoloEDetector wraps the detector and the tracker and keeps every track's short history, KeyframeSelector chooses the representative frames once the window is full, and a RingBuffer holds the window the worker fills.

Each processor is constructed from its own configuration object, so the thresholds and weights that govern the stage are gathered in one place. A deployment can then be retuned for a different camera, detection vocabulary, or window size without changing the code that runs.

SENSE and the SPECULATE stage it feeds run at different rates, so the handoff between them must absorb the case where reasoning falls behind perception. SENSE is that single synchronous worker, while SPECULATE consumes scene summaries asynchronously from a bounded queue. The worker enqueues each completed summary without blocking and continues reading the bag, so perception never stalls on a slow reasoning step. When the queue is full the summary is dropped rather than buffered, and the consumer separately discards any summary older than a staleness bound before reasoning over it. The handoff is therefore lossy by design, favouring the currentness of the scenes that are processed over completeness of the scene stream.

What crosses the queue is a single fixed record. The scene summary is the contract between the two stages, and its structure is what SENSE must keep stable for whatever consumes it. Although scene graph construction is performed at the entry of the SPECULATE stage, it is detailed here because its inputs are entirely the product of perception, and the filtering choices it makes are dictated by what SENSE chose to put in the summary in the first place. SENSE's open-vocabulary YOLO model is configured for completeness rather than risk relevance alone, so the scene summary carries more detail than the language model that drives downstream reasoning can usefully consume within its per-scene token budget. The compression from summary to scene graph is the deliberate consequence of that choice, namely keeping perception broad and re-tunable while letting SPECULATE filter at the stage boundary into a token budget the model can read.

Each detection in the scene summary already carries the egocentric zone it occupies, locating the object relative to the robot, on the ground in its path, in its frontal field of view, on its left flank, or on its right flank. SENSE assigns the zone when YoloEDetector records the detection, taking the zone in which the center of the bounding box falls. The zones come from an H-pattern borrowed from VisionGPT [59], which used the same pattern to flag obstacles for visually impaired users. The pattern divides the camera frame into two vertical strips on the left and right that stand for the robot's flanks, and a wide central column that is split horizontally at a normalized height of 0.45 into a front zone above and a ground zone below, as shown in Figure 7. The split sits at 0.45 rather than at the image midline that VisionGPT uses, tuned empirically for the lower camera mount on the quadruped so that floor-resident objects in the robot's path classify reliably as ground while standing-height objects classify as front. This 0.45 split is an exception made explicitly for the quadruped's camera mount. A platform with a different camera placement may require a different split, or none at all. The scene graph that SPECULATE builds from the scene summary leans heavily on this zone tag to fit within the per-scene token budget the language model can consume.

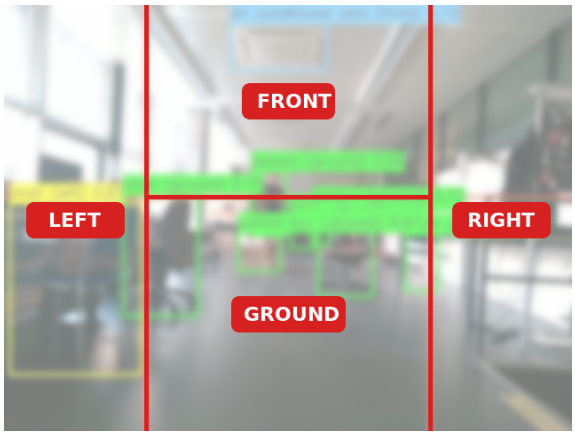


Figure 7: Egocentric H-pattern overlaid on a representative scene window.

Algorithm 1 Scene graph construction

Drop objects whose label is in SkipLabels or whose confidence < 0.3
Sort remaining objects by zone criticality, then by depth
$\mathcal{T} \leftarrow$ ground/front objects, dynamic objects, and static flank objects within d_{flank}
$P \leftarrow$ for each unique class in $\mathcal{T}$, aappend carrying risk tags, retrieval hints, and associations
for each pair (i, j) in $\mathcal{T}$, sorted by $|d_i - d_j|$, attach spatialrelation(i, j) to i
Mark the nearest non-person ground obstacle and the nearest object on each flank as alerts
for all $o \in \mathcal{T}$ in sorted order **do**
    Emit a line with o's id, zone, depth, fused motion, motion-confidence, alerts, and relations
end for
return P followed by the per-object lines

function SPATIALRELATION(i, j) ▷ $\Delta x, \Delta y$ are the in-frame center offsets of j from i
    if IoU$(i, j) > 0.1$ **then**
        return overlap
    else if $|d_i - d_j| > 2$ m **then**
        return front_of if $d_i < d_j$, else behind
    else if $|\Delta x| > |\Delta y|$ and $|\Delta x| > \tau_{xy}$ **then**
        return left_of if $\Delta x > 0$, else right_of
    else if $|\Delta y| > \tau_{xy}$ **then**
        return above if $\Delta y > 0$, else below
    else
        return near
    end if
end function

The construction of the scene graph from the per-scene summary is set out in Algorithm 1. Object classes that carry no safety relevance for the robot, namely the air conditioner vent, the ceiling light, and the window, are removed at the entry to the construction. The same step drops any remaining detection whose mean confidence falls below a low confidence floor of 0.3. The floor sits above the Ultralytics YOLO default of 0.25 and was raised empirically on the laboratory recording to drop low-confidence detections that would otherwise consume tokens the language model has to spend on real hazards.

The surviving objects are sorted by their zone, with the ground zone first because an object directly in the robot's walking path is the most immediate collision risk, the lateral flanks next, since an object there can become a collision risk the moment the robot turns or the object moves, and the frontal zone last in this ordering, with depth used as a tie-breaker so that closer objects come before farther ones within the same zone. The sorted list is then narrowed to a threat set, the objects that could plausibly affect the robot during the scene window. Each object's zone is

determined by where the center of its bounding box falls, and the center of a tall vertical object such as a person can land in either the ground or the front zone depending on how close the person is to the camera and how its bounding box spans the frame, so dropping the front zone on the strength of its low rank alone would risk losing the very objects the rest of the pipeline most needs to reason about. This exception, like the 0.45 split chosen earlier for the quadruped's camera mount, was made explicitly for the quadruped robot. Deployment on other platforms with different camera placements may require a different exception or none at all. The threat set is composed of all ground and front objects, all dynamic objects in the flanks, and any static flank object that falls within a configured distance threshold of the robot, so that a static object on the flank is not ignored if it is close enough to become a risk during the scene window.

The threat set is then written out in two sections, of which the first is a class-level description. Each unique class appearing in the set contributes a single line carrying its risk tags, its retrieval hints, and the classes it is associatively linked to in the scene priors table, so that the full priors of a class are presented exactly once regardless of how many instances of that class appear in the scene.

The second section is a per-object block in which each surviving object occupies one line. The line begins with a short identifier composed of the object's class and its tracker identifier, and is followed by its zone, its depth, and a fused motion field that joins the tracker's inferred direction with the scalar speed when both are available and falls back to the motion state otherwise. A motion confidence value is appended only when it falls below a configured threshold, so that a hedge is shown only where the tracker is genuinely uncertain about the motion call. Two further enrichments are folded into the line where applicable. An alert appears on the nearest object in the ground zone and on the nearest object in each flank zone. Up to two spatial relations per object are included. This simplification was made to accommodate the downstream model's ability and the token and time budgets. Fields from the summary that downstream reasoning never reads, such as the area of the bounding box, the redundant scalar velocity, and the production.

The spatial relation between two objects is chosen by a small priority chain that picks the first description to fit. If their bounding boxes overlap by more than a fraction of their union, the relation is recorded as overlap. Otherwise, if the two objects differ in depth by more than two meters, the closer is labeled front of the farther. Otherwise the in-frame geometry decides, with the larger of the two center offsets, lateral or vertical, naming the relation as left of, right of, above, or below according to its sign. If none of those tests is strong enough to commit, the relation defaults to near. Pairs themselves are restricted to objects whose zones are the same or geometrically adjacent, with the left and right flanks counted as non-adjacent so that the model is not asked to reason across the robot's body, and each object carries at most two such relations, drawn from the closest-in-depth pairs in which it participates.

The result is a compact representation in which the safety-relevant features of the scene. This helps the small model downstream to reason about hazards rather than to recover geometry of the scene.

4.2 Speculate

The cognition layer of the framework is realised in the SPECULATE stage of the pipeline. The scene graph is the input to SPECULATE, which turns one scene graph into a set of FMECA risk entries through five stages, summarized in Figure 8. The scene is first turned into a short natural language scene description and a query embedding of that description. The embedding is then used to search an FMEA knowledge graph for relevant precedent. A sufficiency check then decides whether the retrieved entries supply enough relevant grounding for the model to derive a defensible FMECA estimation for the scene, or whether the scene contains hazards that no retrieved entry speaks to. If deemed sufficient, the FMECA entries are generated grounded in that precedent alone; otherwise, the pipeline draws on a normative document store comprising incident reports of past field failures and academic literature on robot risk, and reasons about the present scene with that wider material as context. For each hazard the model identifies in the scene, it reasons about a process step at which the failure could occur, a failure mode that names the object whose handling could fail, the effect that failure would have, the cause that would produce it, and a candidate control measure, and assigns a severity level on a four-point ordinal scale and a probability level on a five-point ordinal scale with a one-line rationale for each. A deterministic computation then combines the two levels into a criticality category. A writeback at the end returns the entries from the novel path to the same knowledge graph the search began with, so that a scene that started without FMEA precedent contributes precedent for later scenes. The description that follows traces the same scene whose perception was described in the preceding subsection through these five stages. Figure 9 renders the five stages as a sequence of calls between abstract participants, although in the implementation the calls are dispatched by Pipeline.run rather than passed directly between the participants the figure draws.

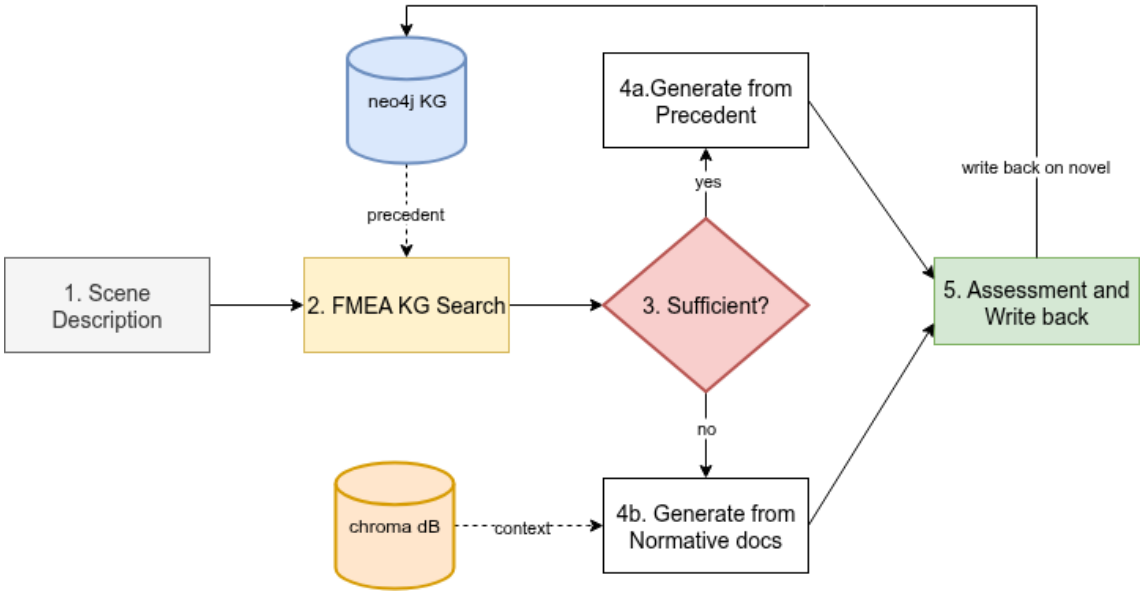


Figure 8: The five stages of the SPECULATE pipeline

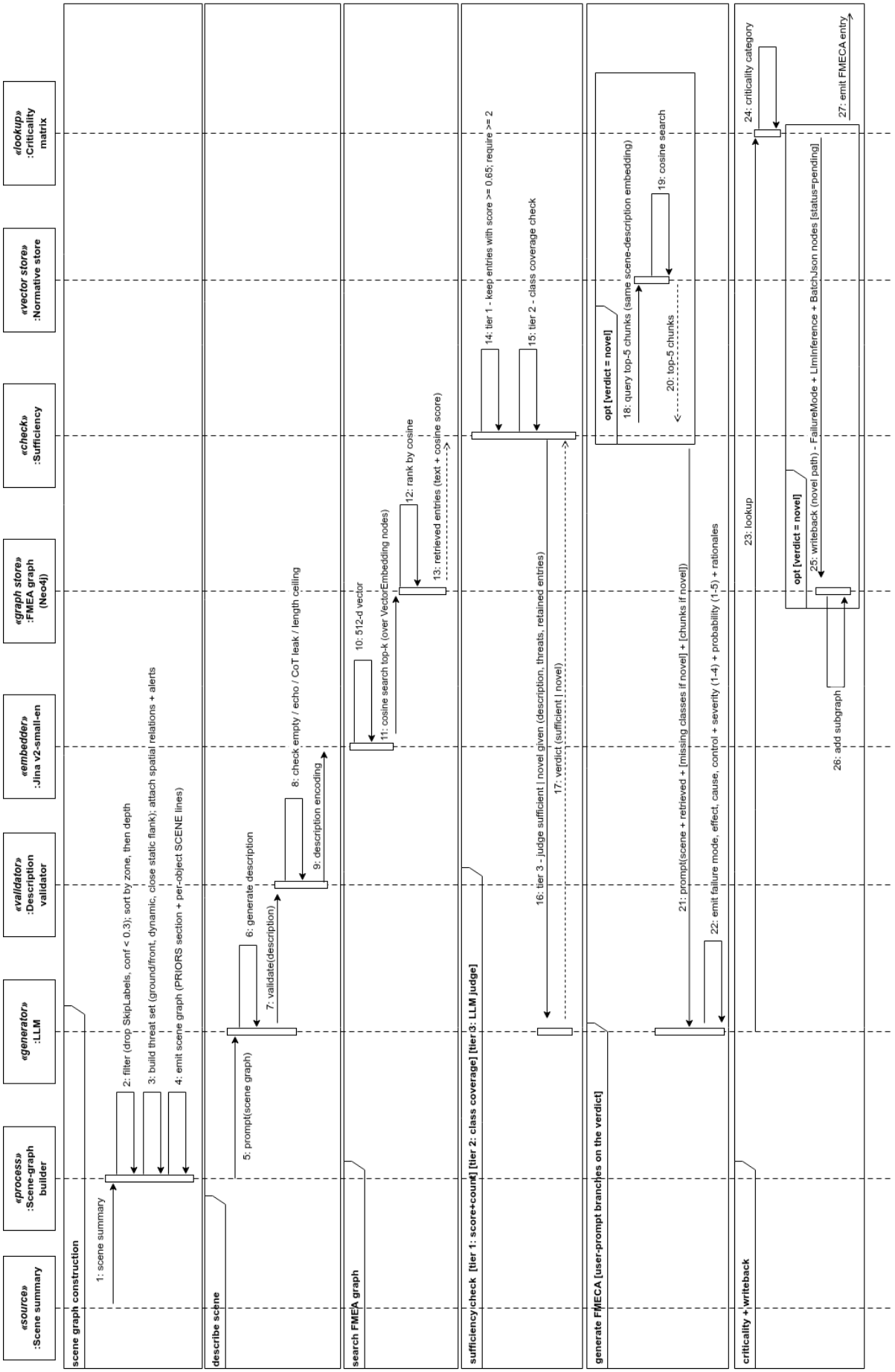


Figure 9: Sequence diagram of one scene through SPECULATE.

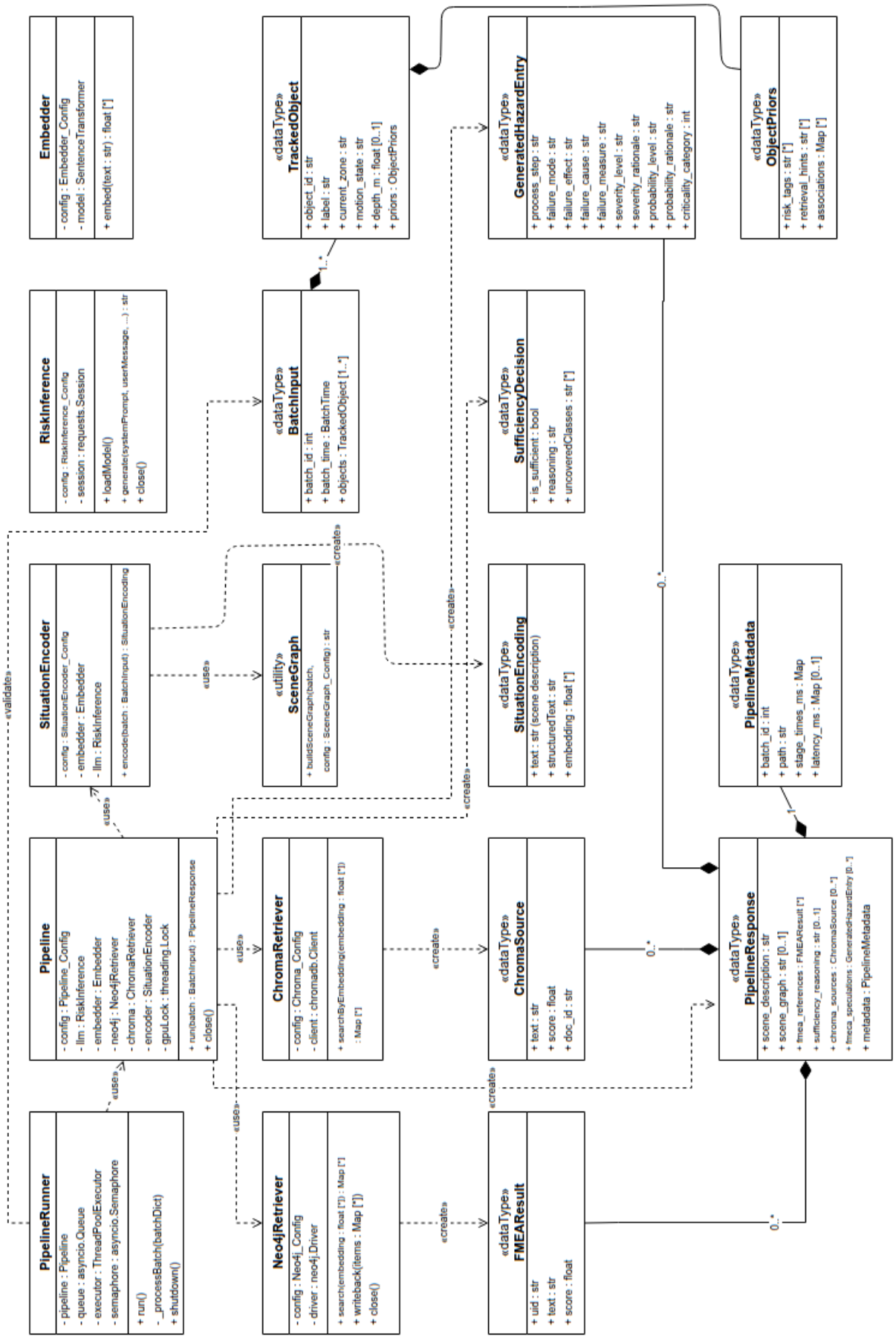


Figure 10: Class diagram of the SPECULATE stage

Figure 10 sets out the static structure of SPECULATE, showing how the sequence diagram in Figure 9 maps onto the implementation. The orchestrator that sequences the five stages is Pipeline, the scene-to-text-and-embedding step is SituationEncoder, the language-model client through which every generation call passes is RiskInference, the embedder that produces the query vector is Embedder, and the two retrievers that look up precedent are Neo4jRetriever for the FMEA knowledge graph and ChromaRetriever for the normative document store. Unlike SENSE, which is driven by a single worker because the only state worth keeping lives inside the buffer and the tracker, SPECULATE is built around Pipeline because the routing between the sufficient and the novel path, the need to serialize every model call on the GPU, and the per-stage timing and error book-keeping are all behavior and state that belong together.

A small set of record types travels between these classes and defines the contract each stage must respect. In turn they carry the scene encoding, the entries returned by each store, the routing decision, and the generated hazard fields. Only PipelineResponse crosses the SPECULATE boundary, and it also carries a PipelineMetadata record of the path taken and the per-stage timing. As with the scene summary in SENSE, its schema is the one place where the contract with anything downstream must remain stable.

PipelineRunner is the asynchronous consumer of the queue that SENSE writes into, the layer that drives Pipeline rather than forming part of it. It validates each scene summary against the schema it requires, the structured per-scene record the first stage turns into the scene graph, before it accepts it, and dispatches accepted batches to a pool of worker threads. This separation keeps queue intake and schema validation out of the per-scene reasoning, so that each can be retuned without touching the other. Each component is built from its own small configuration, with a stage-level configuration holding the sufficiency thresholds, per-stage timeouts, and per-stage decoding parameters. The prompts the language model is shown at every stage are kept in a separate prompt table that RiskInference reads when it runs. Holding the prompts and the thresholds outside the components is what allows a deployment to retune SPECULATE for a different model, a different risk vocabulary, or a different sufficiency policy without changing the implementation.

Five of these classes each depend on one external library that SPECULATE uses but does not own, shown in the figure as the type of a single field on the owning class. PipelineRunner holds the asynchronous queue it consumes from, RiskInference the HTTP session it issues every generation call through, Embedder the sentence-transformer model that produces every embedding, Neo4jRetriever the Neo4j driver, and ChromaRetriever the ChromaDB client. The same Embedder is used both at runtime to encode the scene description and to encode the FMEA entries and the normative document chunks, so that the query vector and the stored vectors share a single embedding space and a query made against either store is comparable on the same scale. Keeping each external library behind the class that owns it is what allows the rest of the system to be examined, swapped, or tested without reaching into the libraries beneath, and concentrates the parts whose internals we do not control, most importantly the language-model weights and the embedding-model weights, into a small set of replaceable seams.

In the first stage, the scene graph is shown to the language model that drives SPECULATE under a system prompt that tells the model to act as a safety observer for a quadruped robot carrying a payload through a shared laboratory, and lists the three failures the robot must avoid in that role, namely dropping the payload, harming a person, and damaging equipment. The prompt then explains the structure of the input the model is about to read. The scene graph is laid out in two sections, a PRIORS section that lists each unique object class in the scene together with the risk tags, retrieval hints, and class associations declared for it, and a SCENE section that lists each detected object on its own line with its zone, depth, motion, alerts, and spatial relations to its neighbours. The prompt names both sections so that the model interprets these structured fields rather than transcribes them. It then asks the model for a forty-five word, first-person description of the scene, and for a speculation on what could go wrong if the robot continued on its current path. The first-person framing fixes the description in the robot's perspective, which is the perspective from which the rest of the pipeline reasons about risk. The full system and user templates are reproduced in Appendix J.

The generated scene description is checked by a structural validator before the pipeline proceeds. The validator rejects an empty response, a response that echoes the scene graph rather than interprets it, a response that begins with reasoning markers such as “First, I need” or “Let me think” that signal a chain-of-thought leak into what should be the answer, and a response that exceeds the character ceiling set at roughly twice the expected output length to capture degenrate outputs. A failure at this gate halts the pipeline for the scene and records it as a scene description failure, the failure mode against which the viability evaluation in Section 5.6 measures candidate model configurations. A description that clears the validator is then embedded by the Jina v2-small-en sentence-transformer model into a five-hundred-and-twelve dimensional unit-norm vector, which is the query vector used by the retrieval that follows.

The graph stores the failure modes the deployment can encounter,fixed at design time and do not depend on the scene the robot is in. Severity, occurrence, and detection are recorded alongside each row at authoring time, as per FMEA practices, but at runtime the ratings are stripped from the reference passed to the model, and severity and probability are both reassigned by the language model during operation and criticality is computed only at runtime. In this work the graph is built from a process FMEA document covering the hazards the quadruped is expected to encounter when delivering a payload across the laboratory. Its schema follows the FMEA formalization introduced by Bahr et al. [75]. Each row of the document begins with the process step in which the failure could occur, names the failure mode, lists the effects on the robot, on the payload, and on any person in the vicinity, attaches a severity rating, enumerates the potential causes, attaches an occurrence rating, names the controls currently in place, attaches a detection rating, records the resulting risk priority number, and closes with the recommended actions and the parties responsible for them. Appendix I tabulates a representative excerpt of the document.

The graph supports symbolic queries by design, but the pipeline interrogates it by vector similarity. To make each entry retrievable by vector similarity against the scene description, the FailureMode subgraph is serialized into a single text chunk and embedded with the same Jina v2-small-en model used for the scene description. The embedding is stored on a VectorEmbedding node linked to its FailureMode, and a cosine index over these nodes is the target the retrieval queries at runtime. Figure 11 summarizes both halves of the FMEA knowledge graph in one view: the offline build that turns the authored process FMEA entries into FailureMode subgraphs and VectorEmbedding nodes, and the runtime path along which the cosine vector index is queried and novel-path entries are written back.

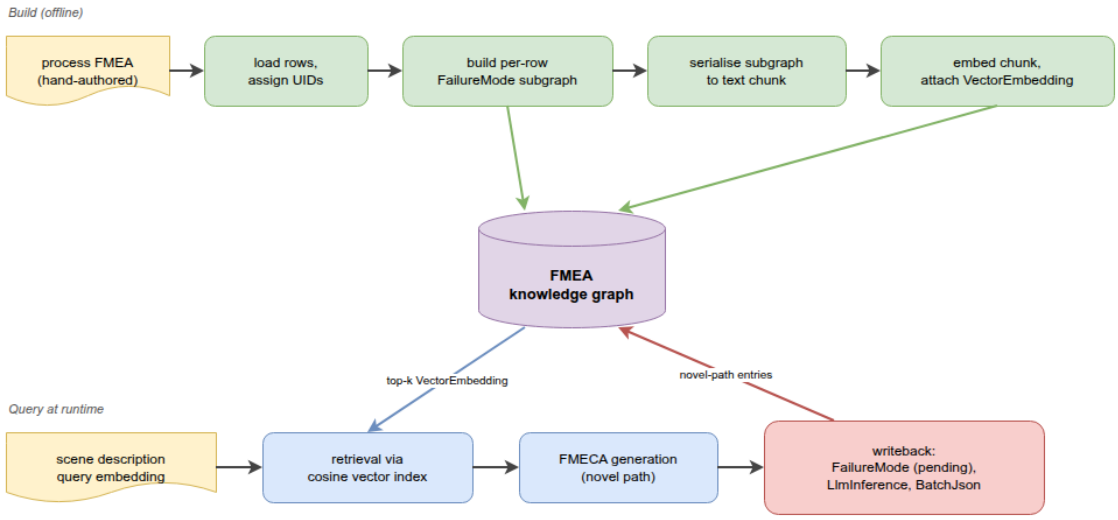


Figure 11: Build and runtime flow of the FMEA knowledge graph.

The second store of normative documents is consulted on the path the sufficiency check takes when the FMEA graph does not cover the hazards in the scene. The corpus is a collection of PDF documents in three classes, namely incident reports drawn from the AI Incident Database cover-

ing autonomous vehicle, shopping mall, warehouse, and security robot incidents, hand-authored synthetic incident reports describing laboratory-relevant hazards that the AI Incident Database does not capture, and academic literature on robot-related injuries, human-robot collaboration risk assessment, and situational risk assessment for autonomous mobile robots. These documents are admitted to the store through an extract-transform-load pipeline, set out in Figure 12.

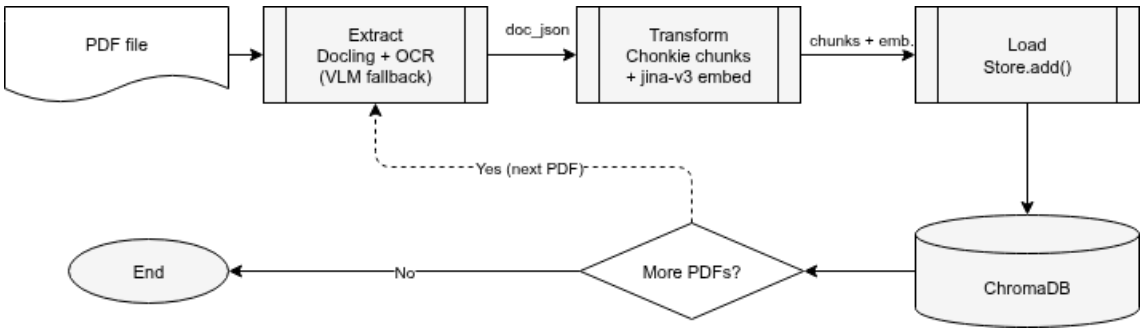


Figure 12: Extract-transform-load pipeline for the normative document store.

The extract stage parses each PDF through Docling [76]. Each page is first passed through a layout analysis model that segments it into regions and assigns each region a structural label, distinguishing body text from headings, captions, tables, figures, and the running headers and footers that repeat across pages. Text regions whose underlying PDF carries a recoverable text layer are read directly from the file; text regions on scanned pages, where no text layer is present, are routed through optical character recognition, so that the two are recovered in the same pass. Regions labeled as tables are passed to Docling's table structure model, which recovers the cell topology and emits a structured representation rather than a flattened bag of cell strings, so that table-borne information is preserved with its row and column relationships intact. Figure regions are recognized but their visual content is not extracted, and only the caption text enters the downstream pipeline. Pages on which the layout model returns low-confidence regions, or on which OCR returns degraded text, are routed to a fallback in which Docling sets the layout pipeline aside and reads the rendered page image directly through a vision-language model that returns the page's content as a single block of structured text. The recovered regions are then serialized in reading order into a single markdown-like document, headers and footers are dropped at this point by the structural label filter, since their repetition across pages would inflate retrieval scores without contributing content, and the resulting running text is what the transform stage chunks.

The transform stage cuts this text into retrievable pieces under a late-chunking strategy [77]. The strategy is chosen to accommodate how academic papers and technical standards are written, since meaning at any one point in such a document typically depends on definitions, framing, or terminology introduced earlier, and a chunk embedded in isolation would lose that scaffolding. The Jina v2-small-en model in this strategy is run once over the whole document, so that each token's representation is computed with the rest of the document available to it through the model's attention, and the chunk's embedding is then formed by averaging these token representations within the chunk's span. The chunk's vector therefore carries context from outside the span itself, which is the property that makes late chunking preferable to embedding each chunk in isolation. Documents longer than the model's input window are first split into overlapping segments, with each cut snapped to the nearest sentence or paragraph boundary so that no segment ends mid-word. A chunk that falls inside the shared overlap region of two adjacent segments has its two embeddings averaged at the merge step and the result unit-normalized, so that a chunk straddling a boundary inherits context from both sides rather than from one alone. Chunks that carry too little context to retrieve usefully are filtered as orphans, namely those shorter than twenty tokens or fifty characters and those whose stripped text consists only of digits, which are typically isolated headings, page numbers, or floating captions left over from extraction. The embedded chunks are then loaded into ChromaDB.The same Jina v2-small-en model is used here as in the FMEA graph, and the two stores therefore share an embedding space; the query vector produced from the scene description retrieves coherently from either without re-encoding.

The query embedding is presented to the FMEA graph through the cosine vector index, which returns the top-k VectorEmbedding nodes ranked by similarity. Each result carries the serialized text of its parent FailureMode subgraph and the cosine score of the match. The sufficiency check then decides whether the retrieved entries cover the hazards in the scene well enough that a generation grounded in those entries would be defensible, or whether the scene is uncovered enough that a fall back to the normative store is warranted. The check is structured as a three-tier cascade in order of increasing cost, so that scenes whose status is decidable from cheap signals never incur the cost of the language-model judgment that the most expensive tier invokes. Algorithm 2 sets out the cascade.

Algorithm 2 Sufficiency check

$R \leftarrow$ retrieved FMEA entries with score $\geq \tau_s$
if $|R| < n_{\min}$ **then**
    return novel ▷ too few relevant entries
end if
$\mathcal{T} \leftarrow$ object classes on collision path (ground/front), plus dynamic classes and close static classes in flanks
$M \leftarrow \emptyset$
for all $t \in \mathcal{T}$ **do**
    $S_t \leftarrow \{t\} \cup$ retrieval_hints(t)
    if no term in S_t appears in any text of R **then**
        $M \leftarrow M \cup \{t\}$
    end if
end for
if $M \neq \emptyset$ **then**
    return novel with missing set M
end if
Language model judges sufficiency given the scene description, the threat list, and R
return sufficient or novel as parsed from the response

The first tier filters the retrieved entries by a similarity score floor of 0.65 and requires at least two entries above the floor. A retrieval that does not clear both routes the scene to the novel path immediately. The second tier checks class coverage. The threat classes for the scene are recomputed using the same filter the scene graph builder applies, namely ground and front zone objects, dynamic objects in either flank, and static flank objects within a configured proximity. Each such class is searched for in the concatenated text of the retained FMEA entries by either its label or any of the retrieval hints declared in its scene priors. A class for which no term appears anywhere in the retained text is added to a missing set, and a non-empty missing set routes the scene to the novel path with the missing set carried into the prompt that the novel path will issue. Retrieval scores can be high yet skewed toward one well-covered class. When the FMEA holds many entries on that class, their text dominates the top-k and crowds out shorter entries about other classes the scene actually contains, so the missing set flags those omissions. Letting more entries through would lengthen the prompt without guaranteeing the underrepresented class is reached, since a top-k drawn from an imbalanced FMEA continues to favor the dominant class. The third tier runs only when both prior tiers pass. The language model is shown the scene description, the threat list, and the retained FMEA entries, and is asked to return either the token sufficient or the token novel with a one-sentence reason. A response the parser cannot resolve to either token is treated as novel, the conservative default that resolves ambiguity in favor of consulting the wider normative store.

The fourth stage generates the structured FMECA entries that the framework is responsible for producing, under one of two prompts according to the path the sufficiency check chose. The system prompt is shared across the two paths and instructs the model to ground each entry in objects and the scene, to name the object identifier for which the failure is asserted in the failure-mode field of every entry, and not to copy text from the FMEA entries that are shown to it as reference. The system prompt then defines the four severity levels and the five probability levels in the form of named rules to be applied in order, with severity tied to whether a person is at injury risk, whether

equipment is destroyed, whether operation is degraded, or whether the consequence is negligible, and with probability tied to whether the object is dynamic and approaching, dynamic and traversing, dynamic and receding, static of a typically static class, or static of a typically dynamic class. Tables 1, 2, and 3 below define these scales.

Level	Name	Definition
IV	Catastrophic	Human injury requiring medical attention (cuts requiring stitches, fractures, concussion, crushing injuries). Permanent damage to major research equipment. Cascading failures that could harm multiple people or systems.
III	Critical	Minor human injury (bruising, startling that causes someone to fall). Damage to equipment that halts ongoing research tasks. Safety system activation requiring investigation before resuming.
II	Marginal	No injury but near-miss events. Minor equipment damage that does not halt operations. Payload damage or spillage requiring cleanup. Significant delays or detours to the mission. Workspace disruption affecting others.
I	Negligible	Brief delays recoverable through normal replanning. Minor inconvenience to personnel. Temporary workspace clutter. Events that are noticed but require no response.

Table 1: Severity levels.

Level	Name	Observable indicators
A	Frequent	Hazard trigger conditions are fully present and actively manifesting. Trajectories are intersecting with no sign of change. Dynamic elements are behaving in ways that lead directly to the hazard. Multiple contributing factors are aligned.
B	Probable	Hazard trigger conditions are present. Dynamic elements are trending toward activation. Reasonable scenarios lead to hazard materialisation. Only minor changes in current trajectories needed for activation.
C	Occasional	Some trigger conditions are present, others are not. Hazard activation requires factors to combine. Current trajectory does not intersect, but changes could create intersection. Moderate uncertainty about dynamic element behaviour.
D	Remote	Hazard source is present but most trigger conditions are absent. Significant changes would be needed for activation. Current situation provides substantial margin. Dynamic elements are behaving predictably and not trending toward hazard.
E	Improbable	Hazard source exists only theoretically. No realistic scenario leads to materialisation. All dynamic elements are stable and distant.

Table 2: Probability levels.

	IV	III	II	I
A	1	1	2	3
B	1	2	3	4
C	2	3	4	5
D	3	4	5	5
E	4	5	5	5

Table 3: Criticality matrix

The user prompt on the sufficient path additionally supplies the retrieved FMEA entries as a reference and asks for a JSON entry on the primary hazard in the scene. The user prompt on the

novel path additionally supplies the top five chunks retrieved from the normative store, queried with the same scene description embedding to avoid re-embedding along with a list of uncovered classes that the second tier of the check produced. Both paths emit a single entry per scene, kept inside a single-element JSON array under abbreviated keys whose expansion is fixed in the parser, so that the structured output stays concise and the time cost of generating it stays within the per-scene token budget.

The fifth stage assigns the criticality category of each entry. The language model returns only the severity level and the probability level, both as ordinal symbols, together with a one-line rationale for each, and the criticality category is computed in Python by a deterministic lookup in the matrix in Table 3. The matrix can also be substituted by the deployment without changing the model's role in the pipeline, so a deployment whose risk management process uses a different categorization can adopt this framework by replacing the lookup table alone.

On the novel path, the entries the parser accepts are written back to the FMEA knowledge graph as a small subgraph distinct from the ones useed for retreival. Each entry is written as a FailureMode node carrying the failure-mode text, and a status field set to pending. The severity and probability levels, their rationales, and the computed criticality category are written onto a separate LlmInference node attached to the FailureMode by a wasInterpretedAs relation. The scene graph that produced the entry is written onto a BatchJson node attached by a hadContext relation, so that the audit trail from the generated entry back to its scene is preserved on the graph itself. These node types isolate the writeback subgraph from the retrieval index, since the cosine index is defined only over VectorEmbedding nodes and the writeback adds none. A pending entry does not influence retrieval until a verifier promotes it. The writeback closes the path the sufficiency check opened. A scene that began without FMEA precedent and was reasoned about with the help of normative material from the second store returns to the first store as a candidate piece of precedent for future scenes.

Every language-model call in the pipeline is issued through a single OpenAI-compatible HTTP client that holds a connection pool against a configurable inference server, the serving backend that hosts the model. Because every backend is reached through the same interface, the client is agnostic to the backend and the model behind it. The client exchanges messages with the server under a uniform request schema that carries the per-stage system prompt, user prompt, maximum-token budget, and sampling temperature. A per-stage timeout aborts a call that hangs, the request optionally carries a backend-specific reasoning-mode flag, and the response is post-processed to strip reasoning content before it is returned. This uniformity allows the evaluation in Section 5 to vary the model and the serving backend along the dimensions of scale, instruction tuning, quantization, and reasoning-mode overhead while holding the surrounding pipeline fixed, so that any difference observed in latency, correctness, or determinism is attributable to the configuration that was varied rather than to the orchestration that consumes it.

5 Evaluation and Results

Autonomous robots are increasingly required to operate alongside people, vehicles, and within unpredictable environments. A delivery robot in a warehouse must avoid forklifts and contact with fragile cargo; an inspection robot in a chemical plant must identify hazards, such as leaked fluids, before they pose risks to workers; and an emergency robot may be tasked with navigating smoke-filled corridors with limited sensor data to deliver first-aid supplies. Such missions necessitate continuous risk assessment under uncertainty, relying solely on onboard computation. A risk inference pipeline operating under these conditions must therefore be fast enough to keep pace with the sensor stream and real time deadlines, accurate enough to support downstream action, and consistent enough to return the same assessment on the same input. These three demands conflict with one another, and no single metric resolves whether a given model configuration meets all three.

To probe the three demands under controlled conditions, the proposed architecture is evaluated on a scenario derived from the supply delivery use case. A Unitree Go1 quadruped navigates a shared robotics laboratory to transport a container from a storage cabinet to a workbench. This setting covers a representative subset of typical operational threats that an autonomous robot might encounter while remaining simple enough for repeatable, controlled experimentation. The assessment itself is conducted offline. A human operator guided the quadruped under open-loop control while an onboard Intel RealSense camera captured visual and odometry data at roughly 15 fps over approximately 19 seconds of continuous forward navigation. The resulting ROS2 bag is replayed as a fixed input across every evaluation run. Because the open-vocabulary object detector, tracker, and keyframe selection logic are deterministic on identical frames, the SENSE stage yields the same scene graph on every replay. All variation across runs, therefore, originates in the SPECULATE stage. The traversal produced 10 scenes, of which the final one was discarded because it offered no meaningful evaluation signal, leaving 9. For each retained scene, a ground-truth scene description and ground-truth FMECA entries were prepared from the same structured scene graph that the model receives as input.

These nine scenes and their corresponding ground-truth references constitute the basis for a four-stage evaluation. The first stage, viability classification, determines whether a specific combination of model weights, quantization scheme, and serving backend can reliably complete the inference pipeline as configured in Table 6. Viable configurations are subsequently assessed along three dimensions: timeliness, correctness, and determinism, corresponding to the operational demands previously identified. Timeliness measures end-to-end latency to characterize operational cost; correctness assesses the alignment of generated safety responses with ground-truth references; and determinism evaluates whether identical inputs yield identical outputs across repeated runs. The writeback mechanism is evaluated separately to determine whether retrieving safety-engineer-authored risk categorizations improves correctness and reduces inference time for scenes not covered by the baseline knowledge graph. The following sections present each evaluation stage in sequence, with methodology and results discussed together.

5.1 Test Scenario

The test recording was captured in the robotics laboratory of the Institute of Industrial Automation and Software Engineering at the University of Stuttgart. The laboratory is an open-plan space with concrete columns, floor-to-ceiling windows along one wall, and ceiling-mounted air conditioning vents. Workbenches, shelving units, packaging, clutter, and office chairs are distributed throughout. The floor is smooth and level, but the navigable path is narrowed in several places by chairs pushed back from desks and by cartons left on the floor. Two people are present during the recording: one walking through the workspace and approaching the robot, and the other seated at the destination workbench, operating a robotic arm.

The risk profile across the trajectory is non-monotonic. The robot passes through low-risk corridor sections, encounters a high-criticality situation when a person closes to short range with lateral escape routes blocked by furniture and clutter, and finally returns to lower risk once the encounter

resolves, only to escalate again during the final approach to the occupied workbench. Figure 13 shows representative keyframes from the recording.



(a) (b) (c)

Figure 13: Representative keyframes from the test recording.

5.2 Models Under Test

The models under test span the Qwen2.5 [102], Qwen3 [103], SmolLM3 [104], and Gemma4 [105] families. Three axes are deliberately varied: model scale, instruction tuning, and reasoning-mode overhead, so that the correctness and latency results reported in the sections that follow can be attributed to each axis in turn. The quantization scheme and serving backend are not free parameters. Each is determined by what the model architecture and the Jetson AGX Orin software stack jointly permit at the time of evaluation, and the assignment is justified at the end of this subsection. Table 4 summarizes the configurations.

Model	Variant	Quantization	Serving Backend
Qwen2.5-3B	Pretrained (no instruction tuning)	INT4 AWQ	TensorRT-LLM 0.12
Qwen2.5-3B-Instruct	Instruction-tuned	INT4 AWQ	TensorRT-LLM 0.12
Qwen2.5-7B-Instruct	Instruction-tuned	INT4 AWQ	TensorRT-LLM 0.12
SmolLM3-3B	Reasoning enabled	BitsAndBytes 4-bit	vLLM 0.16
SmolLM3-3B-NoThink	Reasoning disabled	BitsAndBytes 4-bit	vLLM 0.16
Qwen3-4B-Thinking	Reasoning enabled Reasoning disabled	INT4 AWQ	vLLM 0.9.2
Gemma-4-26B-A4B	Instruction-tuned	Q4_K_M GGUF	llama.cpp

Table 4: Evaluated Model Configurations

Qwen2.5 was selected as the primary family because it provides a size ladder at the 3B and 7B scales and a tuning ladder across base and instruction-tuned variants under a single architecture, enabling controlled comparisons without confounds from differing tokenizers, attention mechanisms, or training corpora. The inclusion of Qwen2.5-3B alongside its instruction-tuned counterpart establishes a controlled comparison of instruction tuning at the 3B scale. The two share identical architecture and quantization, differing only in whether the model was fine-tuned on instruction-following data. Qwen2.5-7B-Instruct provides a scale comparison within the same family.

SmolLM3-3B and Qwen3-4B-Thinking extend the evaluation to chain-of-thought reasoning, which has been shown to improve output quality on mathematical and commonsense tasks when transferred into smaller language models [106]. SmolLM3-3B exposes a runtime toggle between reasoning-enabled and reasoning-disabled inference, isolating the trade-off while holding the weights constant. The Qwen3-4B-Thinking-2507 checkpoint [103, 107], by contrast, is trained exclusively for extended reasoning and cannot be switched off at inference time, providing a second test point for the same trade-off with a differently trained model.

Gemma-4-26B-A4B is included as a recent sparse mixture-of-experts release whose active parameter budget remains within reach of edge hardware. Its 26 billion total parameters occupy

roughly 13 GB at 4-bit, yet only 4 billion active parameters are read from memory per token, placing per-token memory traffic in the same range as the dense 3B and 7B baselines.

The quantization scheme and serving backend listed in Table 4 are constrained by the Jetson AGX Orin software stack at the time of evaluation, namely JetPack 6.2 with TensorRT 10.3 on the Ampere GPU. NVIDIA assigns each GPU architecture a compute capability version, which determines the set of CUDA instructions and tensor-core operations the hardware supports. The Jetson AGX Orin uses the sm87 streaming-multiprocessor variant of the Ampere architecture, corresponding to compute capability 8.7. Several inference paths used in datacenter deployments require compute capability 8.9 or higher and are therefore unavailable on this device. Table 5 maps each remaining constraint to the configurations it forces.

Constraint	**Affects**	**Consequence**
FP8 tensor cores require compute capability 8.9 or higher, Orin is 8.7.	all seven	INT4 weight-only is the supported low-precision path.
The Jetson branch of TensorRT-LLM, version 0.12, predates support for the Qwen3 architecture and the Gemma 3/4 generations.	Qwen3-4B, Gemma-4	Both models must use a non-TensorRT-LLM backend.
No official AWQ or GPTQ checkpoint for SmolLM3-3B or Qwen3-4B-Thinking-2507.	SmolLM3, Qwen3-4B	SmolLM3 takes the calibration-free route. Qwen3 is served from a 4-bit AWQ checkpoint.
vLLM and llama.cpp builds on Jetson but ships no NVIDIA-supported SM87 wheels.	all non-TRT-LLM variants	Backend choice follows model architecture.

Table 5: Jetson software-stack constraints behind each quantization-and-backend pairing.

The three Qwen2.5 configurations use Activation-aware Weight Quantization (AWQ) [93], implemented via TensorRT-LLM [108]. This pairing is the inference path documented by NVIDIA for Qwen2.5 on Jetson AGX Orin [109], and the FP8 alternative is excluded by the hardware constraint in the first row of Table 5.

SmolLM3-3B is served through vLLM [110] using BitsAndBytes NF4 quantization [111]. No first-party AWQ or GPTQ checkpoint is published for the architecture, and producing one would require running the calibration procedure those schemes depend on. Reducing weights to four bits discards precision, and a flat rounding rule loses accuracy because a small fraction of weights carry far more of the model's output than the rest. Calibration uses a small sample of representative text to decide how to round so that this loss is minimised. The calibration-free route in Table 5 refers to this choice. BitsAndBytes NF4 quantizes the weights at model load time from the weight values alone, and is the only 4-bit option in vLLM that operates without a pre-quantized checkpoint.

Qwen3-4B-Thinking is served through vLLM with INT4 AWQ. The Jetson-supported branch of TensorRT-LLM predates support for the Qwen3 architecture. So TensorRT-LLM is not a candidate backend for this model.

Gemma-4-26B-A4B is served through llama.cpp using Q4_K_M GGUF quantization. Neither TensorRT-LLM nor vLLM supported the Gemma-4 mixture-of-experts architecture on Jetson at the time of evaluation. Q4_K_M is the standard 4-bit GGUF variant provided in the llama.cpp documentation as the balanced-quality option.

5.3 Speculation Pipeline Configuration

All model configurations share a common pipeline configuration to ensure that observed differences in output quality, latency, and determinism are attributable to the model and its serving

backend rather than to differences in pipeline settings. Table 6 summarizes the per-stage parameters.

Pipeline Stage	Temp.	Max Tokens	Output
Scene Description	0.3	60	Scene description and vector embedding
Sufficiency Check	0.2[1]	30[1]	Routing decision and list of uncovered object classes (if any)
Risk Assessment	0.3	180	Structured risk entries

Table 6: Pipeline stage configuration

Of the five SPECULATE stages defined in Section 4.2, the three reported here are the LLM-bearing ones, namely Scene Description, Sufficiency Check, and Risk Assessment. Retrieval is a vector-index lookup with no model call, criticality is a deterministic table lookup, and writeback is evaluated separately in Section 5.10.

Token budgets are set per stage to enforce concise, parseable outputs. Token generation dominates per-scene latency because decode is memory-bandwidth-bound, so each stage's budget was tuned empirically on the test recording to the smallest value that still yields a complete, parseable output, trading no quality for the lowest latency. The per-stage budgets in Table 6 apply uniformly across all non-thinking configurations. The thinking-enabled SmolLM3-3B and Qwen3-4B variant receives an additional 512 tokens per stage to accommodate reasoning chains emitted within `<think>` tags. These tokens count toward the generation limit and are stripped before the output is parsed. SmolLM3-3B-NoThink uses the same base weights with thinking explicitly disabled and receives the standard token budgets. The temperatures were chosen the same way, low and non-zero for stable output, tightened to 0.2 at the Sufficiency Check whose output is a routing decision. All other parameters, including temperatures, retrieval thresholds, and prompt templates, remain identical across configurations.

All LLM requests use an HTTP connection pool of size 4 but are issued sequentially with no request-level concurrency.

5.4 GPU Scheduling Modes

Each model configuration is evaluated under two GPU scheduling regimes. In the sequential regime, the SENSE and SPECULATE stages execute behind a shared GPU mutex so that only one stage occupies the accelerator at any time. SENSE processes frames only when SPECULATE is idle, and all frames remain in the queue until consumed, so no input is discarded. The sequential regime governs the correctness and determinism evaluations, where every scene must be available for comparison against ground truth and across repeated runs. In the concurrent regime, SENSE processes frames as they arrive and enqueues them, while SPECULATE performs inference on the current scene. Any frame that remains unconsumed when its timeout expires is dropped. The concurrent regime governs latency evaluation, which determines whether the framework can sustain real-time operation when both stages share a single accelerator and the pipeline must keep pace with a live sensor stream.

5.5 Scoring Instruments

For the timeliness evaluation, wall-clock time is recorded end-to-end for each scene under both the sequential and concurrent GPU scheduling regimes. For the correctness and determinism evaluations, four instruments are used.

- BERTScore [112] measures token-level semantic similarity between a generated text and a reference text. Both texts are passed through RoBERTa-large to obtain contextual token

embeddings. Precision is computed by matching each generated token to its most similar reference token and averaging. Recall is computed symmetrically from the reference side. F1 is the harmonic mean of the two. Raw BERTScore values tend to cluster in a narrow, high range for RoBERTa-large on English text, limiting discriminative power. Baseline rescaling subtracts a corpus-level baseline and renormalizes the scores, spreading the effective range. Both raw and rescaled scores are reported. Where a single summary is needed, the rescaled variant is used.

- G-Eval [113] is an LLM-as-judge framework that scores generated text against task-specific criteria. In this evaluation, GPT-4o serves as the judge at temperature 0.0. Each G-Eval call presents the judge with up to three pieces, namely an input the judge conditions on, an output - the model's generated text under assessment, and, when supplied, an expected output - a human-authored reference the judge compares against. This DeepEval vocabulary is used throughout the remainder of the evaluation. Three G-Eval metrics are defined in the DeepEval framework, each scored on a 0-1 scale with a passing threshold of 0.5: Scene Description Accuracy, FMECA Faithfulness, and FMECA Ground-Truth Alignment. Detailed evaluation steps and scoring rubrics for all three metrics are provided in Appendix B.
- Krippendorff's Alpha [114] is a coefficient that measures the extent to which independent raters agree on the same items beyond what chance alone would produce, on a scale where 1 denotes perfect agreement, 0 denotes agreement no better than chance, and negative values denote systematic disagreement. The coefficient is computed as $\alpha = 1-(D_{\text{observed}}/D_{\text{expected}})$, where D_{observed} measures the disagreement actually present among the raters and D_{expected} measures the disagreement that would be observed if each rater's value were drawn independently from the overall distribution of ratings in the dataset. The disagreement function is selected to match the measurement scale, with ordinal scales using the squared rank difference between ratings so that adjacent levels contribute less disagreement than distant ones. Krippendorff identifies an Alpha of 0.80 as the threshold above which the rated data can be relied on for conclusions and 0.667 as the floor below which conclusions cannot be drawn from the data.
- Jina cosine similarity uses the same Jina v2-small-en embedding model [115] that drives the pipeline's vector search during operation. Each scene description is encoded into a single 512-dimensional vector, and pairwise cosine similarity is computed across runs. Unlike BERTScore, which aligns individual tokens and is sensitive to vocabulary and phrasing differences, Jina cosine similarity operates at the document level and captures whether two descriptions occupy the same region of semantic space regardless of surface wording. The two measures are complementary. A high Jina cosine similarity paired with a low BERTScore indicates that descriptions convey equivalent meaning through different lexical choices.

5.6 Viability

This stage determines whether a given model configuration can complete the full inference pipeline end-to-end, from ingesting the scene graph to generating risk assessment, within the pipeline's token and time budgets. For each evaluation scene, the model must produce a scene description that can be embedded and used for knowledge retrieval, and the retrieved context must, in turn, yield a complete risk assessment. A configuration that fails to close this chain for a given scene is recorded as a failure for that scene. A configuration is considered viable if it completes at least 5 of the 9 evaluation scenes. The threshold is lenient to avoid penalizing models for occasional failures, and instead focuses on filtering out configurations that are fundamentally unable to operate within the pipeline.

Viability is assessed at two checkpoints. At the pipeline entry, a structural check of the scene description determines whether the model's output is usable before retrieval proceeds. This check flags empty outputs, verbatim echoing of the input scene graph, leakage of chain-of-thought reasoning into the description field, outputs exceeding a character-length ceiling designed to catch degenerate generations, and violations of the per-stage time budget. At the exit, the pipeline must

have produced a complete risk assessment. Failures at any intermediate stage, retrieval, memory search, or risk assessment, are not diagnosed individually; they are captured by the absence of a final output. A scene is counted as a success only if both checkpoints are cleared. The quality of a completed output is not assessed here; output quality is evaluated separately in the correctness analysis that follows. Figure 14 shows completion rates for both scheduling regimes.

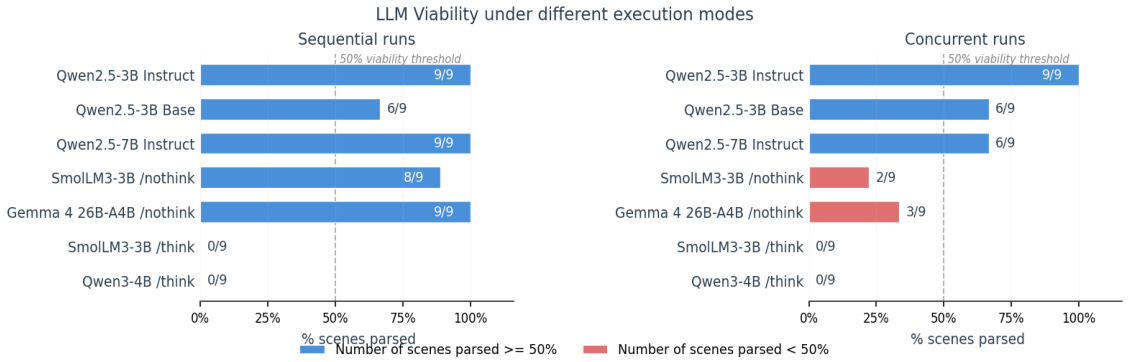


Figure 14: Completion rate per model under the two scheduling regimes.

Under sequential scheduling, three of the six configurations complete all nine scenes without failure. Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, and Gemma-4-26B-A4B each produce a complete risk assessment for every scene. The remaining three configurations exhibit failures that fall into two categories corresponding to the entry and exit checkpoints described above.

SmolLM3-3B fails at the entry checkpoint on every scene. This model emits a reasoning trace inside a `<think>` block before producing its answer. Inspection of the raw outputs across all nine scenes revealed two failure paths. In most cases, the reasoning trace consumed the entire token budget, leaving no tokens for the answer (Figure 15a). In the remaining cases, the reasoning ended early enough for the answer to begin, but the model used the available tokens to generate a scene description that exceeded the character-length ceiling and failed the entry check (Figure 15b). Disabling extended thinking via the `/no_think` flag eliminates both failure paths entirely (Figure 15c). The failure, therefore, stems from the thinking mechanism rather than the model's base language capability. A thinking-enabled model in its default decode path spends an a-priori-unknown share of a fixed token budget on the hidden trace and does not self-limit that trace to preserve room for the answer. The pipeline allocated an explicit thinking-token budget on top of the answer budget for the reasoning-enabled configurations, yet that allowance was still exhausted before a usable answer was produced, and enlarging it would not change the outcome, since each additional decode step adds operation time and this pipeline is assessed for operational-time inference on edge hardware.

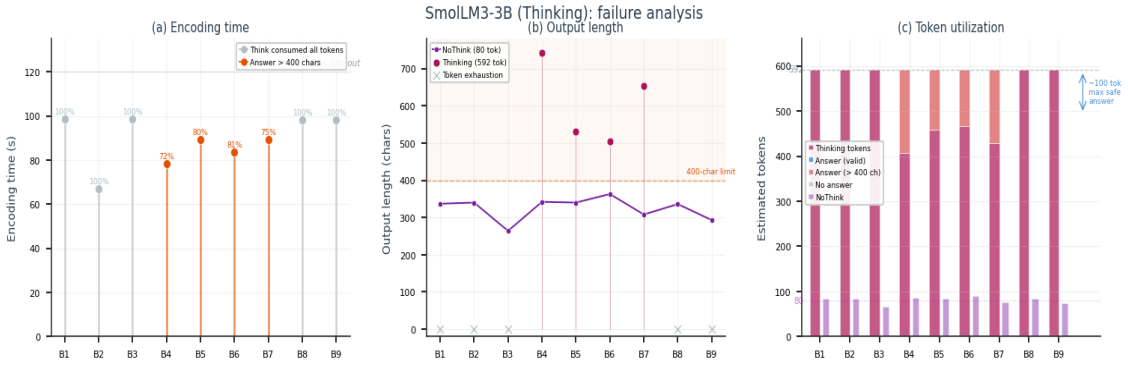


Figure 15: SmolLM3-3B failure analysis: reasoning trace consumption across nine scenes.

Qwen3-4B-Thinking, the second reasoning-enabled configuration in Table 4, also fails on every scene but through a different mechanism that supports the same diagnosis. The model encloses

its reasoning trace inside the <think> delimiters that the inference layer strips successfully, but reproduces a chain-of-thought preamble of the form “First, I need to understand the scene…” in the visible answer and produces a multi-paragraph analysis instead of the requested 45-word description. Every scene fails the entry checkpoint on two counts: a reasoning marker at the start of the answer and an output length four to five times the character ceiling. SmolLM3-3B exhausts the token budget on its hidden trace; Qwen3-4B-Thinking exhausts the budget on a verbose answer. The two reasoning-enabled configurations fail through different mechanisms but converge on the same operational outcome, which is the empirical basis for excluding reasoning-enabled models from this pipeline rather than relying on a single-model finding. The full per-scene trace is in Appendix F.

Qwen2.5-3B presents a different failure pattern that illustrates why the exit checkpoint is necessary. The model passes the entry checkpoint across all 9 scenes, producing scene descriptions that meet the format requirements. In three of these scenes, the risk assessment stage produces no parseable FMECA entry. Qwen2.5-3B-Instruct, which shares identical architecture and quantization but has undergone instruction tuning, completes all nine scenes without failure. The difference is attributable to an instruction-following deficit in the base model; the detailed causal analysis is presented in Section 5.8.

SmolLM3-3B-NoThink completes eight of nine scenes under sequential scheduling. Its single failure occurs on the final scene, where the FMECA stage decodes to the token ceiling without producing a parseable hazard entry. The remaining scenes all produce valid entries. This is a rare edge case in structured-output generation and not a systematic limitation of the model.

Under concurrent scheduling, the configurations that were viable under sequential scheduling degrade to varying degrees. The models lose scenes to the bounded queue of SPECULATE. Qwen2.5-7B-Instruct drops to six completed scenes, Gemma-4-26B-A4B drops to three, and SmolLM3-3B-NoThink drops to two. Every scene that reaches SPECULATE under concurrent scheduling completes successfully. Without the sequential mutex, SENSE processes incoming frames continuously, producing scenes far faster than SPECULATE can consume them. The bounded queue saturates and begins discarding scenes, either by rejecting new arrivals when full or by expiring queued scenes that have aged past the staleness threshold. Slower models exacerbate the imbalance, as the per-scene inference time determines both how long each scene occupies the consumer and how many queued scenes age out before they are dequeued. Per-model timing breakdowns and the full per-scene logs are provided in Appendix C.

Three configurations pass the viability threshold in both scheduling regimes. The two 3B Qwen configurations and Qwen2.5-7B-Instruct complete at least five scenes under both conditions. SmolLM3-3B-NoThink and Gemma-4-26B-A4B pass under sequential scheduling but fall below the threshold under concurrent scheduling. Both are retained for the sequential correctness and determinism evaluations that follow, as they represent the two non-TensorRT-LLM serving backends and their inclusion is necessary to evaluate whether backend differences affect output quality independently of throughput.

5.7 Timeliness

This section reports the end-to-end pipeline latency for each retained configuration. Since the token budgets of each stage in the SPECULATE part of the pipeline are fixed across all configurations, pipeline latency is determined by throughput. Table 7 lists the measured decode throughput for each configuration.

Token generation in transformer-based language models is memory-bandwidth-bound rather than compute-bound [95, 116]. Each decode step requires loading all model weights from DRAM, and the GPU spends most of its time in each forward pass waiting for data to arrive. On the Jetson AGX Orin, the CPU complex, the Ampere GPU, and the deep-learning accelerators share a single LPDDR5 memory pool. NVIDIA specifies the theoretical peak memory bandwidth of the Jetson AGX Orin at 204.8 GB/s [90]. A CUDA memory-copy benchmark executed on the target device under the MAXN power mode measured a sustained bandwidth of 168.9 GB/s, roughly 82

Model	Serving Backend	Tokens/s
Qwen2.5-3B	TensorRT-LLM 0.12	$\sim$36.0
Qwen2.5-3B-Instruct	TensorRT-LLM 0.12	$\sim$40.0
Qwen2.5-7B-Instruct	TensorRT-LLM 0.12	$\sim$20.0
Qwen3-4B	vLLM 0.9.2	$\sim$9.0
SmolLM3-3B	vLLM 0.16	$\sim$5.5
SmolLM3-3B-NoThink	vLLM 0.16	$\sim$6
Gemma-4-26B-A4B (no-think)	llama.cpp	$\sim$12

Table 7: Measured decode throughput on the Jetson AGX Orin 64GB.

percent of this peak. The 18% shortfall matches the device-to-device bandwidth of approximately 170 GB/s, as independently reported for the AGX Orin on NVIDIA's developer forum [117]. A 3-billion-parameter INT4 AWQ model occupies roughly 1.5 GB of weights. Dividing 168.9 GB/s by 1.5 GB gives an upper bound of approximately 112 tokens per second from memory bandwidth alone. The fastest configuration in Table 7 reaches roughly 35 percent of this ceiling, consistent with published measurements on the same platform [93].

The results below begin with concurrent scheduling, which exposes the full cost of running perception and reasoning on a shared accelerator and, as the data will show, the operational infeasibility of all configurations except the fastest two. For SmolLM3-3B-NoThink and Gemma-4-26B-A4B, 70 to 80 percent of scenes are dropped from the queue before they reach SPECULATE, and the reported concurrent latencies for those configurations therefore aggregate over the surviving subset rather than over a representative steady state. The cause is a producer-consumer rate imbalance: SENSE emits scenes at roughly 1.43 per second under concurrent scheduling, while SPECULATE consumes at 0.17 per second in the fastest configuration, an $8.6\times$ ratio that no scheduling discipline can absorb without discarding scenes, as detailed in Appendix C. Sequential scheduling is used as an analytical baseline to decompose the observed latencies into their model-attributable and queue-attributable components, and as the only regime in which every configuration completes every scene without sample selection.

Figure 16 shows the mean pipeline latency of the three models served through TensorRT-LLM under both scheduling regimes. Under concurrent scheduling, backlog wait is the dominant latency component due to slow consumption, accounting for 59-69% of total pipeline time across the three models. Qwen2.5-3B-Instruct completes a full pipeline pass in 17.6 seconds on average and the base Qwen2.5-3B in 22.7 seconds, but the backlog wait exceeds the differences in inference speed between configurations, obscuring the true cost structure of each model. Under sequential scheduling, SENSE waits for SPECULATE to release the GPU before processing the scene, eliminating the backlog queue entirely. The resulting serialization wait is bounded per scene and does not accumulate across the run. Qwen2.5-3B-Instruct completes a full sequential pass in 9.1 seconds on average, 44 percent faster than the 16.2 seconds measured for the base Qwen2.5-3B, while Qwen2.5-7B-Instruct reaches 15.9 seconds. The distinction between regimes is visible in the per-scene latency distributions: the concurrent IQR spans 11.3 to 18.8 seconds for the TensorRT-LLM configurations (Figure 17), while the sequential IQR spans only 0.7 to 4.6 seconds (Figure 18). The monotonically increasing backlog, not GPU contention between perception and reasoning, is the dominant source of the concurrent latency spread.

Separately, an artifact of lock-free GPU sharing was observed on two concurrent runs across two different models. Without the scheduling mutex, SENSE and SPECULATE submit CUDA kernels to the same GPU simultaneously. In the affected runs, a sequence of long generation kernels from SPECULATE occupied the GPU back-to-back, preventing the shorter detection kernels from executing and stalling the perception pipeline for roughly 30 seconds. While SENSE was stalled, SPECULATE drained the backlog queue, resetting the wait time to near zero before it began climbing again. The artifact appeared at different scene boundaries across models, suggesting a property of the shared GPU scheduler rather than a model-specific fault. Appendix D contrasts the affected traces against clean runs.

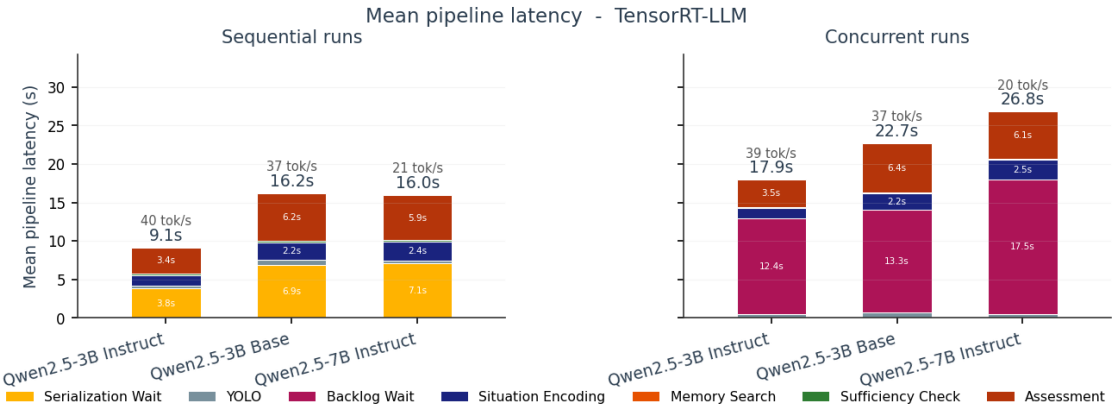


Figure 16: Mean pipeline latency for the models served through TensorRT-LLM

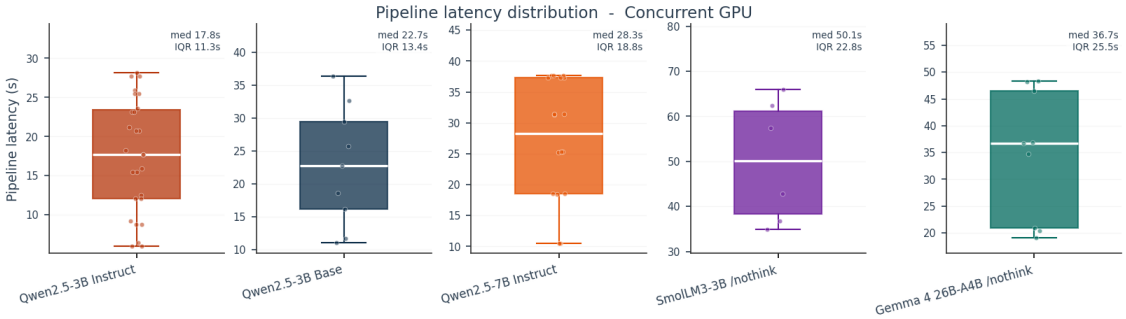


Figure 17: Distribution of per-scene pipeline latency under concurrent GPU scheduling.

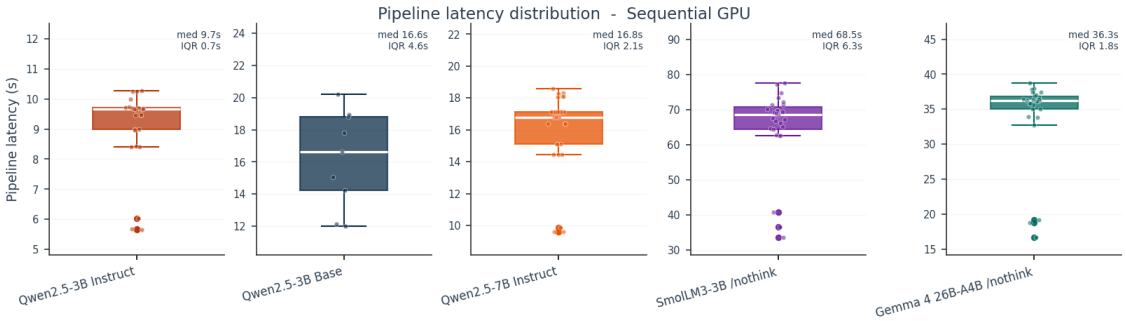


Figure 18: Distribution of per-scene pipeline latency under sequential GPU scheduling.

The backlog accumulates monotonically because SENSE produces scenes faster than SPECULATE can consume them. In an experiment, it was observed that for Qwen2.5-3B-Instruct, queue wait time grew from near zero to 29.9 seconds by scene 9, as shown in Figure 19, exceeding the inference advantage that instruction tuning provides.

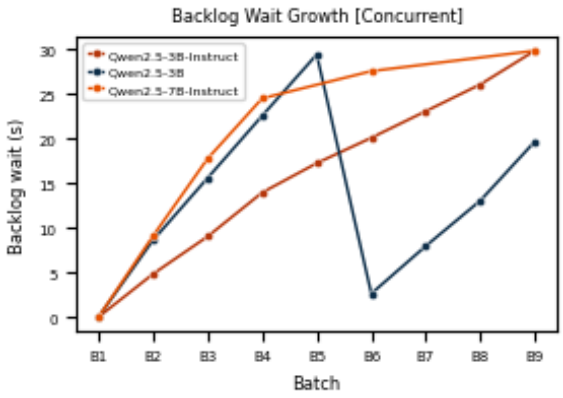
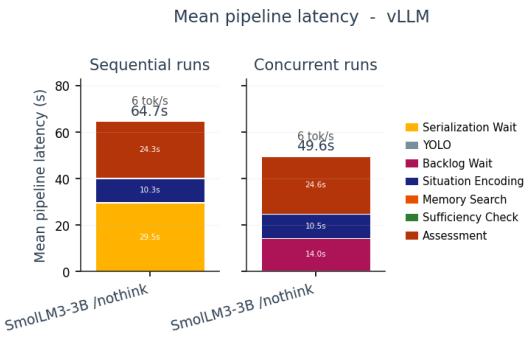


Figure 19: Backlog queue wait time for Qwen2.5-3B-Instruct under concurrent scheduling

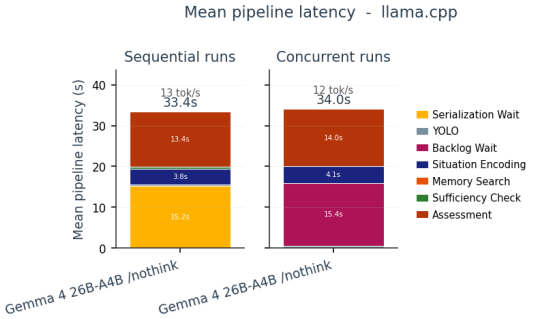
Two strategies were investigated to reduce the backlog pressure under concurrent scheduling. Reducing the queue size or tightening the staleness threshold would cap the wait time, but at the cost of discarding scenes, pushing slower configurations below the viability threshold established in the preceding subsection. Adding a second SPECULATE worker was investigated as an alternative to closing the producer-consumer gap, but it led to a counterintuitive regression in total throughput. Appendix E documents the experiment in full.

The Qwen2.5 instruction-tuned models explicitly target conciseness and instruction-following as reward-model objectives [102], properties that the base model lacks by construction. Both the base and instruction-tuned 3B models share identical architecture and quantization, so the 44 percent latency gap is entirely attributable to the number of tokens each model generates before producing a parseable answer. This verbosity is penalized twice: once in the decode cost of the additional tokens and again in the delay it imposes on perception, since SENSE must wait for SPECULATE to release the GPU before processing the next scene. Qwen2.5-7B-Instruct is comparable to the base 3B despite its lower per-token throughput, because its instruction tuning keeps output sequences short enough to offset the per-token slowdown from the larger weight matrix. The same token-economy explanation accounts for the tighter sequential latency distributions. Instruction tuning produces outputs whose token count varies little from scene to scene, so the number of memory-bound decode steps varies less across inputs, and the model with lower intrinsic variability will provide a tighter worst-case per-scene latency guarantee under any scheduling discipline that bounds or eliminates the backlog component.

The non-TensorRT-LLM configurations exhibit the same backlog effects in a more severe form. Figures 20a and 20b show the mean pipeline latency for SmolLM3-3B-NoThink served through vLLM and Gemma-4-26B-A4B served through llama.cpp. The reported 14-second backlog wait for SmolLM3-3B and 15-second backlog wait for Gemma under concurrent scheduling are means over surviving scenes only. A single vLLM inference takes around 35 seconds per scene and a single llama.cpp inference takes around 18 seconds per scene, while SENSE continues to emit a new scene every 2 to 3 seconds, so every scene that enters the queue while an inference is in flight accumulates wait time at one second per second of SENSE emission rate. Any scene whose wait exceeds the 30-second staleness threshold is dropped before it reaches SPECULATE. It was observed that, across three concurrent runs, 24 of 30 SmolLM3-3B scenes and 21 of 30 Gemma scenes aged out in this way. The surviving scenes are always the first scene, which records a queue wait of zero, plus one or two later scenes that SENSE happens to emit just as the worker becomes free, and that record a wait close to but just under 30 seconds. The scenes that would have recorded waits in the intermediate 10-30 second window never appear in the statistics because the pipeline silently discarded them. The reported figure, therefore, reflects a floor rather than a representative value of the real backlog pressure.



(a) SmolLM3-3B (no-think) served through vLLM.



(b) Gemma-4-26B-A4B (no-think) served through llama.cpp.

Figure 20: Mean pipeline latency for the non-TensorRT-LLM configurations

5.8 Correctness

A model that completes the pipeline within acceptable time budgets may still produce inaccurate risk assessments. An incorrect assessment, whether it fabricates hazards that do not exist, omits hazards that do, or assigns severity and probability ratings inconsistent with the observed scene, compromises the pipeline's value as a safety instrument. Correctness evaluates whether the pipeline's outputs, the scene description, and the FMECA entries reflect the actual scene as captured by the perception stage. The ground-truth references used throughout this evaluation are provided in Appendix K, authored according to the severity, probability, and criticality scales defined in Tables 1, 2, and 3.

Correctness is evaluated from the sequential runs, which complete every scene without queue drops and so give a matched observation for each ground-truth comparison. As established in Section 5.4, the scheduling regime changes when a scene is processed, not the token distribution the model produces, so output quality is unaffected.

Three G-Eval metrics are applied: Scene Description Accuracy, FMECA Faithfulness, and FMECA Ground-Truth Alignment. The task-specific rubrics and scoring criteria for each metric are defined in Appendix B. BERTScore complements G-Eval by providing a deterministic measure of token-level semantic overlap with the reference descriptions, independent of the judge model. Correctness is evaluated in two stages. Scene description accuracy is reported first, followed by FMECA Quality.

5.8.1 Scene Description Accuracy

The scene description is the pipeline's first interpretive output. It translates the structured scene graph from the SENSE stage into a 45-word, first-person natural-language summary that all downstream stages consume. Errors at this stage propagate through the entire pipeline. Scene Description Accuracy therefore evaluates whether the model produces descriptions that are grounded in the detections and safety-relevant.

To fit the token budget, the model must select which elements of the scene are most safety-relevant and discard the rest. Different models, faced with the same perception output, will make different selection choices, and multiple selections can be equally valid as long as each is grounded in what was actually detected. The evaluation was therefore designed to tolerate variation in what the model chose to include while still verifying that what it did include is accurate and safety-relevant. G-Eval's rubric scores the quality of the model's selection and reasoning without penalizing legitimate differences in prioritization against a single reference answer. BERTScore complements this by measuring how much of the reference content the model actually covered,

providing a deterministic, style-independent signal for content overlap that the qualitative rubric does not capture.

The rubric evaluates the description along format compliance, spatial grounding, safety reasoning, and absence of fabricated content. The full evaluation steps and scoring bands are provided in Appendix B. Figure 21 presents the per-scene G-Eval scores, and Table 8 summarizes the BERTScore F1 statistics across the nine evaluation scenes.

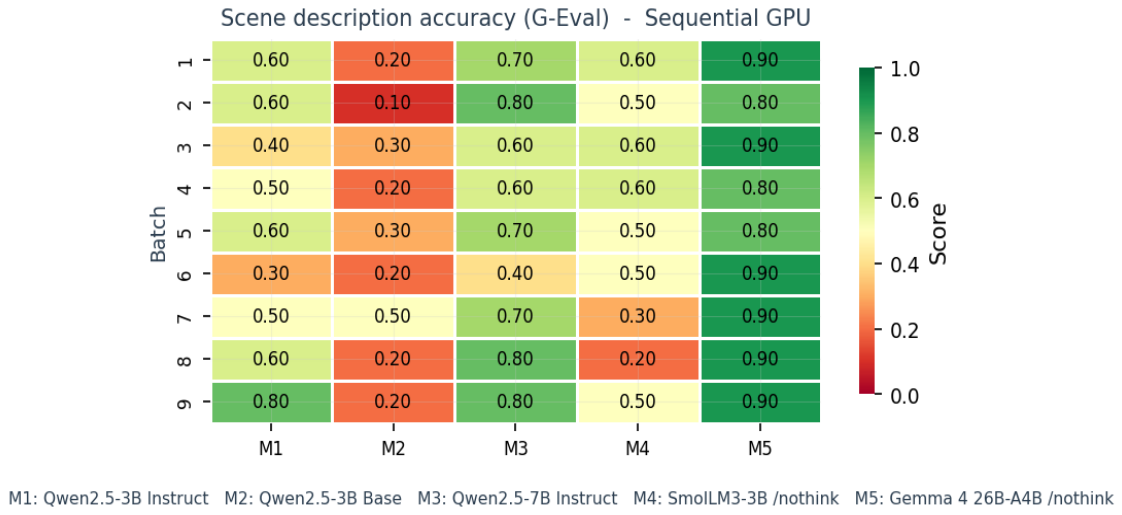


Figure 21: Scene Description Accuracy G-Eval scores per scene

Model	Mean Raw F1	Mean Rescaled F1	σ
Qwen2.5-7B-Instruct	0.906	0.440	0.127
Gemma-4-26B-A4B	0.901	0.415	0.048
Qwen2.5-3B-Instruct	0.887	0.331	0.067
SmolLM3-3B-NoThink	0.884	0.314	0.066
Qwen2.5-3B	0.874	0.256	0.078

Table 8: BERTScore F1 summary statistics across the nine evaluation scenes.

Two patterns in the heatmap merit attention beyond the per-model ranking. The first is that scene description quality is determined by the interaction of model scale and instruction tuning rather than by either factor alone. Gemma-4-26B-A4B, the largest configuration, leads with a mean of 0.87 and never falls below 0.80 on any scene. Among the instruction-tuned configurations, the 7B model outperforms the 3B model, and both score below the 26B configuration. The base Qwen2.5-3B, which lacks instruction tuning at the same 3B scale, fails the threshold on all but one scene. Scale contributes to description quality, but instruction tuning is the prerequisite for the description stage to function at all. The second pattern is that within-model variance tightens as the mean score rises. Gemma-4-26B-A4B maintains consistent scores across scenes with minimal per-scene spread, whereas other models show wider variation and are more sensitive to the specific composition of the scene. A model that reliably produces grounded descriptions tends to do so across scene types, whereas a model near the compliance threshold passes or fails depending on the particular scene graph conditions.

The G-Eval ordering is further shaped by prompt adherence. The rubric instructs the judge to treat scene descriptions as interpretations of the scene graph rather than as transcriptions of it, and directs the judge to treat quoted tracker identifiers and raw metric values as violations of the prompt's interpret-do-not-transcribe rule. An inspection of the scene descriptions produced by the four models shows that Gemma-4-26B-A4B complies with this instruction on every scene. The three other models, not so much. A non-trivial portion of Gemma's G-Eval lead over the other

models is therefore a style-compliance effect rather than a scene-understanding effect, and a more accurate reading of the correctness result is that Gemma adheres to the prompt's interpret-do-not-transcribe instruction most consistently, which the rubric weights heavily.

The G-Eval scores assess the structural quality of each description but do not quantify how much of the reference content each model actually covered. The raw BERTScore F1 values in Table 8 address this gap. All five models score above 0.87, confirming substantial semantic overlap with the reference. The raw scores cluster too tightly to discriminate between models. However, this is a known property of RoBERTa-large, its contextual embeddings assign a baseline F1 of approximately 0.83 even to pairs of unrelated English sentences, compressing all scores into a narrow band near the ceiling [112]. Baseline rescaling normalizes against this floor, so the rescaled values reflect how far each model extends beyond random English toward a perfect reference match. It was observed that the rescaled ranking is dictated by two structural factors. The first is length compliance with the 45-word prompt budget. Qwen2.5-3B writes 69 words on average, 54 percent over budget, diluting precision and scoring lowest. Qwen2.5-7B-Instruct and Gemma-4-26B-A4B, both approximately 42 words, score highest. The second is the paraphrase noise inherent to short constrained outputs. BERTScore compares contextual token embeddings rather than whole descriptions [112], so two scene descriptions that identify the same hazards through different wording score below identical strings.

The rescaled ranking reverses the G-Eval ordering, with Qwen2.5-7B-Instruct leading at 0.440 ahead of Gemma-4-26B-A4B at 0.415. The reversal is consistent with the structural analysis above. Qwen2.5-7B-Instruct mirrors more of the reference vocabulary, which BERTScore rewards directly. Gemma-4-26B-A4B expresses the same safety-relevant content through different word choices. The per-scene variability (σ), defined as the standard deviation of a model's rescaled BERTScore F1 across the nine evaluation scenes, separates the two more decisively. Qwen2.5-7B-Instruct's standard deviation is more than twice that of Gemma-4-26B-A4B. For the downstream pipeline, where the scene description feeds directly into retrieval and risk assessment, consistent output quality provides a tighter worst-case guarantee. On this criterion, Gemma-4-26B-A4B remains the stronger configuration despite trailing on rescaled BERTScore. Taken together, the two metrics serve complementary roles. G-Eval evaluates whether the description is well-structured, grounded, and safety-relevant. BERTScore confirms that the descriptions are on-topic and quantifies where models diverge in vocabulary rather than in content.

The base Qwen2.5-3B's failure is systematic and traces to a fundamental behavioral difference between pretrained and instruction-tuned language models. Qwen2.5-3B is trained to predict the next token given preceding text, not to follow instructions [118]. When it receives a structured prompt containing a scene graph and a request for a concise safety description, it treats the entire prompt as context to continue from rather than a task to complete. The observable consequence is self-repetition of spatial descriptions until the token budget is exhausted, leaving no capacity for the safety reasoning that the prompt requests. Without safety-relevant content in the scene description, the risk assessment stage has no content from which to derive failure modes. This results in blank FMECA entries for three of the nine scenes reported in the viability evaluation. Qwen2.5-3B-Instruct, sharing identical architecture and quantization, produces valid scene descriptions and complete FMECA entries on all nine scenes. The improvement from base to instruction-tuned is roughly twice as large as the subsequent improvement from 3B-Instruct to 7B-Instruct. On these grounds, Qwen2.5-3B is dropped from all subsequent evaluation.

The scene description accuracy scores serve a second function beyond ranking the models. They condition the interpretation of the faithfulness metric that follows. Faithfulness evaluates whether the FMECA entry is derivable from the scene description, a check that is only as informative as the description itself is grounded in the scene. When a model's G-Eval score is high for a given scene, a high faithfulness score on the same scene can be taken at face value, because the description from which the hazard is derived has already been independently validated against a separate rubric. When the G-Eval score is low, a high faithfulness score indicates only that the hazard is consistent with a description that was already judged problematic. The G-Eval heatmap therefore, serves as the first check in the pipeline's validation sequence and determines how much trust can be placed in the downstream metrics for each model and scene.

Across the four retained models, the hierarchy of scene description accuracy is as follows. Gemma-4-26B-A4B leads with the highest mean and the lowest variance. Qwen2.5-7B-Instruct follows with a lower mean but an acceptable pass rate. Qwen2.5-3B-Instruct and SmolLM3-3B-NoThink occupy the middle band with comparable pass rates but higher per-scene variability.

5.8.2 FMECA Quality

The FMECA entries constitute the pipeline's actionable safety output. A correct scene description is necessary but not sufficient. The model must also generate plausible failure modes,effects, and causes, assign appropriate severity and probability ratings, and cover the hazards that a safety engineer would identify. Two sub-metrics, faithfulness and ground-truth alignment, evaluate these properties independently. Read together with the scene description accuracy scores from the preceding subsection (Figure 21), the three metrics describe a chain of responsibility through the pipeline. Scene description accuracy indicates whether the scene was correctly read. Faithfulness establishes whether the structured hazard follows from that description. Alignment establishes whether the structured hazard matches what a safety engineer would have written for the same scene.

Faithfulness measures whether the generated FMECA entries are grounded in the model's own scene description. The judge receives the scene description as input and the serialized FMECA entries as output. No ground truth is provided. The rubric evaluates each entry along referential grounding and rating consistency, with the full evaluation steps and scoring rubric provided in Appendix B.

Ground-truth alignment assesses whether the generated entries capture the hazards identified by a safety engineer in the same scene. The judge receives the structured scene graph as input, the generated entries as output, and the hand-authored FMECA entries as the expected output. The ground-truth entries used by the judge are provided in Appendix K. The rubric evaluates the entries along hazard coverage, rating agreement, and completeness, where omitting a ground-truth hazard incurs a larger penalty than generating additional plausible entries. The ground truth represents one valid assessment, not an exhaustive enumeration. Supplementary entries that are grounded in the scene graph are not penalized.

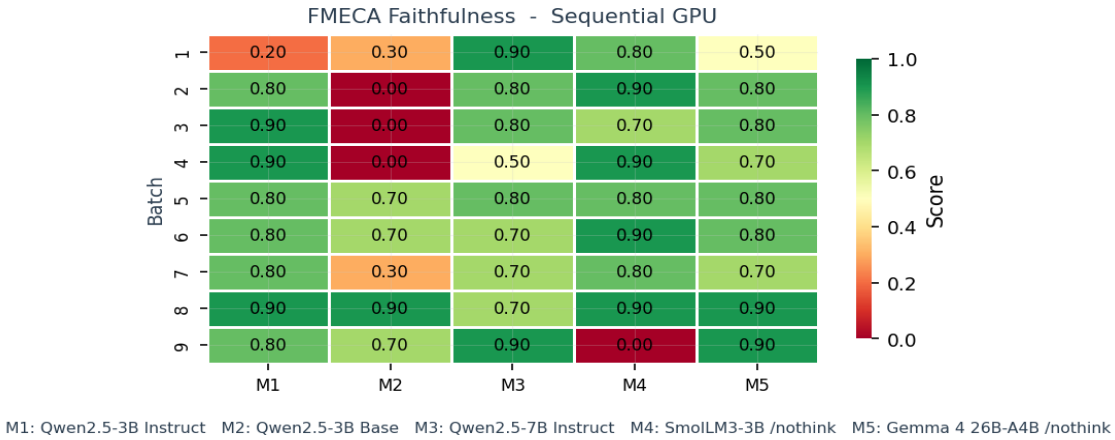


Figure 22: FMECA Faithfulness per scene

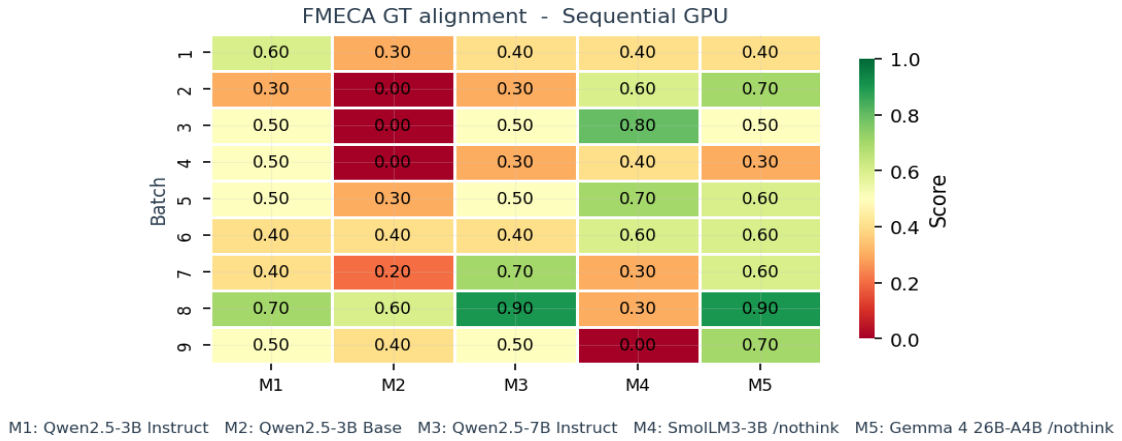


Figure 23: FMECA Ground-Truth Alignment per scene

Faithfulness scores across the four retained models are broadly high, with means between 0.77 and 0.84 (Figure 22). The isolated failures that appear are not concentrated in particular scenes but are distributed differently across models, indicating that they are stochastic rather than scene-driven. The high scores are largely due to the metric's structure rather than model strength. Faithfulness compares two artifacts, the same model produced, namely its scene description and its FMECA entries, with no external reference involved. When the model makes a grounding error in the description and then constructs a risk entry that follows from that error, the faithfulness judge rates the risk as well-supported because the description and the risk assessment agree. Faithfulness, therefore, partly rewards internal coherence regardless of whether that coherence rests on a faithful reading of the scene.

Ground-truth alignment (Figure 23) confronts the model with an outside standard of correctness. The judge compares the generated hazard entries against a human-authored reference that did not inherit any errors from the model's description stage. Scores drop substantially, with most cells falling within the 0.30-0.60 range. Unlike faithfulness, alignment scores correlate with the trajectory. The mid-trajectory scenes, where spatial relationships between the robot, obstacles, and people become more complex, are uniformly difficult across models. The drop-off point, the highest-criticality encounter, is where the models diverge most. Gemma-4-26B-A4B and Qwen2.5-7B-Instruct both reach 0.90, confirming that the larger models correctly identify the hazards flagged by the safety engineer when the threat is most direct. SmolLM3-3B-NoThink drops to 0.30 on both scenes 7 and 8, and its scores degrade progressively toward the end of the trajectory.

Every retained model scores lower on alignment than on faithfulness, and the gap persists across the full trajectory rather than concentrating in a few scenes. Two explanations are consistent with this gap: the generated hazards may differ from the reference only in presentation, or they may contain substantive errors. Reading the G-Eval heatmap (Figure 21) alongside the faithfulness and alignment heatmaps reveals two independent contributors to the gap. For models with low G-Eval scene accuracy, the high faithfulness scores are partly explained by the model's grounding errors at the description stage propagating cleanly into the risk assessment stage. The description is distorted, the hazard is built on the distortion, and the faithfulness judge rewards the self-consistency of the pair without detecting that the pair is anchored to something not present in the actual scene. For models with high G-Eval scene accuracy, this explanation does not hold. When the description stage passes the judge's grounding check and alignment is still well below faithfulness, the divergence must originate at the risk assessment generation stage itself rather than being inherited from the description. The scene is being correctly read, and a reasonable-looking hazard is being generated from it, yet the final structured output still diverges from the ground truth.

Gemma-4-26B-A4B provides the clearest evidence that this second contributor is real. It has the highest G-Eval scene description scores across the trajectory and still shows a substantial gap

between faithfulness and alignment. The model is not failing to understand the scene. It is failing to agree with the ground truth on which object in the scene constitutes the primary hazard, on the severity or probability level that should be assigned. This is a categorization divergence rather than a comprehension failure, and it would not be resolved by scaling the model, changing the serving backend, or tuning decoding parameters alone. It is the gap between what a general-purpose language model can plausibly infer about a scene and what a safety engineer would write for the same scene within a specific categorization taxonomy.

The sufficient and novel path split that the sufficiency check produces provides direct evidence of architectural compensation for this gap. Figure 27 reports the per-scene path each configuration took. Across the nine evaluation scenes, only two scenes were judged to have sufficient FMEA coverage for the objects they contained. Scene 8 describes a single person approaching the robot at close range on the forward path, a pattern for which the seeded FMEA knowledge graph already holds an applicable entry, so every viable model routed it through the sufficient path. Scene 9 describes a similar close-range configuration involving a person and a table, and four of the five viable models also routed it through the sufficient path, with SmolLM3-3B-NoThink being the only model that fell back to novel generation for that scene. The remaining seven scenes involve combinations of objects that the seeded graph does not fully cover, typically chairs and carton boxes, so every viable model routed those through novel generation and produced the hazard from scratch.

On the two sufficient-path scenes, the mean FMECA alignment across the viable models is noticeably higher than on the seven novel-path scenes. The mean faithfulness is also higher on the sufficient path, but the shift is proportionally larger on the alignment path. On either path, the model still generates the failure mode, effect, severity, probability, and audit rationale for the specific scene; only the criticality category is assigned deterministically once severity and probability are chosen. What the sufficient path adds is an in-context exemplar from the FMEA knowledge graph, a safety-engineer-authored entry for a comparable situation, which the prompt places alongside the scene description with an explicit instruction not to copy. The exemplar biases the model's generation toward the safety engineer's framing, namely toward the failure modes a safety engineer would prioritize for the class of object at hand, the effects and causes they would flag, and the phrasing they would use. Severity and probability are not supplied by the exemplar; the model assigns them to each scene based on the observed conditions, following the guideline provided by the system prompt. In novel-path scenes, no such exemplar is available, and the model falls back to its own default taxonomy, which the alignment metric shows diverges more from the safety-engineer reference. Because only two scenes took the sufficient path, these aggregate figures are suggestive; the dedicated evaluation of the retrieval-plus-writeback loop appears in Section 5.10.

The writeback mechanism progressively expands the FMEA knowledge graph during deployment, increasing the fraction of scenes routed through the sufficient path; the dedicated analysis is presented in Section 5.10.

5.9 Determinism

Determinism asks whether the pipeline produces the same output on repeated runs of the same input, a property that LLM-based systems do not provide automatically because sampling introduces stochasticity by construction. ISO 21448 [119] addresses performance insufficiencies in machine-learning-based functions and identifies non-deterministic behavior among the limitations to be controlled, placing such concerns outside the E/E-fault scope of ISO 26262. The determinism evaluation, therefore, measures how much of that stochasticity surfaces in the pipeline's output under the configurations used by the deployed system and explains where it comes from when it does.

Each candidate model is run three times on the same ROS2 bag, and the variation in outputs across those three runs is the quantity of interest. The models evaluated are Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, SmolLM3-3B-NoThink, and Gemma-4-26B-A4B. Qwen2.5-3B, the untuned base model, is excluded because of the correctness values.

5.9.1 Scene Description Consistency

The scene description is the pipeline's first interpretive output, and every downstream stage consumes it directly. Token-level drift at this stage therefore has the potential to propagate through the full SPECULATE pipeline, altering which retrieval entries are matched, which sufficiency decision is taken, and which hazards are framed. The determinism of the scene description was measured first to isolate the earliest point at which run-to-run variability can enter the pipeline.

The consistency of the scene description stage was measured pairwise between the three repeated runs of every scene using Jina v2-small-en cosine similarity and rescaled BERTScore F1, both defined in Section 5.5. In this evaluation, a Jina cosine of 1.0 between two repeated runs indicates identical semantic content, and lower values indicate semantic drift across the runs. A rescaled BERTScore F1 of 1.0 indicates byte-identical token sequences, and lower values indicate textual drift even where the underlying meaning is preserved. The two were reported separately to distinguish between semantic stability, to which Jina cosine is primarily sensitive, and textual stability, which rescaled BERTScore additionally captures; the latter is therefore the more stringent of the two for a determinism evaluation. Where the two metrics diverge, the pattern distinguishes the kind of drift observed. A high Jina cosine paired with a lower rescaled BERTScore indicates that the repeats convey the same meaning through different wording, a case of lexical drift with preserved semantics. The converse pattern, a low Jina cosine with a high rescaled BERTScore, is uncommon in practice because changes in meaning typically induce observable changes at the token level as well.

Figure 24 reports the per-model mean and standard deviation of the pairwise similarity scores across all nine evaluation scenes and all three pairings per scene.

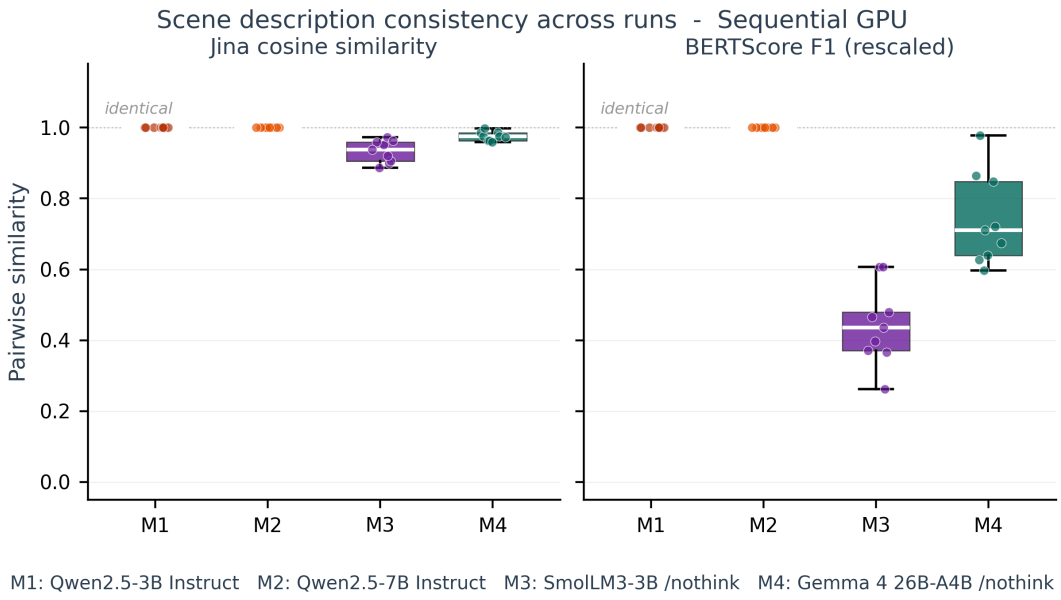


Figure 24: Inter-run scene description similarity across the three repeated runs for each model.

Both Qwen model configurations produce byte-for-byte identical scene descriptions across every repeat on every scene, with Jina cosine and rescaled BERTScore both at exactly 1.00 with zero variance. Gemma-4-26B-A4B produces near-identical semantic content across repeats, with a Jina cosine of 0.98 that indicates the three repeats describe the same scene to within small phrasing differences, but a rescaled BERTScore of 0.74 that indicates the token sequences are not identical. SmolLM3-3B-NoThink has the widest spread on both metrics, with a Jina cosine of 0.93, which still places the three repeats in the same semantic region, and a rescaled BERTScore of 0.44, indicating substantial token-level variation between them.

Jina cosine and rescaled BERTScore did not move together. Across the four models, Jina cosine remained above 0.93 while rescaled BERTScore spanned 0.44 to 1.00, so semantic stability was preserved even where token-level reproduction was not.

5.9.2 FMECA Ordinal Agreement

The consistency of the FMECA generation stage was measured using Krippendorff's Alpha, as defined in Section 5.5, across the three ordinal rating fields in the FMECA schema: severity (I to IV), probability (A to E), and criticality (1 to 5). The three repeats of a scene act as three raters rating the same item, and for each scene, the worst-case rating across that scene's FMECA entries is used, since the highest-risk hazard determines the operational response the pipeline would dispatch. Three Alpha coefficients are computed independently, one per scale.

Figure 25 reports Krippendorff's Alpha for each model on each of the three ordinal FMECA fields.

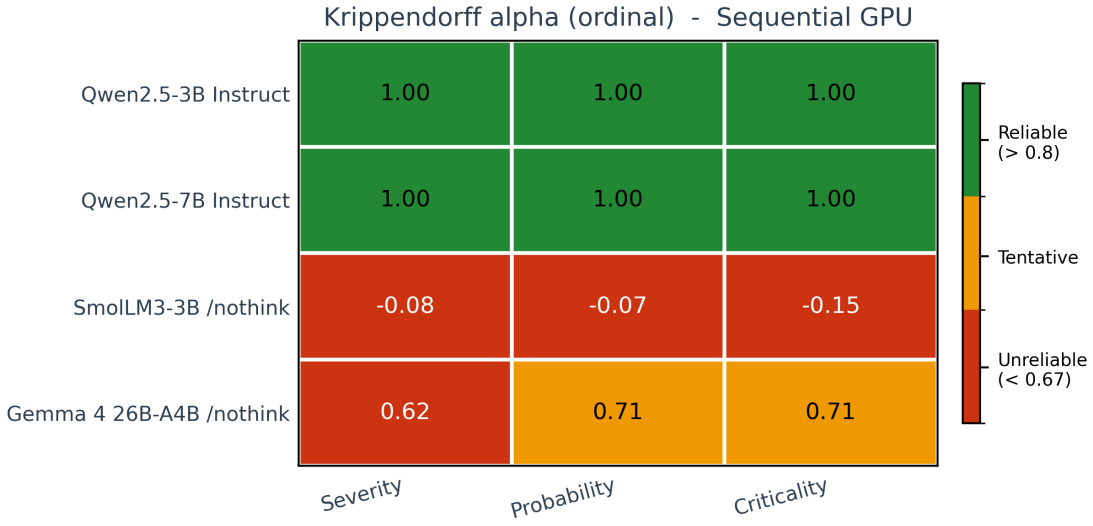


Figure 25: Krippendorff's Alpha on ordinal values across the three repeated runs for each model

Both Qwen checkpoints achieve perfect agreement on all three fields. Gemma-4-26B-A4B's probability and criticality Alphas both fall within the tentative band above 0.667 at 0.71, and its severity Alpha is just below that threshold at 0.62. SmolLM3-3B-NoThink's Alpha is negative on all three fields, at -0.08 on severity, -0.07 on probability, and -0.15 on criticality, indicating that the three repeats disagree more on the ordinal choice than chance alone would predict.

The full mechanism linking a GPU forward pass to run-to-run variation in the ordinal ratings, through logits, the softmax draw, and non-associative floating-point accumulation, is developed in Appendix G, on which the backend-specific analysis below builds. The observed values are governed by the serving backend rather than by any property of the models themselves. All three SPECULATE stages run at low, non-zero temperatures (0.3 for scene description and FMECA generation, 0.2 for the sufficiency check), and the pipeline does not pass an explicit random seed via the API request. Floating-point matrix multiplications on a GPU are not guaranteed to produce identical logits across runs if the kernel launch configuration, the accumulation order, or the memory layout of the computation varies between runs, because floating-point addition is not associative [120]. When the logits drift between runs, the distribution drifts, and the sampler can select different tokens even with the same seed and temperature. The determinism observed on each serving backend, therefore, reflects the degree to which that accumulation path is constrained by the backend's implementation.

The two Qwen checkpoints are served via TensorRT-LLM v0.12, where each token is selected

by a pseudo-random draw from a temperature-scaled distribution. The draw is seeded by the runtime's `randomSeed` parameter, an Int64 parameter documented with a default value of 0 [121]. Because the pipeline sets no explicit `randomSeed`, every request begins from seed 0. At the 0.3 sampling temperature used here, the distribution concentrates around its top token, so the reproducible draw selects the same token at each position on every repeat. The logits that produce the distribution remained stable across the three runs because the prebuilt engine binary was reused unchanged, and the serving pipeline replayed an identical input trace under sequential scheduling with a per-request batch size of 1. In this regime, the kernel chosen for each operation is fixed at build time, and the kernels are exercised on the same shape sequence on every run, so the floating-point reduction paths that would otherwise expose non-associative accumulation [120] are not perturbed. NVIDIA documents this determinism profile as achievable when a single request is in flight with a maximum batch size of 1, at a throughput cost relative to batched serving [122]. The engine workflow described here is the one used by TensorRT-LLM versions prior to 1.0; later releases transitioned to a PyTorch workflow without prebuilt engines, for which determinism behavior has not been publicly characterized. With the draw, the distribution, and the logits all stable, the token sequence is reproduced on every repeat, and this reproduction carries through the FMECA generation stage into identical structured output, which is what the Alpha values of exactly 1.0 on all three ordinal fields reflect.

Gemma-4-26B-A4B is served via llama.cpp with GGUF quantization, which performs matrix multiplication using floating-point arithmetic after dequantizing the weights from their packed GGUF representation. Floating-point matrix multiplication on a GPU is subject to non-associative accumulation when the kernel scheduling or block-dimension layout varies between runs [120], and this is the path through which small drift enters the logits between otherwise-identical repeats. The drift is small enough that the downstream scene descriptions remain semantically stable, as the Jina cosine of 0.98 confirms, but large enough to cause the sampler to pick a different token even at low temperature, as reflected in the rescaled BERTScore of 0.74. Probability and criticality sit at 0.71, while severity sits at 0.62. It was observed that Gemma uses only two of severity's four levels with any frequency, so random pairing would already yield close to perfect agreement on most items, leaving little room for the observed agreement to exceed it. Criticality uses all five levels with no single level dominating, so the baseline is well below perfect agreement, and the observed agreement scores are proportionally higher.

SmolLM3-3B-NoThink is served via vLLM [110] with BitsAndBytes 4-bit quantization [111]. BitsAndBytes stores weights in 4-bit form but dequantizes them to 16-bit floating-point at compute time, so the matrix multiplications are floating-point and therefore subject to the same accumulation non-determinism as the llama.cpp path. Additionally, vLLM's internal scheduler uses paged attention and dynamic memory allocation [110], and while the pipeline sends one request at a time, the scheduling of attention blocks and key-value cache pages can still vary between repeats and change the accumulation order independently of the sampling draw. The two sources of floating-point drift compound into substantially larger token-level variation than Gemma shows, which the rescaled BERTScore of 0.44 indicates, and the variation propagates into the ordinal FMECA ratings through the same mechanism.

To isolate the backend as the cause of these differences, Qwen2.5-3B-Instruct was rerun on vLLM under the same sequential single-request regime that delivered Krippendorff Alpha of 1.0 on TensorRT-LLM. The same model weights served through vLLM produce Alpha of 0.41 on severity, -0.12 on probability, and -0.05 on criticality (Figure 26), all below the tentative reliability band at 0.667. Holding the model fixed and varying only the serving stack reduces ordinal agreement, confirming that the determinism profile observed in the preceding paragraphs is a property of the backend rather than the model.

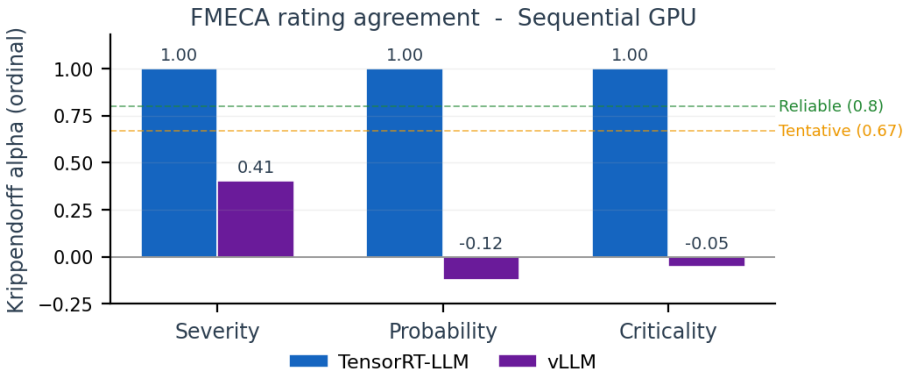


Figure 26: Krippendorff's Alpha TensorRT-LLM vs vLLM

5.9.3 Pipeline Path Agreement

The scene-description and FMECA-rating consistency reported above measure two endpoints of the SPECULATE pipeline; the routing decision between them is a third dimension of determinism. The sufficiency check that assigns each scene to the sufficient or the novel path is set out in Algorithm 2. A scene that routes differently across repeated runs has its hazard entries produced by structurally different mechanisms: retrieved-and-adapted on the sufficient path and generated-from-context on the novel path. The resulting ratings are therefore not directly comparable across those runs.

Path agreement is reported as the fraction of the nine evaluation scenes on which all three repeated runs of a configuration selected the same path. Figure 27 shows the per-scene assignments.

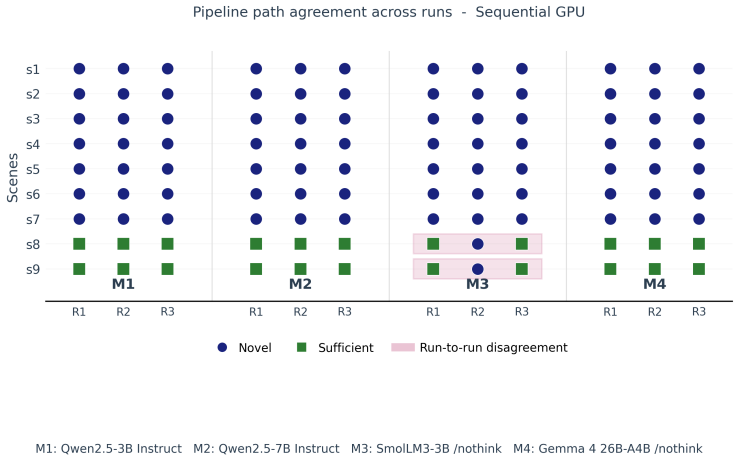


Figure 27: Pipeline path assignments across the three repeated serialized runs for each model.

Both Qwen2.5 configurations and Gemma-4-26B-A4B achieve 100% path agreement. For Qwen, this follows directly from byte-identical scene descriptions: identical strings produce identical embeddings, identical retrievals, and identical sufficiency outcomes. For Gemma, the chain is preserved despite lexical drift in the scene description because retrieval scores on each scene remain within their respective decision regions, and the embedding shifts induced by alternate phrasing do not cross the 0.65 sufficiency threshold.

SmolLM3-3B-NoThink achieves 78% path agreement, splitting on Scenes 8 and 9, the only two scenes the baseline graph covers and whose retrieval scores sit just above the 0.65 sufficiency threshold, which the other configurations route unanimously to the sufficient path. The mechanism is the lexical variability documented in the scene-description consistency results, where a Jina cosine of 0.93 with a rescaled BERTScore of 0.44 across SmolLM3's runs means that while the descriptions convey the same scene content, the word-level differences shift the 512-dimensional embedding by enough to drop those two scenes below the 0.65 threshold and reroute them to the novel path.

The mechanism through which class-level priors enter the path-selection logic is the retrieval-hint expansion of the class-coverage test in Algorithm 2. A class for which neither its label nor any of its retrieval hints appears in the retained FMEA text is added to the uncovered set, and a non-empty uncovered set routes the scene to the novel path. The priors table is loaded as a static configuration at startup, and the same hint expansions are applied to every run, so priors do not introduce run-to-run variability into the routing decision themselves; the splits observed for SmolLM3 above are entirely upstream-embedding-induced. Priors function here as a fixed condition on which embedding drift operates, not as a source of variation.

5.10 Effects of Writeback

The alignment gap between the pipeline-generated hazards and the hand-authored reference was largest when the model generated entries from scratch and narrowed when the two scenes whose retrieved FMEA entries already covered the detected objects were included. This subsection tests whether a modest expansion of the FMEA knowledge graph, simulating the state it would reach after a small number of writeback cycles in a real deployment, produces the two effects the writeback loop is designed to deliver, namely lower end-to-end latency on scenes where the expansion opens a sufficient-branch path that retrieval did not previously have, and higher correctness on the risk entries the pipeline produces on those same scenes.

The experiment is scoped to a single configuration, Qwen2.5-3B-Instruct served through TensorRT-LLM, and the choice follows directly from the preceding subsection. Only the TensorRT-LLM configurations cleared the Krippendorff reliability threshold on all three ordinal rating fields, and a writeback evaluation run against a non-reproducible backend would measure shifts in a metric the pipeline cannot report stably in the first place. Also, Qwen2.5-3B-Instruct is the model whose correctness evaluation left the largest alignment gap with the reference, making it the configuration for which a retrieval-based mechanism has the greatest potential for improvement.

For each of the two GPU scheduling regimes defined in Section 5.4 and for each of the two knowledge-graph configurations (baseline and writeback-enabled), the pipeline is run three times on the same ROS2 bag as before. Latency is measured from concurrent runs because concurrent scheduling is the operationally relevant regime, and latency claims made against the sequential baseline would not translate into the behavior the system exhibits in operation. Correctness is obtained from the sequential runs because this regime completes every scene without queue drops and provides every paired comparison with a matched observation. The categorization quality of the FMECA-style risk assessment, which is what writeback is hypothesized to affect, is independent of the scheduling regime.

In a live deployment the writeback loop returns each novel-path entry to the FMEA graph with a pending status, and that entry enters the retrievable set only once a verifier promotes it. This experiment does not run that loop online. It pre-loads the FMEA graph with the five entries the loop would have produced and a verifier would have promoted, so the graph is in the state it would reach after a small number of writeback-and-promotion cycles. The five were derived from the baseline runs themselves, since whenever the sufficiency check routes a scene to the novel path its class-coverage test reports an uncovered set, namely the object classes for which the retrieved FMEA entries hold no precedent, and one class-level entry per class in that uncovered set was hand-authored and grounded in the scene that raised it. The construction is coverage-targeted by design, activating only on the scenes where the baseline graph did not cover what the robot encountered, so the entries that accumulate over time come from the same distribu-

tion as the entries authored here. What the experiment tests, therefore, is not whether arbitrary additions to the graph help retrieval, but whether the additions the pipeline asks for alter routing and improve output quality in the direction the loop is designed to deliver. All runs hold the top-K retrieval at $K = 5$ across both configurations. K was raised from the earlier default of 3 to 5 to widen the retrieval window so that written-back entries could surface alongside the pre-existing FMEA entries in the knowledge graph; since both the pre-writeback and post-writeback runs use $K = 5$, the comparison remains fair, and any difference observed cannot be attributed to a change in retrieval capacity. Five candidates supply the sufficiency check with enough entries to cover scenes containing multiple object categories while keeping the adapt-from-retrieval prompt concise enough to preserve decoding efficiency.

The first observable effect of swapping the graphs is that six of the nine evaluation scenes transition from the novel path to the sufficient path. Two scenes were already on the sufficient path under baseline and remain there; one scene remains on the novel path because the writeback library does not cover the objects it contains. The six transitioning scenes are those on which the writeback mechanism can affect both latency and correctness, because they are the scenes where a new retrieval path becomes available, and the unchanged scenes serve as controls, isolating the mechanism's effect from incidental differences between the two configurations. The N→S, N→N, and S→S labels used in the result tables refer to these three transition categories, namely N→S for the six transitioning scenes, N→N for the single scene that stays on the novel path, and S→S for the two scenes that were already on the sufficient path.

The path flips themselves are driven by the new entries, not by an incidental reshuffling of the baseline retrieval order. On every one of the six N→S batches, the top-ranked retrieval at $K = 5$ is a writeback entry. It was observed that baseline entries that surfaced in the baseline top-5 are pushed out of the top-5 once the writeback entries are present. The sufficiency check, therefore, consumes a prompt in which the new entries dominate the head of the retrieval list, which is the mechanism through which the writeback library reroutes the pipeline.

5.10.1 Latency

Latency is reported as the end-to-end pipeline time per scene, averaged across the three repeats of each cell, and grouped by transition category.

Regime	Transition	scene IDs	Baseline	Writeback	Δ
Sequential	N→S	S1, S2, S3, S5, S6, S7	5.44 s	5.40 s	-0.03 s
Sequential	N→N	S4	5.70 s	5.63 s	-0.07 s
Sequential	S→S	S8, S9	4.62 s	4.81 s	$+0.19$ s
Concurrent	**N→S**	**S1, S2, S3, S5, S6, S7**	**15.65 s**	**15.36 s**	$\mathbf{-0.29}$ **s**
Concurrent	N→N	S4	16.39 s	15.88 s	-0.52 s
Concurrent	S→S	S8, S9	27.83 s	27.62 s	-0.21 s

Table 9: End-to-end latency per transition category

The N→S row under concurrent scheduling is the primary result, showing that all six flipped scenes complete end-to-end with lower latency in the writeback condition than in the matched baseline, with a mean reduction of 0.29 seconds. The path flip is structural rather than a label change, with the novel-generation LLM call that builds a risk assessment from scratch replaced by an adapt-from-retrieval risk assessment that edits an existing FMEA entry into shape for the current scene. The end-to-end delta, however, is not a stage-level substitution. The stage-level decomposition in Table 10 shows that on the N→S scenes the final stage saves roughly 0.6 seconds when adapt-from-retrieval replaces novel generation, while the sufficiency check on those same scenes grows by roughly 0.5 seconds as on the novel path the sufficiency check exits as soon as it finds the first uncovered object and costs almost no time, but on the sufficient path it has to confirm every detected object against the retrieved entries, which is a full LLM call. The writeback mechanism, therefore, swaps a cheap early-exit check and an expensive novel generation for a

full check and a cheaper hazard assessment, and the two exchanges nearly offset each other; the net inference-layer delta is -0.03 seconds under sequential scheduling and -0.10 seconds under concurrent scheduling.

Stage	Seq. BL	Seq. WB	Δ seq.	Conc. BL	Conc. WB	Δ conc.
Scene description	1.40 s	1.39 s	-0.02	1.42 s	1.41 s	-0.01
Memory search	0.04 s	0.02 s	-0.02	0.02 s	0.02 s	0.00
Sufficiency check	0.07 s	0.61 s	$+0.54$	0.00 s	0.53 s	$+0.53$
Risk Assessment stage	3.92 s	3.38 s	-0.54	4.09 s	3.46 s	-0.62
Queue wait	0.00 s	0.00 s	0.00	10.13 s	9.95 s	-0.18
Lock wait	4.02 s	3.89 s	-0.13	0.00 s	0.00 s	0.00
YOLO	0.62 s	0.62 s	-0.01	0.69 s	0.69 s	0.00
End-to-end	**5.44 s**	**5.40 s**	$\mathbf{-0.03}$	**15.65 s**	**15.36 s**	$\mathbf{-0.29}$

Table 10: Stage-level latency decomposition on the six N→S scenes, averaged across runs

The plausible explanation for the larger end-to-end improvement under concurrent scheduling lies in prior-conditioning, which also accounts for the observed queue-wait reduction. Under writeback, the retrieval set returned to the risk-assessment stage contains entries that were themselves generated from structurally similar scenes seen earlier in the recording, and these entries act as strong priors that fix the failure-mode label and the phrasing conventions before decoding begins. If this conditioning shortens the generated output, then part of the -0.62 second final-stage delta would be attributable to prior conditioning rather than to the path flip alone. The same per-scene inference saving then propagates under concurrent scheduling, leading to faster per-scene service time, which reduces queue pressure on subsequent scenes, as shown in Table 10 as a queue-wait reduction of approximately 0.18 seconds. The present experiment cannot isolate the prior-conditioning component from the structural exchange between novel generation and adapt-from-retrieval, and the attribution is therefore speculative.

The attribution of the N→S improvement to the writeback mechanism hinges on the N→N and S→S control rows, read across both scheduling regimes. N→N is the single scene whose objects fall outside the writeback library, so it stays on the novel path under both configurations. Its end-to-end latency falls slightly under writeback in both regimes, which rules out any global cost from enabling the mechanism on scenes it cannot reroute. S→S does more work as a control when its two regime values are read together rather than in isolation. Under sequential scheduling, the pair mean is $+0.19$ seconds slower under writeback, driven by a consistent slowdown on S8 that a partial speedup on S9 does not fully offset, with the per-scene breakdown in Appendix H. Sequential scheduling carries no queue to relieve, so the $+0.19$ second figure captures only the direct effect of reshuffling the top-K retrieval context on already-sufficient scenes, and it shows that reshuffling the top-K retrieval context on its own does not speed up these scenes. Under concurrent scheduling, the same pair flips to -0.21 seconds faster under writeback, comparable in magnitude to the N→S mean reduction. The per-cell decomposition in Appendix H attributes this concurrent improvement to a mix of queue relief inherited from upstream N→S path flips and a direct inference-side gain on one of the two scenes, and confirms that the queue-relief half of the mechanism propagates as expected to scenes independent of the main result. The two controls together rule out a global regression from enabling writeback and an indiscriminate speedup from context reshuffling, leaving the -0.29 second N→S reduction attributable to the path flip itself and the queue relief it propagates downstream.

5.10.2 Correctness

Three G-Eval metrics are reported across the 27 paired observations per condition (9 scenes $\times$ 3 repeats), each scored by GPT-4o under the rubrics defined in Appendix B, with the per-cell standard deviation in parentheses.

Metric	Baseline	Writeback	Δ absolute
Scene Description Accuracy	0.303 (± 0.239)	0.303 (± 0.239)	0.000
FMECA Faithfulness	0.656 (± 0.255)	**0.800** ($\mathbf{\pm 0.096}$)	$\mathbf{+0.144}$
FMECA Ground-Truth Alignment	0.344 (± 0.153)	**0.478** ($\mathbf{\pm 0.115}$)	$\mathbf{+0.134}$

Table 11: Correctness metrics under baseline and writeback-enabled configurations

The Scene Description Accuracy delta is zero by construction. The determinism subsection established that Qwen2.5-3B-Instruct served through TensorRT-LLM produces byte-identical output on the Jetson, so the description written under baseline and under writeback is the same string, and the G-Eval judge at temperature 0.0 scoring the same string returns the same score. If writeback were producing its downstream effects through a generic behavioral change in the model rather than through retrieval, that change would also appear at the description stage. It does not, so the downstream deltas can be attributed to the stage at which retrieval contributes.

Both judge-scored risk metrics rise by roughly comparable amounts under writeback, and both deltas exceed the per-cell standard deviation in the writeback condition. The faithfulness improvement alone could be read as the risk assessment simply becoming more internally consistent with its own description, which would be a modest and somewhat circular gain, since faithfulness evaluates the risk assessment against the model's own description rather than an external reference. What makes the result meaningful is that alignment improves in parallel by a comparable amount. Alignment is the metric that checks the hazard against a reference ground-truth that the model did not author, so an improvement there can only come from the model's categorization moving closer to the safety engineer's choice. The only sources of domain-expert categorization in the pipeline are the knowledge graph and the writeback condition.

The simultaneous gains, therefore, trace back to the same mechanism. Writeback supplies risk templates that are both structurally well-formed and expert-aligned, and the adapt-from-retrieval risk assessment call inherits both properties into the generated output.

The second-order finding in Table 11 is the narrowing of the per-scene spread. The baseline FMECA Faithfulness has a per-cell standard deviation of 0.255, indicating that individual scenes vary widely in how well the risk assessment follows from the description; under writeback, the same spread narrows to 0.096, and the alignment spread narrows from 0.153 to 0.115. When the hazard is being adapted from a retrieved entry, the range of plausible outputs is narrower than when it is being invented from scratch, and the per-scene variance drops as a consequence. Writeback, therefore, raises not only the mean correctness but also the consistency of that correctness across scenes.

The experiment did not cover the alternative path of populating the normative store with targeted incident reports or comparable external documents. The architecture places the FMEA knowledge graph and the normative store in distinct roles. The graph is queried first, and the normative store is consulted only when the graph fails to cover the present scene. Both stores share an embedding space, so what differs between writeback and incident-report ingestion is the store the entries populate and the retrieval branch they bear on, not the retrieval mechanism. Incident reports drawn from external deployments would supply content that is speculative against this deployment's operating environment, with classes that need not match those the system encounters in service. Writeback entries instead come from scenes the deployed system actually encountered, are FMECA-schema-conformant by construction, and target precisely the classes the deployed system could not cover. A failure of the deployed system itself would produce an incident from which a report could be authored, and that report would not be speculative against this deployment. But it would still sit on the secondary path, complementary to writeback rather than equivalent to it.

6 Discussion and Conclusion

An autonomous robot tasked with operational risk inference is subject to three requirements that must be satisfied together. A framework must be fast enough to keep pace with a sensor stream and the real time deadlines defined by the operational constraints, accurate enough to support the action that follows from its output, and consistenet enough that an auditor can correlate its output with the input that produced it. Each requirement is necessary and none is sufficient on its own. The measurements in Chapter 5 were designed to test the proposed framework against each of them on the hardware for which it was intended. The pattern they form has consequences for how and for what purpose the proposed framework can be deployed, and those consequences, rather than the measurements themselves, are the subject of this chapter.

Reproducibility is taken up first. Whether the proposed framework returns the same output for the same input is a property of the serving backend alone, and once fixed it conditions every subsequent finding, because a correctness or latency measurement on a configuration whose output is not reproducible describes that configuration on a single run and nothing beyond. Correctness and latency are therefore read against a fixed deterministic backend.

Figure 28 profiles the four configurations retained after viability filtering along the three requirements simultaneously, and the subsections that follow expand each axis in turn.

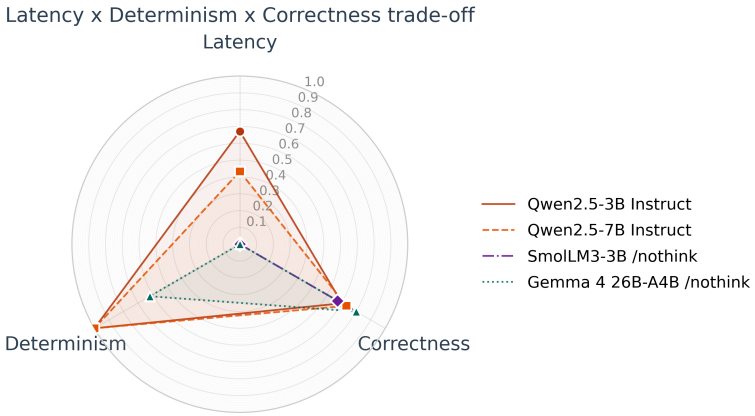


Figure 28: Latency, correctness, and determinism tradeoffs

Of the four configurations that cleared viability, only the TensorRT-LLM path returned byte-identical output across repeated runs (Figure 25), and the recommendation for any deployment bound by a safety audit obligation is therefore single and straightforward. The reproducible configuration on this hardware is TensorRT-LLM with INT4 AWQ quantization, no explicit seed, single-request serving, and low sampling temperature. The two remaining backends produced run-to-run drift at the same temperatures, and a deployment built on either would carry that drift into production in a form an audit process cannot accommodate. The recommendation carries a trade-off with it, TensorRT-LLM on the Orin is frozen at version 0.12, which supports only a subset of model architectures. A deployment that fixes this backend for reproducibility inherits the architecture-support envelope of the frozen branch, and that envelope is not expected to widen.

Correctness depends on instruction tuning first and scale second, in that order of importance. The proposed framework's reliance on structured output at every stage is what makes the tuning dependence decisive. The untuned base model tested in this thesis returned text the downstream stages could not parse on a non-trivial share of scenes, whereas its instruction-tuned counterpart, built from identical weights and quantization, cleared the same scenes without failure. The claim here is therefore not that base models universally fail at structured-output tasks, but that instruction tuning was the load-bearing property in the configurations this thesis measured. Once instruction tuning is in place, scale lifts scene-description accuracy and ground-truth alignment monotonically, visible on the correctness axis of Figure 28.

Correctness is also not a property of the weights and the backend in the way reproducibility is. The proposed framework conditions the model's output at every scene on prior information the deployment engineer controls directly, and three levers are available. The first is the class-level priors the designer declares for each object class the perception stage can report, listing the risk tags, retrieval hints, and typical associations the model should consider when a class appears in a scene. The second is the FMEA knowledge graph the retrieval stage queries, whose coverage decides which scenes follow the sufficient path with a domain expert authored exemplar in context and which fall back onto novel generation. The third is the normative documents store the novel path consults for contextual grounding, into which the designer can place targeted material the deployment is expected to reason against. All three represent what the designer knows about the operating environment before the robot is deployed, and the writeback evaluation (Section 5.10) showed that ground-truth alignment and the per-scene consistency of the risk entries both rise in step with the share of scenes the knowledge graph covers. A deployment whose operating environment has been well characterized at design time therefore begins with sharp priors, a dense graph, and normative coverage aligned to the expected environment, and accumulates writeback entries slowly. A deployment into a less well-characterized environment begins with weaker priors and a sparser graph, routes more scenes through novel generation, and must pass through more iterations of the writeback loop before its alignment figures converge on the levels the sufficient path produces.

Latency and correctness trade off through scale, but not directly. Decode on the Orin is memory-bandwidth-bound, every decode step loads the full weight set from the shared LPDDR5 pool, so per-token cost rises with parameter count. Per-scene latency is that per-token cost multiplied by the number of tokens the model emits before its answer parses. Scale raises scene-description accuracy and ground-truth alignment monotonically, as the correctness axis of Figure 28 shows, and raises the per-token term simultaneously. Instruction tuning modulates the second term independently of the first. The 3B base model and its instruction-tuned counterpart share weights and quantization, and the instruction-tuned variant completes a sequential pipeline pass in 9.1 seconds against the base model's 16.2 seconds, a 44 percent reduction attributable entirely to shorter outputs (Section 5.7). The same mechanism keeps the 7B instruction-tuned model within latency parity of the 3B base, since the shorter sequence offsets the heavier per-token decode. A deployment that fixes a latency budget therefore chooses along two axes, not one, with instruction tuning acting as the lever that buys scale without paying its latency cost in full.

The serving backend sets the per-token term that multiplies through the chain above. TensorRT-LLM reaches roughly 40 tokens per second on the 3B-Instruct configuration against 5.5 to 12 tokens per second on the alternative backends measured (Table 7), and the compiled-engine property that produced byte-identical output is the same property that produced this throughput. The latency recommendation therefore identifies the same backend the determinism recommendation identified.

The measurements place every tested configuration outside the response window a run-time risk inference function would require under concurrent scheduling, where perception and reasoning share the accelerator. The proposed framework as instantiated on this hardware is therefore not a component that closes a safety loop around the robot's actuation. TensorRT-LLM on the Orin has been frozen on the `v0.12.0-jetson` branch since November 2024 [123]; mainline TensorRT-LLM has since moved onto Hopper and Blackwell code paths that the Orin's Ampere silicon cannot execute, and its FP8 kernels require hardware the Ampere generation does not provide. JetPack 7.0 and 7.1, which ship the successor TensorRT Edge-LLM stack, support only Jetson Thor [124]; the backport of Edge-LLM to the Orin has not shipped as of the date of this thesis. The throughput reported here is the ceiling of the prior-generation NVIDIA software stack the Orin can currently host, and an operation-time instantiation of the proposed framework therefore requires either the next-generation Jetson or a non-Jetson platform with a supported inference runtime.

What the proposed framework is suited to follows from inverting that verdict. The same pipeline that is too slow to arbitrate commands runs fast enough to consume a recording offline, and the artifacts it produces constitute an auditable record of what the robot saw and what a safety reasoning process inferred from it. Three uses follow from that record, and the deployment recommendation of this thesis rests on them.

The first, is as a Post-Market Monitoring function within the Risk Management System that Article 9 of the EU AI Act requires every provider of a high-risk AI system to establish, and whose continued operation after the system is placed on the market Article 72 formalizes as an obligation on that provider [8]. The proposed framework satisfies the properties such a function requires on this hardware. The reproducible backend identified above makes every inference reproducible on demand by an auditor; the writeback loop returns operational observations to the FMEA knowledge graph that the risk management process reads from; and the offline operating mode imposes no real-time obligation on the control stack it runs beside.

The second use is as a scenario accumulator for continuous improvement of the safety case and expansion of the risk registry. The same writeback signal that drives the Post-Market Monitoring loop above delivers, over deployment time, a growing library of verified hazard entries. Each entry broadens the coverage of the risk registry that subsequent deployments inherit and supplies structured supervision for a successor model simultaneously. That said the actual realization of this requires swapping of the SENSE stage to a mechanism that is not bounded by vocabulary, like the one highlighted in section 7.

The third use is as a verification instrument for the design-time assumptions the Risk Management System itself rests on. Risk artifacts authored before deployment rest on assumptions about the operating environment. The proposed framework, run over a representative recording, surfaces the classes of situation the deployed robot actually encountered and the hazards its reasoning process associated with them, and the comparison between the pre-deployment assumption set and this operational record either substantiates the design-time analysis or flags the deltas it did not anticipate.

The deployment recommendation that follows from the measurements in this thesis is therefore not that the proposed framework replaces the robot's control stack, but that it runs alongside it, recording and reasoning, and returning the artifacts that the Risk Management System, the Post-Market Monitoring obligation, and the design-time assumption base require to the processes that still largely produce them by hand today.

7 Future Scope

Five extensions to the proposed framework follow from the measurements and the deployment analysis. Each targets a specific finding of this thesis and opens a distinct development path.

7.1 Task-Specific Fine-Tuning for Hazard Identification

The correctness analysis in Section 5.8 isolated a categorization divergence between the generated FMECA entries and the expert ground truth that persisted even on configurations that read the scene correctly. The divergence is not a comprehension failure but a mismatch between the default hazard taxonomy a general-purpose language model produces and the categorization conventions a safety engineer applies to the same scene, and scaling the base model alone does not close it. Supervised fine-tuning on a corpus of expert-authored FMECA entries is the direct lever, since it teaches the model the taxonomy the risk management process already uses. A side effect of the same pass is that outputs shorten as probability mass concentrates on the phrasing the training set exhibits, which cuts per-scene latency as a by-product. The writeback library accumulated during deployment is the natural training corpus, coupling this extension to the deployment loop the thesis recommends.

7.2 Perception and a Tighter Loop

The scene summary handed to SPECULATE is a list of whole-object records, and the spatial relations attached to it range over pairs of whole objects. Hazards that reside in the composition between objects, namely a small tool resting on a workbench, a cable bundle running through a cable tray, or a test tube held inside an open container, fall outside this representation, since the constituents either share a bounding box with their container or are too small in pixel extent to receive a separate box at the resolution the present construction uses. Routing only the subregions whose class priors mark them as containers, namely workbenches, trays, and shelves, through a higher-resolution second pass would let SPECULATE reason over hazards that live in the composition rather than the geometry of two whole objects.

A second extension addresses the closed loop the framework offers at operation time. The current configuration accumulates velocity history within a thirty-frame window and resets the tracker between scenes, so downstream reasoning operates on a settled scene at window granularity, with the operation-time response set by the window length plus the reasoning latency reported in Chapter 5. A streaming service that consumes detection events as they arrive moves the framework from windowed integration toward an event-driven loop in which scene understanding accumulates continuously rather than being rebuilt from a fresh thirty-frame buffer at every window close. The class-level priors table that SENSE already maintains pairs every class with a set of pre-approved hazard tags, so each new detection seeds a running picture of the scene with the hazards associated with its class, and subsequent detections revise that picture as more of the scene becomes visible. This becomes a dynamic risk annoted scene graoh, similar to the one discussed in [125]. The windowed FMECA generation that the correctness analysis examined would still be needed, both because FMECA expects a stable description of a settled scene rather than a running commentary and because the SPECULATE pipeline that produces it is dominated by a language-model call whose latency cannot be paid per detection event, but a windowed pass could plausibly run in parallel against the accumulated state rather than rebuilding the scene description from frame zero.

A third bound on the perception layer follows from the open-vocabulary detector itself. SENSE prompts YOLO-E with a fixed class list at export time, and writeback expands the FMEA knowledge graph until every prompted class is covered by precedent. Once that asymmetry between detected classes and registry coverage closes, novel-hazard entries stop entering the registry. Sustaining novel-hazard discovery beyond that point requires a perception layer whose label set is not fixed at export time. Schiel [125] addresses this with an online VLM-driven scene-graph

pipeline that produces open-text node labels and pose-related edge relations from RGB-D observations, with hallucination mitigation targeted at smaller VLMs. Substituting this scene-graph layer for the YOLO-E and H-grid construction used here would restore the class-set asymmetry that sustains writeback expansion and would supply the relational structure that the present per-pair spatial relations only approximate.

7.3 Offloading Normative Retrieval to a Near-Edge Service

The Jetson AGX Orin shares a single LPDDR5 memory pool across the CPU, the GPU, and the deep-learning accelerators, and the latency analysis in Chapter 5 established that the decode path on this platform is bound by memory bandwidth rather than compute. The normative documents store currently runs on the same device, consuming memory and bandwidth the shared pool would otherwise allocate to decoding, and the corpus size is capped by what the Jetson can host alongside the perception stack and the language model. Moving the store and its retrieval logic onto a near-edge server, with a language model on that server arbitrating retrieval, relieves the on-board memory pressure and lifts the ceiling on corpus size. The cost is one network round trip per novel-path scene, which is tolerable at the offline operating mode this thesis recommends for the deployment. Offloading the retrieval and its arbitration also reduces the compute burden on the on-device language model, which then handles a smaller prompt and shorter generated context per scene.

7.4 Automating Writeback Verification with a Larger Fine-Tuned Model

The writeback loop evaluated in Section 5.10 requires a human verifier to confirm each generated FMEA entry before the knowledge graph accepts it, and the correctness gains the loop delivers therefore accrue only as fast as the verifier can audit the output. At deployment scale, where the novel-path rate on a newly characterized environment is high, this bottleneck sets the rate at which the sufficient-path share grows and the alignment metrics improve. A larger fine-tuned language model, hosted off the Jetson and served against the deployment's write queue, could serve as a first-pass verifier that approves entries whose structure and content match the expectations the fine-tune installed and flags the remainder for human review. Loop throughput then rises by the ratio of auto-approved to flagged entries, and the corpus that trains the hazard-identification fine-tune also trains the verifier, so the two models share the training pipeline.

7.5 Deployment on Jetson Thor

Porting the proposed framework to Thor produces a direct measurement of per-scene latency on a platform with greater memory bandwidth and compute capacity than the Orin, and on a software stack the Orin cannot host, namely JetPack 7.1 with the TensorRT Edge-LLM SDK [124, 126]. TensorRT Edge-LLM is the inference SDK NVIDIA ships for the Thor-class embedded platforms, namely Jetson Thor and DRIVE AGX Thor; the v0.12.0-jetson branch of TensorRT-LLM that the Orin runs has received no feature updates since its open-sourcing in November 2024 [123]. The newer SDK restores access to the kernel-optimization track of mainline NVIDIA inference development, including quantization formats such as FP8 that depend on hardware features Thor's Blackwell-architecture GPU provides and the Orin's Ampere silicon does not. The port quantifies how much the additional bandwidth and the newer kernel paths together shorten the per-scene decode latency that this thesis identified as the dominant cost on the Orin, and admits language-model architectures the Orin's frozen TensorRT-LLM 0.12 could not serve, including the Qwen3-Thinking configuration evaluated in this thesis, into the comparison set.

Bibliography

[1] Lars Kunze et al. 'Artificial Intelligence for Long-Term Robot Autonomy: A Survey'. In: *IEEE Robotics and Automation Letters* 3.4 (2018), pp. 4023–4030. DOI: 10.1109/LRA.2018.2860628.

[2] Panos J. Antsaklis. 'Autonomy and metrics of autonomy'. In: *Annual Reviews in Control* 49 (2020), pp. 15–26. DOI: 10.1016/j.arcontrol.2020.05.001.

[3] Manuel Müller et al. 'Situational Risk Assessment Design for Autonomous Mobile Robots'. In: *Procedia CIRP* 109 (2022), pp. 72–77. DOI: 10.1016/j.procir.2022.05.216.

[4] Patrick Wolf, Rasmus Adler and Daniel Schneider. 'Situational Risk Awareness for Autonomous Robots: Maximizing Safety and Operational Availability Across Domains'. In: *1st German Robotics Conference (GRC)*. 2025.

[5] Jan Reich and Mario Trapp. 'SINADRA: Towards a Framework for Assurable Situation-Aware Dynamic Risk Assessment of Autonomous Vehicles'. In: *2020 16th European Dependable Computing Conference (EDCC)*. IEEE, 2020, pp. 47–50. DOI: 10.1109/EDCC51268.2020.00017.

[6] Responsible AI Collaborative. *AI Incident Database*. Public catalogue of reported AI-related incidents. 2024. URL: https://incidentdatabase.ai (visited on 4th May 2026).

[7] Peter Slattery et al. 'The AI Risk Repository: A Comprehensive Meta-Review, Database, and Taxonomy of Risks From Artificial Intelligence'. In: (2024). arXiv: 2408.12622 [cs.AI]. URL: https://arxiv.org/abs/2408.12622.

[8] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L series. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/1689/oj. (accessed: 22.04.2026).

[9] International Organization for Standardization and International Electrotechnical Commission. *ISO/IEC Guide 51:2014: Safety aspects — Guidelines for their inclusion in standards*. Guide ISO/IEC Guide 51:2014. Geneva, Switzerland: International Organization for Standardization, Apr. 2014. URL: https://www.iso.org/standard/53940.html. (accessed: 01.05.2026).

[10] International Organization for Standardization. *ISO 12100:2010: Safety of machinery — General principles for design — Risk assessment and risk reduction*. Standard ISO 12100:2010. Geneva, Switzerland: International Organization for Standardization, Nov. 2010. URL: https://www.iso.org/standard/51528.html. (accessed: 01.05.2026).

[11] International Organization for Standardization. *ISO 13482:2014: Robots and robotic devices — Safety requirements for personal care robots*. Standard ISO 13482:2014. Geneva, Switzerland: International Organization for Standardization, Feb. 2014. URL: https://www.iso.org/standard/53820.html. (accessed: 01.05.2026).

[12] Nancy G. Leveson and John P. Thomas. *STPA Handbook*. Tech. rep. Cambridge, MA, USA: MIT Partnership for Systems Approaches to Safety and Security, Mar. 2018. URL: https://psas.scripts.mit.edu/home/get_file.php?name=STPA_handbook.pdf.

[13] Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011. DOI: 10.7551/mitpress/8179.001.0001.

[14] Andre Lavor Dakwat and Emilia Villani. 'System safety assessment based on STPA and model checking'. In: *Journal of Loss Prevention in the Process Industries* 61 (2019), pp. 313–326.

[15] Jon Arne Glomsrud et al. 'STPA for autonomous systems'. In: *Safety Science* 132 (2020), p. 104939.

[16] International Electrotechnical Commission. *IEC 60812:2018: Failure modes and effects analysis (FMEA and FMECA)*. Standard IEC 60812:2018. Geneva, Switzerland: International Electrotechnical Commission, Aug. 2018. URL: https://webstore.iec.ch/publication/26359. (accessed: 01.05.2026).

[17] Dariusz Plinta, Ewa Golinska and Ľuboslav Dulina. 'Practical application of the new approach to FMEA method according to AIAG and VDA reference manual'. In: *Communications — Scientific Letters of the University of Zilina* 23.4 (2021), B325–B335. DOI: 10.26552/com.C.2021.4.B325-B335.

[18] Joseph DeRosier et al. 'Using Health Care Failure Mode and Effect Analysis: The VA National Center for Patient Safety's Prospective Risk Analysis System'. In: *The Joint Commission Journal on Quality Improvement* 28.5 (May 2002), pp. 248–267. DOI: 10.1016/S1070-3241(02)28025-6.

[19] Anam Parand et al. 'A prospective risk assessment of informal carers' medication administration errors within the domiciliary setting'. In: *Ergonomics* 61.1 (2017), pp. 104–121. DOI: 10.1080/00140139.2017.1330491.

[20] Y. J. Ng et al. 'Application of an adapted FMEA framework for robot-inclusivity of built environments'. In: *Scientific Reports* 12.1 (2022), p. 3408. DOI: 10.1038/s41598-022-06902-4.

[21] P. Schmitt et al. 'Introducing the ML FMEA'. In: *WCX SAE World Congress Experience*. SAE Technical Paper 2025-01-8078. Detroit, MI, 2025. DOI: 10.4271/2025-01-8078.

[22] Jérémie Guiochet. 'Hazard analysis of human-robot interactions with HAZOP-UML'. In: *Safety Science* 84 (2016), pp. 225–237. DOI: 10.1016/j.ssci.2015.11.024.

[23] Homa Alemzadeh et al. 'Systems-Theoretic Safety Assessment of Robotic Telesurgical Systems'. In: *34th International Conference on Computer Safety, Reliability and Security (SAFECOMP)*. 2015. DOI: 10.1007/978-3-319-24255-2_16.

[24] Jérémie Guiochet, Mathilde Machin and Hélène Waeselynck. 'Safety-critical advanced robots: A survey'. In: *Robotics and Autonomous Systems* 94 (2017), pp. 43–52. DOI: 10.1016/j.robot.2017.04.004.

[25] Pericle Salvini, Diego Paez-Granados and Aude Billard. 'On the Safety of Mobile Robots Serving in Public Spaces: Identifying Gaps in EN ISO 13482:2014 and Calling for a New Standard'. In: *ACM Transactions on Human-Robot Interaction* 10.3 (2021), pp. 1–27. DOI: 10.1145/3442678.

[26] Pericle Salvini, Diego Paez-Granados and Aude Billard. 'Safety Concerns Emerging from Robots Navigating in Crowded Pedestrian Areas'. In: *International Journal of Social Robotics* 14.2 (2022), pp. 441–462. DOI: 10.1007/s12369-021-00796-4.

[27] Sami Haddadin. 'Physical Safety in Robotics'. In: *Formal Modeling and Verification of Cyber-Physical Systems*. Ed. by Rolf Drechsler and Ulrich Kühne. Wiesbaden: Springer Vieweg, 2015, pp. 249–271. DOI: 10.1007/978-3-658-09994-7_9.

[28] International Organization for Standardization. *ISO/TS 15066:2016: Robots and robotic devices — Collaborative robots*. Technical Specification ISO/TS 15066:2016. Geneva, Switzerland: International Organization for Standardization, Feb. 2016.

[29] S. Robla-Gómez et al. 'Working Together: A Review on Safe Human-Robot Collaboration in Industrial Environments'. In: *IEEE Access* 5 (2017), pp. 26754–26773. DOI: 10.1109/ACCESS.2017.2773127.

[30] Zhenrui Liu et al. 'Dynamic risk assessment and active response strategy for industrial human-robot collaboration'. In: *Computers & Industrial Engineering* 141 (2020), p. 106302. DOI: 10.1016/j.cie.2020.106302.

[31] Georgios Katranis et al. *Dynamic Risk Assessment for Human-Robot Collaboration Using a Heuristics-based Approach*. 2025. arXiv: 2503.08316 [cs.RO].

[32] Robotic Industries Association. *ANSI/RIA R15.08-1-2020: American National Standard for Industrial Mobile Robots — Safety Requirements — Part 1: Requirements for the Industrial Mobile Robot*. Standard ANSI/RIA R15.08-1-2020. Ann Arbor, MI: Association for Advancing Automation (A3), 2020.

[33] Association for Advancing Automation. *ANSI/RIA R15.08-2-2023: American National Standard for Industrial Mobile Robots — Safety Requirements — Part 2: Requirements for IMR Systems and IMR Applications*. Standard ANSI/RIA R15.08-2-2023. Ann Arbor, MI: Association for Advancing Automation (A3), 2023.

[34] International Organization for Standardization. *ISO 3691-4:2023: Industrial trucks — Safety requirements and verification — Part 4: Driverless industrial trucks and their systems*. Standard ISO 3691-4:2023. Geneva, Switzerland: International Organization for Standardization, 2023.

[35] Diego Paez-Granados and Aude Billard. 'Crash test-based assessment of injury risks for adults and children when colliding with personal mobility devices and service robots'. In: *Scientific Reports* 12 (2022), p. 5402. DOI: 10.1038/s41598-022-09349-9.

[36] Nathan E. Sanders, Elif Şener and Karen B. Chen. 'Robot-related injuries in the workplace: An analysis of OSHA Severe Injury Reports'. In: *Applied Ergonomics* 121 (2024), p. 104324. DOI: 10.1016/j.apergo.2024.104324.

[37] Pranav Pandey, Ramviyas Parasuraman and Prashant Doshi. *FRESHR-GSI: A Generalized Safety Model and Evaluation Framework for Mobile Robots in Multi-Human Environments*. 2025. arXiv: 2501.03467 [cs.RO]. URL: https://arxiv.org/abs/2501.03467.

[38] Moo Jin Kim et al. 'OpenVLA: An Open-Source Vision-Language-Action Model'. In: *Conference on Robot Learning (CoRL)*. 2024.

[39] Haotian Liu et al. 'Improved Baselines with Visual Instruction Tuning'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024. arXiv: 2310.03744.

[40] Xuweiyi Chen et al. 'Multi-Object Hallucination in Vision-Language Models'. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. arXiv: 2407.06192 [cs.CV].

[41] Michael Ahn et al. 'Do As I Can, Not As I Say: Grounding Language in Robotic Affordances'. In: *Proceedings of the 6th Conference on Robot Learning (CoRL)*. Vol. 205. Proceedings of Machine Learning Research. 2022, pp. 287–318. arXiv: 2204.01691.

[42] Wenlong Huang et al. 'Inner Monologue: Embodied Reasoning through Planning with Language Models'. In: *Proceedings of the 6th Conference on Robot Learning (CoRL)*. Vol. 205. Proceedings of Machine Learning Research. 2022, pp. 1769–1782. arXiv: 2207.05608.

[43] Peiqi Liu et al. 'OK-Robot: What Really Matters in Integrating Open-Knowledge Models for Robotics'. In: *Robotics: Science and Systems (RSS)*. 2024. arXiv: 2401.12202.

[44] Li Liu et al. 'Deep Learning for Generic Object Detection: A Survey'. In: *International Journal of Computer Vision* 128.2 (2020), pp. 261–318. DOI: 10.1007/s11263-019-01247-4. arXiv: 1809.02165.

[45] Tianheng Cheng et al. 'YOLO-World: Real-Time Open-Vocabulary Object Detection'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024. arXiv: 2401.17270.

[46] Ao Wang et al. 'YOLOE: Real-Time Seeing Anything'. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2025. arXiv: 2503.07465.

[47] Lukas Rustler, Vojtech Volprecht and Matej Hoffmann. 'Empirical Comparison of Four Stereoscopic Depth Sensing Cameras for Robotics Applications'. In: *IEEE Access* 13 (2025), pp. 67564–67577. DOI: 10.1109/ACCESS.2025.3559874.

[48] Yifu Zhang et al. 'ByteTrack: Multi-Object Tracking by Associating Every Detection Box'. In: *European Conference on Computer Vision (ECCV)*. Vol. 13682. Lecture Notes in Computer Science. 2022, pp. 1–21. DOI: 10.1007/978-3-031-20047-2_1.

[49] Nir Aharon, Roy Orfaig and Ben-Zion Bobrovsky. *BoT-SORT: Robust Associations Multi-Pedestrian Tracking*. 2022. arXiv: 2206.14651 [cs.CV]. URL: https://arxiv.org/abs/2206.14651.

[50] José Luis Pech-Pacheco et al. 'Diatom autofocusing in brightfield microscopy: a comparative study'. In: *Proceedings 15th International Conference on Pattern Recognition (ICPR)*. Vol. 3. 2000, pp. 314–317. DOI: 10.1109/ICPR.2000.903548.

[51] Vasilii Korolkov. *Scene Detection Policies and Keyframe Extraction Strategies for Large-Scale Video Analysis*. May 2025. arXiv: 2506.00667 [cs.CV]. URL: https://arxiv.org/abs/2506.00667.

[52] Hao Liang et al. *KeyVideoLLM: Towards Large-scale Video Keyframe Selection*. 2024. arXiv: 2407.03104 [cs.CV]. URL: https://arxiv.org/abs/2407.03104.

[53] Wonkyun Kim et al. *An Image Grid Can Be Worth a Video: Zero-shot Video Question Answering Using a VLM*. 2024. arXiv: 2403.18406 [cs.CV]. URL: https://arxiv.org/abs/2403.18406.

[54] Iro Armeni et al. '3D Scene Graph: A Structure for Unified Semantics, 3D Space, and Camera'. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 5664–5673. DOI: 10.1109/ICCV.2019.00576.

[55] Antoni Rosinol et al. 'Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping'. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2020. arXiv: 1910.02490.

[56] Antoni Rosinol et al. '3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans'. In: *Robotics: Science and Systems (RSS)*. 2020. arXiv: 2002.06289.

[57] Nathan Hughes, Yun Chang and Luca Carlone. 'Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization'. In: *Robotics: Science and Systems (RSS)*. 2022. DOI: 10.15607/RSS.2022.XVIII.050.

[58] Qiao Gu et al. 'ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning'. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. 2024. arXiv: 2309.16650.

[59] Hao Wang et al. *VisionGPT: LLM-Assisted Real-Time Anomaly Detection for Safe Visual Navigation*. 2024. arXiv: 2403.12415 [cs.CV]. URL: https://arxiv.org/abs/2403.12415.

[60] Boyuan Chen et al. 'SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024. arXiv: 2401.12168.

[61] An-Chieh Cheng et al. 'SpatialRGPT: Grounded Spatial Reasoning in Vision-Language Models'. In: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. 2024. arXiv: 2406.01584.

[62] Jianwei Yang et al. *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*. 2023. arXiv: 2310.11441 [cs.CV]. URL: https://arxiv.org/abs/2310.11441.

[63] Soroush Nasiriany et al. 'PIVOT: Iterative Visual Prompting Elicits Actionable Knowledge for VLMs'. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 2024. arXiv: 2402.07872 [cs.RO]. URL: https://arxiv.org/abs/2402.07872.

[64] Simon Diemert and Jens H. Weber. 'Can Large Language Models Assist in Hazard Analysis?' In: *Computer Safety, Reliability, and Security. SAFECOMP 2023 Workshops*. Vol. 14182. Lecture Notes in Computer Science. Springer, 2023. DOI: 10.1007/978-3-031-40953-0_35.

[65] Yi Qi et al. 'Safety Analysis in the Era of Large Language Models: A Case Study of STPA using ChatGPT'. In: *Machine Learning with Applications* 19 (2025), p. 100622. DOI: 10.1016/j.mlwa.2025.100622.

[66] Ibtissam El Hassani et al. 'Integrating large language models for improved failure mode and effects analysis (FMEA): a framework and case study'. In: *Proceedings of the Design Society* 4 (2024), pp. 2019–2028. DOI: 10.1017/pds.2024.204.

[67] Junseo Lee et al. 'Can large language models automate the HAZOP process without human intervention?' In: *Safety Science* 194 (2026), p. 107039. DOI: 10.1016/j.ssci.2025.107039.

[68] Patrick Lewis et al. 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'. In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2020, pp. 9459–9474.

[69] Oded Ovadia et al. 'Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs'. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 237–250. DOI: 10.18653/v1/2024.emnlp-main.15.

[70] Padma Iyenghar. 'On the Development and Application of a Structured Dataset for Data-Driven Risk Assessment in Industrial Functional Safety'. In: *2025 IEEE 21st International Conference on Factory Communication Systems (WFCS)*. 2025, pp. 1–8. DOI: 10.1109/WFCS63373.2025.11077569.

[71] Varun Magesh et al. *Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools*. 2024. arXiv: 2405.20362 [cs.CL]. URL: https://arxiv.org/abs/2405.20362.

[72] Florin Cuconasu et al. 'The Power of Noise: Redefining Retrieval for RAG Systems'. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. 2024, pp. 719–729. DOI: 10.1145/3626772.3657834.

[73] Shirui Pan et al. 'Unifying Large Language Models and Knowledge Graphs: A Roadmap'. In: *IEEE Transactions on Knowledge and Data Engineering* 36.7 (2024), pp. 3580–3599. DOI: 10.1109/TKDE.2024.3352100.

[74] Yuchen Xia, Nasser Jazdi and Michael Weyrich. 'Enhance FMEA with Large Language Models for Assisted Risk Management in Technical Processes and Products'. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2024, pp. 1–4. DOI: 10.1109/ETFA61755.2024.10710996.

[75] Lukas Bahr et al. 'Knowledge graph enhanced retrieval-augmented generation for failure mode and effects analysis'. In: *Journal of Industrial Information Integration* 45 (2025), p. 100807. DOI: 10.1016/j.jii.2025.100807.

[76] Christoph Auer et al. *Docling Technical Report*. 2024. arXiv: 2408.09869 [cs.CL]. URL: https://arxiv.org/abs/2408.09869.

[77] Michael Günther et al. *Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models*. 2024. arXiv: 2409.04701 [cs.CL]. URL: https://arxiv.org/abs/2409.04701.

[78] Carlo Merola and Jaspinder Singh. *Reconstructing Context: Evaluating Advanced Chunking Strategies for Retrieval-Augmented Generation*. Second Workshop on Knowledge-Enhanced Information Retrieval, ECIR 2025. 2025. arXiv: 2504.19754. URL: https://arxiv.org/abs/2504.19754.

[79] Soyeong Jeong et al. 'Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity'. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL): Long Papers*. 2024, pp. 7036–7050. DOI: 10.18653/v1/2024.naacl-long.389.

[80] Aditi Singh et al. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG*. 2025. arXiv: 2501.09136 [cs.AI]. URL: https://arxiv.org/abs/2501.09136.

[81] Morteza Jalali Alenjareghi et al. 'LLM-Driven FMEA for Safe Human-Robot Collaboration in Disassembly'. In: *2025 IEEE 5th International Conference on Human-Machine Systems (ICHMS)*. 2025, pp. 295–301. DOI: 10.1109/ICHMS65439.2025.11154327.

[82] Yara Mahmoud et al. 'SafeHumanoid: VLM-RAG-Driven Impedance Control of Humanoid Robot'. In: *Companion of the 2026 ACM/IEEE International Conference on Human-Robot Interaction (HRI Companion '26)*. 2026, pp. 974–978. DOI: 10.1145/3776734.3794539.

[83] Olalekan Sotomi et al. *Embodied Hazard Mitigation using Vision-Language Models for Autonomous Mobile Robots*. 2025. arXiv: 2509.06768 [cs.RO]. URL: https://arxiv.org/abs/2509.06768.

[84] Hossein Naderi et al. *Autonomous Construction-Site Safety Inspection Using Mobile Robots: A Multilayer Vision-Language Model and Large Language Model Pipeline*. 2025. arXiv: 2512.13974 [cs.RO]. URL: https://arxiv.org/abs/2512.13974.

[85] Wenxiao Zhang et al. *SafeEmbodAI: A Safety Framework for Mobile Robots in Embodied AI Systems*. 2024. arXiv: 2409.01630 [cs.RO]. URL: https://arxiv.org/abs/2409.01630.

[86] Rohan Sinha et al. 'Real-Time Anomaly Detection and Reactive Planning with Large Language Models'. In: *Proceedings of Robotics: Science and Systems (RSS)*. 2024. arXiv: 2407.08735 [cs.RO]. URL: https://arxiv.org/abs/2407.08735.

[87] Reiner Pope et al. ‘Efficiently Scaling Transformer Inference’. In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023. arXiv: 2211.05102.

[88] Zhihang Yuan et al. *LLM Inference Unveiled: Survey and Roofline Model Insights*. 2024. arXiv: 2402.16363 [cs.CL]. URL: https://arxiv.org/abs/2402.16363.

[89] Pol G. Recasens et al. ‘Mind the Memory Gap: Unveiling GPU Bottlenecks in Large-Batch LLM Inference’. In: *2025 IEEE 18th International Conference on Cloud Computing (CLOUD)*. 2025. arXiv: 2503.08311.

[90] Leela S. Karumbunathan. *NVIDIA Jetson AGX Orin Series: A Giant Leap Forward for Robotics and Edge AI Applications, Technical Brief*. Tech. rep. TB-10749-001_v1.2. NVIDIA Corporation, July 2022. URL: https://www.nvidia.com/content/dam/en-zz/Solutions/gtcf21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf. (accessed: 10.04.2026).

[91] Yue Zheng et al. ‘A Review on Edge Large Language Models: Design, Execution, and Applications’. In: *ACM Computing Surveys* (2025). DOI: 10.1145/3719664.

[92] Elias Frantar et al. ‘GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers’. In: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.17323.

[93] Ji Lin et al. *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. 2024. arXiv: 2306.00978 [cs.CL]. URL: https://arxiv.org/abs/2306.00978.

[94] Amir Zandieh et al. ‘TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate’. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2504.19874.

[95] Mayank Arya and Yogesh Simmhan. ‘Understanding the Performance and Power of LLM Inferencing on Edge Accelerators’. In: *Proceedings of the 7th Workshop on Parallel AI and Systems for the Edge (PAISE)*. Co-located with IEEE IPDPS 2025. 2025. arXiv: 2506.09554 [cs.DC]. URL: https://arxiv.org/abs/2506.09554.

[96] Xiangxiang Chu et al. *MobileVLM V2: Faster and Stronger Baseline for Vision Language Model*. 2024. arXiv: 2402.03766 [cs.CV]. URL: https://arxiv.org/abs/2402.03766.

[97] Anthony Brohan et al. ‘RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control’. In: *Proceedings of the 7th Conference on Robot Learning (CoRL)*. 2023. arXiv: 2307.15818.

[98] Manoj Vishwanathan, Suvinay Subramanian and Anand Raghunathan. ‘Characterizing VLA Models: Identifying the Action Generation Bottleneck for Edge AI Architectures’. In: *arXiv preprint* (2026). arXiv: 2603.02271 [cs.PF].

[99] Jacky Liang et al. ‘Code as Policies: Language Model Programs for Embodied Control’. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 2023, pp. 9493–9500. DOI: 10.1109/ICRA48891.2023.10160591.

[100] Yuxuan Chen, Yixin Han and Xiao Li. *FASTNav: Fine-tuned Adaptive Small-language-models Trained for Multi-point Robot Navigation*. 2024. arXiv: 2411.13262 [cs.RO]. URL: https://arxiv.org/abs/2411.13262.

[101] Javier Ortiz-Tudela et al. ‘Schema-driven prediction effects on episodic memory across the lifespan’. In: *Philosophical Transactions of the Royal Society B* 379.1913 (2024), p. 20230401. DOI: 10.1098/rstb.2023.0401.

[102] Qwen An Yang et al. ‘Qwen2.5 Technical Report’. In: *ArXiv* abs/2412.15115 (2024). URL: https://api.semanticscholar.org/CorpusID:274859421.

[103] An Yang et al. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.

[104] Elie Bakouch et al. ‘SmolLM3: smol, multilingual, long-context reasoner’. In: *Hugging Face Blog* (2025). URL: https://huggingface.co/blog/smollm3. (accessed: 08.04.2026).

[105] Google DeepMind. *Gemma-4-26B-A4B-it model card*. Hugging Face Hub. 2026. URL: https://huggingface.co/google/gemma-4-26B-A4B-it. (accessed: 11.04.2026).

[106] Leonardo Ranaldi and André Freitas. 'Aligning Large and Small Language Models via Chain-of-Thought Reasoning'. In: *Conference of the European Chapter of the Association for Computational Linguistics*. 2024. URL: https://api.semanticscholar.org/CorpusID:268417079.

[107] Qwen Team. *Qwen3-4B-Thinking-2507 model card*. Hugging Face Hub. 2025. URL: https://huggingface.co/Qwen/Qwen3-4B-Thinking-2507. (accessed: 18.04.2026).

[108] NVIDIA. *TensorRT-LLM*. 2024. URL: https://github.com/NVIDIA/TensorRT-LLM. (accessed: 08.04.2026).

[109] NVIDIA. *TensorRT-LLM (Edge LLM Inference Tutorial)*. NVIDIA Jetson AI Lab. 2024. URL: https://www.jetson-ai-lab.com/tutorials/tensorrt-edge-llm/. (accessed: 12.05.2026).

[110] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG]. URL: https://arxiv.org/abs/2309.06180.

[111] Tim Dettmers et al. 'QLoRA: Efficient Finetuning of Quantized LLMs'. In: *arXiv preprint arXiv:2305.14314* (2023).

[112] Tianyi Zhang et al. 'BERTScore: Evaluating Text Generation with BERT'. In: *ArXiv* abs/1904.09675 (2019). URL: https://api.semanticscholar.org/CorpusID:127986044.

[113] Yang Liu et al. 'G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment'. In: *ArXiv* abs/2303.16634 (2023). URL: https://api.semanticscholar.org/CorpusID:257804696.

[114] Klaus Krippendorff. 'Reliability in Content Analysis: Some Common Misconceptions and Recommendations'. In: *Human Communication Research* 30 (2004), pp. 411–433. URL: https://api.semanticscholar.org/CorpusID:15407535.

[115] Michael Günther et al. *Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents*. 2023. arXiv: 2310.19923 [cs.CL].

[116] MosaicML Engineering Team. *LLM Inference Performance Engineering: Best Practices*. Oct. 2023. URL: https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices. (accessed: 10.04.2026).

[117] NVIDIA Developer Forum. *Memory Bandwidth on Orin*. Online forum thread. 2024. URL: https://forums.developer.nvidia.com/t/memory-bandwidth-on-orin/277639. (accessed: 12.05.2026).

[118] Qwen Team. *Qwen2.5-3B model card*. Hugging Face Hub. 2024. URL: https://huggingface.co/Qwen/Qwen2.5-3B. (accessed: 19.04.2026).

[119] International Organization for Standardization. *ISO 21448:2022: Road vehicles — Safety of the intended functionality*. Standard ISO 21448:2022. Geneva, Switzerland: International Organization for Standardization, June 2022. URL: https://www.iso.org/standard/77490.html. (accessed: 13.04.2026).

[120] Sanjif Shanmugavelu et al. *Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications*. 2024. arXiv: 2408.05148 [cs.DC]. URL: https://arxiv.org/abs/2408.05148.

[121] NVIDIA Corporation. *C++ GPT Runtime — TensorRT-LLM Documentation*. Online documentation. 2024. URL: https://nvidia.github.io/TensorRT-LLM/advanced/gpt-runtime.html. (accessed: 18.04.2026).

[122] NVIDIA TensorRT-LLM Contributors. *Determinism in TensorRT-LLM (Issue #2474)*. GitHub issue. 2025. URL: https://github.com/NVIDIA/TensorRT-LLM/issues/2474. (accessed: 12.05.2026).

[123] NVIDIA. *TensorRT-LLM for Jetson (v0.12.0-jetson branch)*. NVIDIA Developer Forums. Nov. 2024. URL: https://forums.developer.nvidia.com/t/tensorrt-llm-for-jetson/313227. (accessed: 22.04.2026).

[124] NVIDIA. *JetPack 7.0 / Jetson Linux 38.2 for NVIDIA Jetson Thor is now live*. NVIDIA Developer Forums. Aug. 2025. URL: https://forums.developer.nvidia.com/t/jetpack-7-0-jetson-linux-38-2-for-nvidia-jetson-thor-is-now-live/343128. (accessed: 22.04.2026).

[125] André Schiel. *Risk-Annotated Scene Graph*. https://github.com/bonsai-garden/Risk-Annotated-Scene-Graph. Master's thesis project, University of Stuttgart, Institute of Industrial Automation and Software Engineering. 2026.

[126] NVIDIA. *Accelerating LLM and VLM Inference for Automotive and Robotics with NVIDIA TensorRT Edge-LLM*. https://developer.nvidia.com/blog/accelerating-llm-and-vlm-inference-for-automotive-and-robotics-with-nvidia-tensorrt-edge-llm/. 2025.

[127] NVIDIA Corporation. *Multi-Process Service*. Release r590. Accessed 2026-05-12. NVIDIA Corporation. Dec. 2025. URL: https://docs.nvidia.com/deploy/mps/.

[128] Aman's AI Journal. *Token Sampling Methods*. Aman's AI Journal. 2024. URL: https://aman.ai/primers/ai/token-sampling/. (accessed: 19.05.2026).

[129] IBM. *What is LLM Temperature?* IBM Think Topics. 2025. URL: https://www.ibm.com/think/topics/llm-temperature. (accessed: 19.05.2026).

[130] Horace He and Thinking Machines Lab. *Defeating Nondeterminism in LLM Inference*. Thinking Machines Lab: Connectionism. 2025. URL: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/. (accessed: 19.05.2026).

[131] Raja Gond et al. *LLM-42: Enabling Determinism in LLM Inference with Verified Speculation*. 2026. arXiv: 2601.17768 [cs.LG]. URL: https://arxiv.org/abs/2601.17768.

[132] Jiayi Yuan et al. *Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference*. Conference on Neural Information Processing Systems (NeurIPS 2025); arXiv v1 was titled "Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning". 2025. arXiv: 2506.09501 [cs.CL]. URL: https://arxiv.org/abs/2506.09501.

[133] vLLM Project. *Batch Invariance — vLLM Documentation*. Online documentation. 2025. URL: https://docs.vllm.ai/en/latest/features/batch_invariance/. (accessed: 19.05.2026).

[134] Aleksa Gordić. *Inside vLLM: Anatomy of a High-Throughput LLM Inference System*. Personal blog. 2025. URL: https://www.aleksagordic.com/blog/vllm. (accessed: 19.05.2026).

[135] LMSYS Org. *Towards Deterministic Inference in SGLang and Reproducible RL Training*. LMSYS Org Blog. 2025. URL: https://www.lmsys.org/blog/2025-09-22-sglang-deterministic/. (accessed: 19.05.2026).

Appendix

A ROS2 bag consumed by SENSE

The ROS2 bag that drives the SENSE stage was recorded on the quadruped during a manually-guided pass through the laboratory using the command below. The recording targets ROS 2 Jazzy and an Intel RealSense D455 camera, and is stored as `jazzy_d455_rgbd_rtabmap_odom_v1`.

```
source /opt/ros/jazzy/setup.bash

ros2 bag record -o jazzy_d455_rgbd_rtabmap_odom_v1 \
  /camera/camera/color/image_raw \
  /camera/camera/color/camera_info \
  /camera/camera/aligned_depth_to_color/image_raw \
  /camera/camera/aligned_depth_to_color/camera_info \
  /tf /tf_static \
  /rtabmap/odom
```

The seven topics enter SENSE through the roles listed in Table 12. The colour topic is mandatory and the reader aborts if it is absent. Non-colour topics absent from the bag are silently dropped from the subscription set.

Topic	Role in SENSE
`/camera/camera/color/image_raw`	Colour stream, drives the per-frame loop.
`/camera/camera/aligned_depth_to_color/image_raw`	Depth pre-aligned to colour at the recording side, used for back-projection of detection centres.
`/camera/camera/color/camera_info`	Colour intrinsics, consumed at back-projection time.
`/camera/camera/aligned_depth_to_color/camera_info`	Depth intrinsics, retained for diagnostics.
`/tf`	Per-tick transforms used to lift detection positions into the world frame.
`/tf_static`	Static transforms cached once per session.
`/rtabmap/odom`	Robot pose published by RTAB-Map, retained for diagnostics and per-frame timing.

Table 12: Roles of the recorded ROS2 topics inside the SENSE stage.

B G-Eval Rubrics

All metrics use GPT-4o as the judge model at temperature 0.0. Scores are on a 0 to 10 scale, normalised to 0 to 1 for reporting, with a passing threshold of 0.5.

B.1 Scene Description Accuracy

Evaluation summary. The judge, namely GPT-4o at temperature 0.0, receives the structured scene graph as input, the model's generated 45-word first-person safety description as actual output, and the human-authored reference description as expected output. The judge is told that the description must speculate on what could go wrong, namely dropping the payload, harming anyone, damaging equipment, or running extremely late, given PRIORS and per-object SCENE fields, namely zone, depth, motion state, speed, confidence, and spatial relations. The four evaluation steps assess output format compliance, grounding of mentioned objects in the scene graph,

coverage of the four failure modes, and absence of hallucinated entities. The four-band rubric in Table 13 anchors the 0–10 score, which DeepEval rescales to [0, 1] with a passing threshold of 0.5. Generation failures, namely empty outputs, scene-graph echoes, reasoning chains, or outputs above 400 characters, are excluded from this metric and instead recorded by the intent-to-treat variant as 0.

Evaluation steps.

1. *Format check.* Is the output a concise first-person scene description as the prompt requested? If the output is instead a reasoning chain, a step-by-step analysis, a copy of the input PRIORS or SCENE data, or a multi-paragraph explanation, it has failed the task regardless of content. Score near zero for outputs that are not a scene description. The reference shows the expected format.
2. *Spatial structure reflection.* Does the generated description reflect the scene graph's spatial structure? Check that objects and spatial relationships mentioned are traceable to the scene graph. The model need not mention every object, as the 45-word limit forces prioritisation, but objects it does mention must be grounded in detections.
3. *Safety consequence reasoning.* Does the generated description speculate on safety consequences connected to the four failure modes defined in the prompt: dropping the payload, harming anyone, damaging equipment, or running extremely late? The model need not raise the same concerns as the reference, but the concerns it raises must follow from the scene graph.
4. *Hallucination check.* Does the generated description fabricate objects, actors, or spatial relationships that contradict the scene graph? Generalisations and inferences about what could happen are valid interpretation, not hallucination.

Score	Expected outcome
0–1	Not a scene description. Output is empty, a reasoning chain, a copy of the input data, or a multi-paragraph analysis.
2–4	Attempts a scene description but has major problems, namely circular or repetitive text, hallucinated objects not in the scene graph, or completely wrong spatial relationships. Safety reasoning is absent or disconnected from the scene.
5–7	A coherent scene description that mentions some objects from the scene graph and raises at least one plausible safety concern. May omit some objects or lack spatial detail, but omissions are acceptable given the 45-word limit. The description is grounded in the detections.
8–10	Identifies the most relevant threats from the scene graph, reflects spatial structure and proximity, and speculates on safety consequences connected to the failure modes. Prioritises ground-zone and nearest objects. Minor omissions of low-priority flank objects are not penalised.

Table 13: Scoring rubric passed to the judge for Scene Description Accuracy.

FMECA entry serialisation. Both FMECA metrics evaluate the model's generated entries in a serialised text form rather than as structured JSON, since the GPT-4o judge processes plain text more reliably than nested objects. Each entry is rendered on one line in the format below, where `n` is the entry index within a batch and the named fields correspond to the PFMECA schema defined in Appendix J. The same serialisation is applied to ground-truth entries when they are passed as expected output for Ground-Truth Alignment.

```
[n] Step: {process_step} | Mode: {failure_mode} | Effect: {failure_effect} |
Cause: {failure_cause} | Severity: {severity_level} |
Probability: {probability_level} | Criticality: {criticality_category}
```

B.2 FMECA Faithfulness

Evaluation summary. The judge receives the model's own approximately 45-word first-person safety summary as input and the serialised FMECA entries as actual output, with no expected output provided. The judge is asked to decide whether each entry's failure mode, effect, cause, severity, and probability is grounded in what the scene description states or implies. The four evaluation steps direct the judge to identify objects and concerns in the description, check each entry for grounding, verify rating consistency with the urgency and proximity implied by the description, and assign a holistic score that is high when all entries are grounded and low when entries reference absent objects. No explicit rubric bands are passed to the judge, so DeepEval falls back to its default scoring procedure [113], namely a probability-weighted integer in 0–10 derived from the criteria and evaluation steps and rescaled to [0, 1]. When the model produces no parsed FMECA entries the metric returns 0 directly without invoking the judge. Scores are otherwise returned in [0, 1] with a passing threshold of 0.5.

Evaluation steps.

1. *Input analysis.* Read the scene description. Identify objects, spatial relationships, and safety concerns.
2. *Groundedness check.* For each FMECA entry, check whether the scene description mentions or implies the object or situation that the failure mode addresses. An entry referencing an object absent from the scene description is unfaithful.
3. *Rating consistency.* Check whether severity and probability ratings are consistent with the urgency and proximity implied by the scene description.
4. *Holistic scoring.* Assign a holistic score, high if all entries are grounded, moderate if most are, low if entries largely reference objects or situations absent from the scene description.

B.3 FMECA Ground-Truth Alignment

Evaluation summary. The judge receives the structured scene graph as input, the serialised generated FMECA entries as actual output, and the serialised ground-truth FMECA entries as expected output. The judge is told that the ground truth represents one valid expert assessment, namely not the only correct answer. The five evaluation steps direct the judge to enumerate ground-truth hazards, check coverage of each by the generated set, compare severity and probability levels treating one-level deviations as minor, penalise missing hazards more heavily than extra plausible ones, and assign a holistic score. As with Faithfulness, no explicit rubric is supplied and the default G-Eval scoring procedure is used. When the model produces no parsed FMECA entries the metric returns 0 directly without invoking the judge. Scores are otherwise returned in [0, 1] with a passing threshold of 0.5.

Evaluation steps.

1. *Reference analysis.* Read the ground-truth entries. Note failure modes, objects, severity levels, and probability levels.
2. *Coverage check.* For each ground-truth hazard, check whether any generated entry covers the same or an equivalent threat, even if worded differently.
3. *Rating comparison.* For matching hazards, compare severity and probability levels. Deviations of one level are treated as minor.
4. *Gap analysis.* Missing ground-truth hazards are penalised more heavily than the inclusion of additional plausible hazards. Failing to identify a known threat is worse than over-reporting.
5. *Holistic scoring.* Assign a holistic score, high if coverage is complete with similar ratings, low if primary hazards are missed.

C Producer-Consumer Rate Measurement

To quantify the throughput imbalance between SENSE and SPECULATE under concurrent scheduling, three rate metrics were computed from an instrumented run of Qwen2.5-3B-Instruct over the ten-scene evaluation set.

- **SENSE rate.** The number of scenes produced divided by cumulative detection time. Under sequential scheduling this figure includes GPU-lock wait time, since SENSE is blocked while SPECULATE holds the mutex, and therefore measures effective production throughput rather than raw detection speed. Under concurrent scheduling there is no lock wait, and the figure reflects unblocked YOLO processing speed taken over every scene the ROS bag would emit, including scenes subsequently discarded from the queue.
- **SPECULATE rate.** The number of completed scenes divided by cumulative model inference time, summed across scene description, memory search, sufficiency check, and final risk assessment, with queue wait excluded. This isolates model throughput from scheduling artifacts. The model runs at the same speed under both scheduling regimes, so the SPECULATE rate is identical across the two columns of Table 14.
- **Rate ratio.** The SENSE rate divided by the SPECULATE rate. A ratio near 1.0 indicates a balanced producer-consumer pair with a stable queue. A ratio well above 1.0 indicates that SENSE is producing faster than SPECULATE can consume, so the queue grows, scenes age, and scenes are eventually discarded.

Metric	Sequential	Concurrent
SENSE rate	0.18 scenes/s	1.43 scenes/s
SPECULATE rate	0.17 scenes/s	0.17 scenes/s
Rate ratio	1.06×	8.6×
Queue wait growth	∼0 ms/scene	∼3,200 ms/scene
Scenes completed	10/10	7/10

Table 14: Producer-consumer throughput, Qwen2.5-3B-Instruct (instrumented run).

The bottleneck sits entirely on the consumer side. The SPECULATE rate is constant across both regimes at 0.17 scenes per second, while the SENSE rate rises by roughly a factor of eight once the mutex is removed. Under sequential scheduling the mutex throttles SENSE to nearly the consumption rate, and the queue stays stable. Under concurrent scheduling SENSE runs unblocked at its native detection speed, and the resulting $8.6\times$ mismatch causes scenes to accumulate faster than they can be consumed. Queued scenes age while SPECULATE finishes its current inference, and any scene whose wait exceeds the 30-second staleness threshold is discarded. Queue wait grows by approximately 3.2 seconds per scene under concurrent scheduling, and 3 of the 10 scenes in this run were lost. The two drop mechanisms, namely queue-full rejection and staleness expiry, both result in silent loss with no distinguishing trace in the evaluation output, so the table reports only the net loss. Every scene that survives the queue completes inference successfully, consistent with the observation in the results section, that degradation under concurrent scheduling is a scheduling problem rather than a model-capability problem.

D Lock-Free GPU Scheduling Stall

Under concurrent GPU scheduling, the hand-off queue between SENSE and SPECULATE fills steadily because detection completes faster than reasoning, and each new scene waits longer than the last for its turn on the accelerator. This monotonic growth is the shape visible for most traces in Figure 19. The Qwen2.5-3B-Instruct trace departs from the pattern. It climbs for five scenes, drops back almost to zero, and then climbs again.

The drop is not a measurement glitch but an observed transient in the perception pipeline. In the affected run, SENSE emitted no new scene for roughly thirty seconds of wall-clock time even though the underlying ROS2 bag advanced only two seconds of logical time, meaning perception was stuck rather than short of input. While it was stuck, SPECULATE kept draining the queue, so by the time the next scene finally arrived the queue was empty and the wait had reset. Figure 29 contrasts this stalled run against a clean run of the same model on the same input.

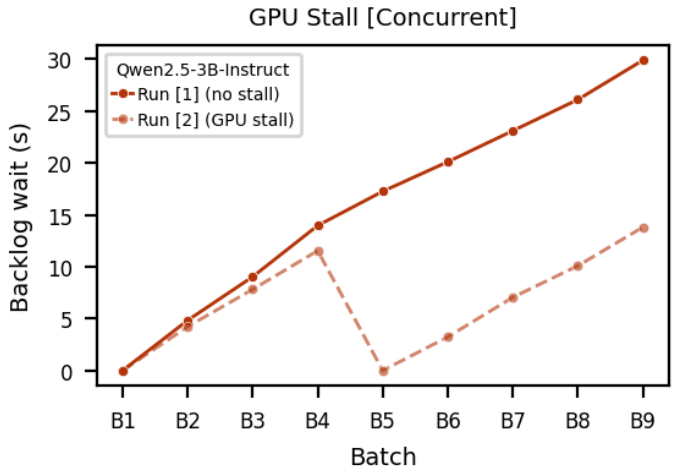


Figure 29: Per-batch backlog queue wait time for two concurrent runs of Qwen2.5-3B-Instruct

The cause lies in how the two stages share the Jetson GPU once the mutex is removed. SENSE issues short detection kernels and SPECULATE issues long generation kernels, and the CUDA dispatcher is free to run either at any given moment. In most runs the two interleave without incident, but occasionally a sequence of long generation kernels occupies the GPU back to back and the shorter detection calls are starved for the duration of that burst. This same pattern was observed in two concurrent runs, across two different models and two different scene boundaries (Figure 29 and Figure 19), which points to a scheduling artefact of lock-free GPU sharing rather than a model-specific or input-specific fault. Every affected run still completes all of its scenes and the results themselves remain valid, but the transient is a reminder that concurrent mode is not a free latency win over the serialized configuration used for the main measurements.

E Parallel Consumer Experiment

Appendix C showed SENSE producing scenes 8.6× faster than a single SPECULATE worker consumes them. A second SPECULATE worker should, by arithmetic, roughly halve that rate gap. To test this, the concurrent configuration from Section 5.4 was rerun with two SPECULATE worker threads instead of one and the queue `maxsize` raised to 6. All other parameters were held fixed.

The expected throughput gain did not materialise, for two related reasons. The TRT-LLM engine was built with `max_batch_size=1`, so concurrent requests from the two workers were queued and executed sequentially by the server rather than in parallel. Even if the server had admitted them concurrently, LLM decoding at batch size 1 is memory-bandwidth-bound, and the Jetson AGX Orin exposes a single, shared 204.8 GB/s LPDDR5 bus across the CPU, GPU, and memory with no per-client bandwidth reservation [90]. NVIDIA's Multi-Process Service provisions compute share between CUDA processes but offers no equivalent isolation for memory bandwidth [127]. Two concurrent inference streams therefore contend for the same fixed pool.

Total inference per batch nearly doubled, from roughly 6.0 s under serialised execution to 11.1 s with two workers, and 6 of 10 scenes failed at the scene description stage. Under sustained bandwidth pressure the autoregressive decoder emitted token sequences that passed vocabulary

checks but did not conform to the expected scene schema, and were rejected by the pipeline's structural validation. The two scenes that survived took 14.3 and 15.6 seconds end-to-end, more than double the serialised baseline. Per-scene timings across all three configurations are shown in Figure 30. Pink crosses on the figure mark the scenes where the LLM produced unparseable output.

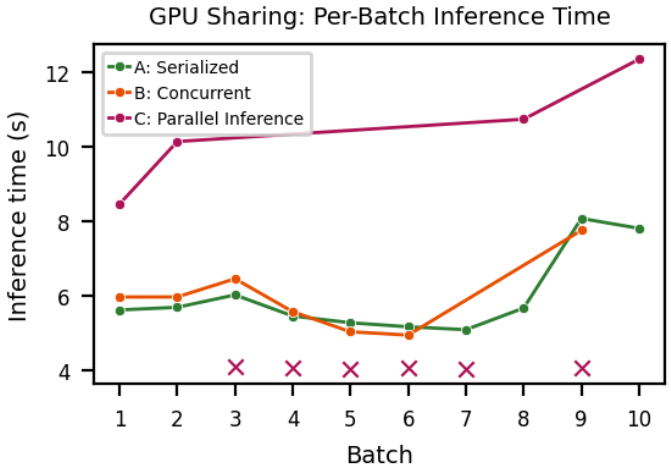


Figure 30: Per-batch inference time, single worker and two concurrent workers.

The two-worker configuration appears to stabilise the queue, with average wait time falling from 11.5 s under a single concurrent worker to 2.3 s. This is an artifact of the slowdown, not an improvement. Per-batch inference time rose to the point where the consumer no longer trails the producer, so the queue stops accumulating. Net usable output still falls from 7 of 10 scenes under a single concurrent worker to 3 of 10 with two. Thus, on a single-GPU edge platform with unified memory, software-level consumer parallelism does not improve throughput.

F Qwen3-4B Thinking Model Failure Analysis

Qwen3-4B is a 4-billion parameter reasoning model from the Qwen3 family, served through vLLM 0.9.2 with INT4 AWQ quantisation. Like SmolLM3-3B, it supports extended thinking via `<think>` blocks and receives the same 592-token budget, namely 60 answer tokens plus 512 thinking overhead. The scene description timeout is set to 120 seconds.

The failure mechanism and per-scene gate violations are reported in the viability analysis, Section 5.6. This appendix supplies the per-scene quantitative trace behind that summary. Across the nine scenes, the visible output length lies between 1,921 and 2,405 characters, roughly five times the 400-character ceiling. Scene description times span 52 to 54 seconds, well within the 120-second scene description timeout, and token utilization spans 81 to 102 percent of the 592-token budget. The model therefore generates substantial output tokens and completes within budget, and the failure is neither a resource nor a time constraint. It just emits thinking tokens. The three panels of Figure 31 show (a) the scene description time and percentage token budget consumed, (b) the output length per scene, and (c) the token utilization.

Table 15 gives the per-scene counts behind the summary reproduced in the main text.

Unlike SmolLM3, where a non-thinking variant was evaluated by disabling extended reasoning via `enable_thinking=False`, a corresponding evaluation was not conducted for Qwen3-4B. The Qwen3 family supports a hybrid thinking toggle through the `enable_thinking` chat template parameter, but disabling it for Qwen3-4B would produce a configuration functionally equivalent to the Qwen2.5 instruction-tuned models already covered by the primary evaluation. The value of the Qwen3-4B experiment lies in confirming that the thinking mechanism itself is the source of failure.

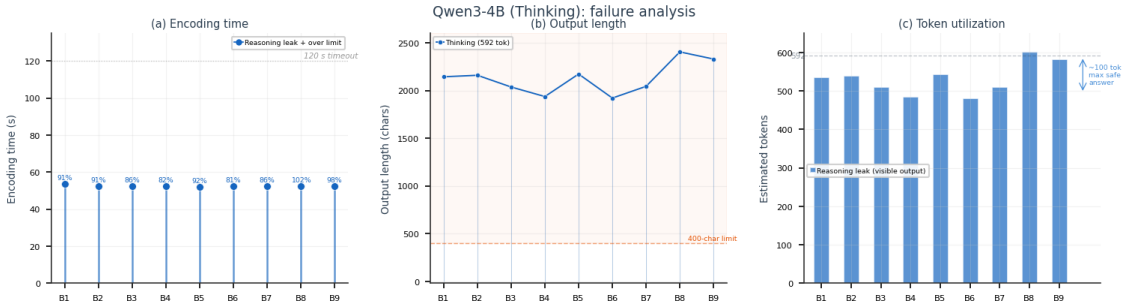


Figure 31: Qwen3-4B (Thinking) Failure

Failure mode	SmolLM3-3B	Qwen3-4B
Scenes completed	0/9	0/9
Empty output (think exhausted)	5/9	0/9
Reasoning leak	0/9	9/9
Over 400-character limit	4/9	9/9
Mean scene description time	85 s	53 s
Mean output length	270 chars	2,128 chars
Root cause	Token budget exhaustion	Format non-compliance

Table 15: Failure mode comparison between thinking-enabled models.

G Mechanism of Run-to-Run Variability in Language Model Inference

This appendix expands the chain referenced in Section 5.9, tracing the mechanism by which the floating-point arithmetic of a GPU forward pass propagates into run-to-run differences in the ordinal FMECA ratings the pipeline emits. The chain has six steps, presented in order.

G.1 Autoregressive decoding

A language model does not produce a response in one shot. It produces one token at a time, and each new token is selected on the basis of every token that came before it [128]. The mechanism is the following. The model receives a sequence of input tokens, namely the prompt, and runs a forward pass that produces a logits vector at the position immediately after the last input token. That vector is converted to a distribution, a token is sampled from it, and the sampled token is appended to the input sequence. The model is then run a second time, with the prompt now one token longer than it was before, and the forward pass produces a fresh logits vector at the new end of the sequence. A second token is sampled and appended. The loop continues until a stop token is emitted, the maximum token budget is reached, or some other termination condition is met.

The loop is autoregressive because each output is conditioned on every prior output, including the ones the model itself just produced [128]. The property that matters for reproducibility is that a single different token early in the sequence does not merely substitute itself for the token that would have been selected. It also changes the input to every subsequent forward pass and therefore changes every logits vector the model computes after that point. Token-level drift compounds along the sequence rather than staying local.

G.2 Logits, softmax, and sampling

The forward pass over a prompt ends with a vector of unnormalised real values, one per token in the model's vocabulary, called logits [128]. For a vocabulary of one hundred thousand tokens, the

vector has one hundred thousand entries, each representing how strongly the model prefers the corresponding token as the next output. The logits cannot be sampled from directly. A softmax operation exponentiates each entry and normalises the result so that the entries sum to one, producing a probability distribution over the vocabulary.

The sampling temperature divides every logit by a scalar T before the softmax is applied, with lower T sharpening the distribution toward the highest-probability tokens and higher T flattening it [129]. Greedy decoding, which corresponds to T approaching zero, always selects the single highest-probability token and is theoretically deterministic, since no random draw is involved [130]. Non-zero temperatures introduce a pseudo-random draw from the distribution, and a random seed parameter determines the sequence of draws so that, in principle, the same seed and the same distribution should select the same token at every position [128].

If the forward pass produced bit-identical logits on each repeat, the softmax would produce a bit-identical distribution, the seeded draw would select the same token, and the autoregressive loop would unroll into the same token sequence on every run [130]. A run-to-run identical token sequence is therefore the property that any reproducibility argument for a language-model-based component must establish.

G.3 Matrix multiplication and accumulation

A neural network's forward pass is built almost entirely on matrix multiplications, dozens of them per layer across dozens of layers. A single matrix multiplication is a large set of dot products, and each dot product is a sum of many small products of floating-point numbers. Taking a four-element row vector $A_{\text{row}} = [a_1, a_2, a_3, a_4]$ and a four-element column vector $B_{\text{col}} = [b_1, b_2, b_3, b_4]$, their dot product is

$$a_1b_1 + a_2b_2 + a_3b_3 + a_4b_4.$$

This sum is computed by accumulation. The hardware starts with a running total set to zero, computes a_1b_1 and adds it into the running total, computes a_2b_2 and adds it into the running total, and so on. The running total is the accumulator, each addition is an accumulation step, and the order in which the additions happen is the accumulation order.

For a transformer layer whose hidden dimension is several thousand and whose input is a sequence of several hundred tokens, a single matrix multiplication performs on the order of billions of floating-point additions, and a forward pass involves dozens of matrix multiplications at every layer across dozens of layers. The total floating-point addition count per token produced therefore reaches into the tens of billions.

G.4 Floating-point non-associativity

Floating-point addition is not associative [120]. Because a value is stored as a fixed-width mantissa and an exponent, adding operands of very different magnitudes discards low-order bits of the smaller one, by an amount that depends on the order of the additions [130]. A GPU does not sum a dot product sequentially on one compute unit. It splits the products into groups, reduces each group in parallel, and combines the partial sums [120]. Two repeats whose split or combination order differs evaluate the same dot product as $(a_1b_1 + a_2b_2) + (a_3b_3 + a_4b_4)$ on one run and $a_1b_1 + (a_2b_2 + (a_3b_3 + a_4b_4))$ on the other, and the two parenthesisations round differently in finite precision. The per-entry difference is small, but it is incurred at every entry, carried through every subsequent layer, and accumulated into the final logits vector [131, 132].

G.5 Where the accumulation order is chosen

The accumulation order inside a dot product is not fixed by the mathematical definition of the operation. It is chosen by the GPU kernel, namely the program that performs the matrix multiplication

on a specific tensor shape, on the basis of the shape it sees and the performance trade-offs the kernel author selected. For a given input matrix size, the kernel tiles the work into blocks, assigns each block to a compute unit, and then reduces the partial sums those units produce into the final output [131]. Two kernels written for the same operation can tile the work differently, choose different block sizes, or reduce the partial sums in a different sequence, and each of these choices changes the accumulation order.

Inference engines differ in when and how they select kernels. TensorRT-LLM is a compiled-engine backend, namely a build-time tool that takes the model weights and a fixed configuration, including the maximum batch size, the maximum sequence length, and the precision, and produces a single binary engine in which the kernels for every operation have already been selected, tuned, and frozen [121]. When a request arrives at runtime, the engine executes the same compiled kernels in the same order on every call. NVIDIA documents that this regime is deterministic when a single request is in flight and the maximum batch size is one [122], because the kernel sees the same shape on every call and was tuned for that shape at build time, so the accumulation paths that would otherwise vary with batch size do not vary across repeats. The trade-off NVIDIA's documentation flags is throughput, since serving every request at batch size one and forbidding co-batching means the GPU runs at lower utilisation than it would under dynamic batching. The trade-off applies to deployments that serve multiple requests concurrently, namely interactive endpoints or batched offline jobs, but does not apply to the pipeline described in this thesis, since SPECULATE issues at most one model request at a time and never asks the inference server to co-batch. The engine workflow described here is the one used by TensorRT-LLM versions prior to 1.0. Later releases transitioned to a PyTorch workflow without prebuilt engines, for which determinism behaviour has not been publicly characterised.

vLLM is a dynamic-kernel backend. It retains the ability to choose among multiple kernel implementations at runtime, varies tiling and split strategies based on the shape of the input each request presents, and combines requests through continuous batching [133, 134]. By the same structural reasoning, llama.cpp is also a dynamic-kernel backend, since it performs no build-time engine compilation that would fix kernel selection ahead of execution, although this characterisation is inferred from the absence of an engine-build step rather than separately documented. The same logical operation is therefore evaluated through different accumulation paths whenever the runtime context differs, and the resulting drift propagates through the forward pass into the token sequence. The same effect has been demonstrated directly for production inference servers, where varying batch composition changes the reduction order of shared kernels and yields run-to-run differences unless batch-invariant kernels are substituted [130, 135].

G.6 From token drift to ordinal disagreement

The token sequence the model emits is parsed downstream into the FMECA fields the pipeline ingests, namely severity, probability, and the textual rationale that accompanies each. A drift of a single token at a position where the model emits the severity letter, or at a position whose value the parser maps to a different ordinal level, is sufficient to change the rating in that cell. Krippendorff's Alpha [114] then registers the change as inter-rater disagreement among the repeats, since the three runs are treated as three raters of the same item, and Alpha can fall below the 0.667 floor that Krippendorff identifies as the lowest limit for even tentative conclusions, beneath which the rated data are discarded as unreliable.

H Writeback Per-Cell Latency on Already-Sufficient Scenes

The S$\rightarrow$S row in Table 9 reports two regime means that sit on opposite sides of zero, namely $+0.19$ s under sequential scheduling and -0.21 s under concurrent. The per-cell values below show that the sign flip is not measurement noise and that the two regime means describe two different effects rather than one effect that changes direction with scheduling.

Table 16 lists the per-repeat end-to-end latency on S8 and S9 under sequential scheduling.

Regime	Cell	Baseline e2e	Writeback e2e	Δ
Sequential	S8, run 1	3.99 s	4.78 s	$+0.79$
Sequential	S8, run 2	4.00 s	4.77 s	$+0.77$
Sequential	S8, run 3	3.96 s	4.74 s	$+0.79$
Sequential	S9, run 1	5.28 s	4.82 s	-0.45
Sequential	S9, run 2	5.29 s	4.87 s	-0.42
Sequential	S9, run 3	5.22 s	4.86 s	-0.36

Table 16: Per-repeat end-to-end latency on the two S→S scenes under sequential scheduling.

The run-to-run standard deviation in every cell is approximately 0.02 s, so the $+0.78$ s delta on S8 and the -0.41 s delta on S9 are stable scene-specific effects. The pair mean of $+0.19$ s is the residual after the S8 slowdown and the partial S9 speedup, and sequential scheduling carries no queue between scenes, so this residual measures only the direct effect on the inference layer of reshuffling the top-K retrieval context on already-sufficient scenes. Reshuffling does not, on its own, speed these scenes up.

Table 17 lists the per-repeat values under concurrent scheduling, with the end-to-end time separated into the queue-wait that precedes the scene and the inference time spent on it.

Cell	Cond.	wait r1	wait r2	wait r3	Inf r1	Inf r2	Inf r3
S8	Baseline	22.85	22.40	22.53	3.84	3.84	3.84
S8	Writeback	21.97	21.93	21.66	4.55	4.60	4.58
S9	Baseline	24.38	23.95	24.08	5.11	5.10	5.08
S9	Writeback	24.21	24.24	23.96	4.68	4.70	4.68

Table 17: Queue-wait and inference time on the two S→S scenes under concurrent scheduling

The end-to-end delta on S8 is -0.01 s but its components are not flat. The inference component rises by $+0.74$ s under writeback, mirroring the sequential S8 slowdown and confirming that the reshuffle penalty on S8 persists into the concurrent regime. The queue-wait component falls by -0.74 s, cancelling the inference cost at the end-to-end level. The S9 end-to-end delta is -0.42 s, all of it on the inference component, and the queue-wait component is flat because by the time S9 is reached the queue has already plateaued and no further relief is available.

The concurrent pair mean of -0.21 s therefore arises from two separate sources. On S8, queue relief inherited from the upstream N→S path flips cancels the reshuffle penalty that the inference layer continues to pay. On S9, the inference layer benefits directly from the reshuffle, and there is no further queue relief to add. The S→S row in the main body therefore corroborates the queue-relief half of the writeback mechanism on S8 and shows that the inference-layer effect on S9 carries the remainder of the concurrent pair mean.

I Seed process FMEA

This appendix reproduces an excerpt of the FMEA used to seed the FMEA knowledge graph in Section 4.2, covering navigation hazard-zone entries.

Process step	Failure mode	Effects	S	Causes	O	Controls	D	RPN	Recommended actions	Responsible
Navigation (hazard zone)	Wet surface not visually distinguishable from dry floor by perception system	Robot enters wet area without recognizing hazard; loss of traction causes slip; uncontrolled motion during slip may contact equipment or persons; fall and payload loss	7	Wet floor appearance similar to dry floor under current lighting; no explicit wet floor signage in area; perception not trained to detect wet surface texture; lighting angle eliminates reflective cues that indicate wetness	4	Physical wet-floor signage when placed; personnel expected to report spills	6	168	Train perception on wet vs dry floor appearance under varied lighting; implement floor texture analysis for anomaly detection; add traction estimation via motor current analysis detecting slip onset; conservative speed in uncertain areas	Developer; Lab Safety Lead
Navigation (hazard zone)	Cable detected but leg trajectory planning fails to avoid contact	Leg catches cable during step; cable pulled from equipment causing power loss or data interruption; cable connector damaged; displaced cable creates trip hazard for humans	6	Cable height at boundary of leg swing trajectory; gait pattern not adjusted for cable presence; cable position changed between detection and leg placement; multiple cables create complex avoidance geometry	4	Cable detection in perception; standard gait patterns	5	120	Implement cable-aware gait with increased leg clearance when cables detected; add real-time leg trajectory validation against detected obstacles; require cable covers at designated crossing points; verify cable positions during approach	Developer; Lab Technician
Navigation (hazard zone)	Robot displaces cable creating new trip hazard for humans	Human trips on displaced cable; injury from fall; equipment pulled if human catches cable; secondary incident affects uninvolved personnel	7	Leg contact with cable during transit despite detection; cable not adequately secured to floor; gait pattern creates lateral force on cable; cable slack allows significant displacement	4	Cable taping protocol; operator walkthrough before mission	7	196	Post-transit inspection of cable crossing zones before personnel resume movement; operator walkthrough after mission; mandate cable covers preventing displacement; add cable zone status to mission summary	AMR Operator; Lab Technician

Table 18: Excerpt of the hand-authored process FMEA covering the navigation entries.

J Pipeline Prompt Templates

This appendix lists the prompts used by the SPECULATE pipeline.

Token	Source	Used in
{objects_description}	serialised scene graph (PRIORS+SCENE)	Scene Description
{situation_text}	Scene Description output	Sufficiency, FMECA
{detected_objects}	per-object detection list	Sufficiency, FMECA
{fmea_entries}	Neo4j top-k cosine retrieval	Sufficiency, FMECA
{normative_context}	ChromaDB incident reports	FMECA (novel)
{uncovered_classes}	classes below 0.65 similarity	FMECA (novel)

Table 19: Runtime token substitutions.

J.1 Scene Description

Output. 45-word first-person scene description.

System prompt. Scene Description

You are a safety observer for a quadruped robot carrying a payload through a shared laboratory environment. The robot must not drop the payload, harm anyone, or damage equipment or run extremely late. The scene graph has two sections. PRIORS lists each object class with its risk tags, retrieval hints, and typical associations. SCENE lists each detected object with zone, depth, motion state, speed, confidence, and spatial relations. Zone ground is the immediate forward path. Left and right are flanks and front is further ahead. Interpret the scene, do not transcribe tracker IDs or raw metric values from the input. In exactly 45 words, in first person, describe the scene and speculate what could go wrong if the robot continues on its current path. Consider: 1. What is happening around the robot right now? 2. Who or what could the robot interact with. Nearest objects on the ground path are the most immediate threats. 3. What could prevent safe and timely delivery?

User prompt. Scene Description

Analyze this scene:
{objects_description}

J.2 Sufficiency Check

Output. `SUFFICIENT|NOVEL: <one-sentence reason>`.

System prompt. Sufficiency Check

Given the situation and the retrieved FMEA references, determine if a reasonable FMECA-style risk estimation can be made. The FMEA entries do not need to cover every object exactly. If the major hazards in the scene can be assessed using the retrieved references as a basis, answer SUFFICIENT. If the scene contains hazards with no relevant FMEA precedent, answer NOVEL.
Format: SUFFICIENT|NOVEL: <one-sentence reason>

User prompt. Sufficiency Check

Situation:
{situation_text}
Objects:
{detected_objects}
FMEA entries:
{fmea_entries}
Can a reasonable FMECA risk estimation be made from these references?
Answer SUFFICIENT or NOVEL with a brief reason

J.3 FMECA Generation

Two user-prompt variants share one system prompt, routed by the sufficiency check. Output is a JSON array whose object keys follow Table 20.

Key	Field	Constraint
ps	process_step	free text
fm	failure_mode	what robot fails to do with <object_id>, max 12 words
fe	failure_effect	1 sentence
fc	failure_cause	1 sentence
fme	failure_measure	1 sentence
sl	severity	I / II / III / IV
sr	severity_reason	brief text
pl	probability	A–E
pr	probability_reason	brief text

Table 20: FMECA JSON schema.

System

System prompt. FMECA, shared by both paths Generate PFMECA failure mode entries for a quadruped robot in a university lab. Ground each entry in the specific objects and geometry of this scene. Do NOT copy text from existing FMEA entries. Write failure modes that describe what the robot could fail to do given the specific objects detected. Name the object ID (e.g. chair_6, person_1) in every failure_mode.
Severity levels. Use IV if a person is at risk of injury. Use III if equipment is destroyed or the robot suffers major damage. Use II if operation is degraded or minor damage occurs. Use I if the consequence is negligible.
Probability levels. Apply the first rule that matches.
If the object is static and its class is typically static, use E.
If the object is static and its class is typically dynamic, use D.
If the object is dynamic and approaching in the ground zone, use A.
If the object is dynamic and approaching in the front zone, use B.
If the object is dynamic and traversing laterally, use C.
If the object is dynamic and receding, use D.
If motion state is unknown, use C.

User prompt. FMECA Sufficient path

FMEA reference (do not copy):
{fmea_entries}
Situation:
{situation_text}
Objects in this scene:
{detected_objects}
Generate 1 JSON object for the primary hazard in this scene.
Output a JSON array. No preamble, no explanation. Keys per Table 20. /no_think

User prompt. FMECA Novel path

FMEA reference (do not copy):
{fmea_entries}
Normative context:
{normative_context}
Situation:
{situation_text}
Objects in this scene:
{detected_objects}
Objects with no FMEA precedent: {uncovered_classes}
Prioritize hazard entries for these uncovered objects.

Generate 1 JSON object for the most critical hazard from uncovered objects. Output a JSON array. No preamble, no explanation. Keys per Table 20. /no_think

K Ground-Truth Scene Descriptions and FMECA Entries

This appendix presents the ground truth used in the correctness evaluation. For each of the nine evaluation scenes, a reference scene description and a reference FMECA entry were authored using the same structured scene graph that the pipeline receives as input. Severity, probability, and criticality assignments follow the definitions in Tables 1, 2, and 3. Table 21 lists the scene descriptions. Table 22 lists the structured fields of the FMECA entries, and Table 23 lists the corresponding severity and probability rationales.

Scene	Description
1	The robot is near several chairs and a carton box on its path ahead. A person is also ahead and moving. The nearest chair could collide with the robot or block the path. The robot risks startling the person or colliding with the carton box.
2	The robot is near chairs and a carton box on its path ahead. A chair is also to the left. Two persons are further ahead in the front. The robot could collide with the nearest chair or the box, blocking the path or causing payload loss.
3	The robot is approaching a chair and a carton box ahead on its path. A person is walking further ahead. The robot could collide with the chair or carton box, blocking the path. The person could be startled or trip over the robot.
4	The robot is approaching a person who is very close and moving toward it. A carton box is on the left near the path. A shelf and another person are further ahead. The robot could collide with the person, risking injury, or trip over the carton box.
5	The robot is approaching a chair on the ground path with a person walking further ahead. The robot could collide with the chair, blocking the path or causing payload loss. The person could be startled or trip over the robot if it continues forward.
6	The robot is approaching chairs and a person ahead on the ground path. The nearest chair could block the path or cause a collision. The person is further ahead and could be startled or trip over the robot if it continues forward.
7	The robot is approaching a table ahead with a chair to the left and a person in front. The chair could block the robot's path or cause a collision. If the robot continues, it may collide with the table or the person and lose the payload.
8	A person is approaching the robot on the ground path at very close range. The person could trip over the robot or collide with it. If the robot continues, it risks causing injury or losing the payload. An emergency stop is needed to avoid a collision.
9	The robot is very close to a person and a table on the ground path. The person is to the left of the table and could trip over the robot or collide with it. If the robot continues, it risks causing injury or colliding with the table.

Table 21: Ground-truth scene descriptions for the nine evaluation scenes.

Scene	Process Step	Failure Mode	Effect	Cause	Control Measure	S	P	C
1	Forward navigation toward destination	Robot fails to avoid chair_5 or box_9 on the path	Path blocked; payload may be lost on impact	Detection or planning failure on static obstacle	Reduce speed and steer around; stop if clearance is insufficient	II	E	5
2	Forward navigation toward destination	Robot fails to avoid chair_5 or box_9 on the path	Path blocked; payload may be lost on impact	Detection or planning failure on static obstacle	Reduce speed and steer around; stop if clearance is insufficient	II	E	5
3	Forward navigation with dynamic person at a distance	Robot fails to avoid chair_3 and box_9 while person_1 approaches	Path blocked from collision; person may be startled if approach is abrupt	Detection failure on static obstacles compounded by person presence	Reduce speed and yield to person; navigate around obstacles or stop	II	D	4
4	Forward navigation with dynamic person at close range	Robot fails to avoid person_1 who is approaching very close	Collision with person causing injury requiring attention	Insufficient stopping distance for dynamic person in front zone	Immediate stop; resume only after person clears	III	B	1
5	Forward navigation toward destination	Robot fails to avoid chair_6 on the ground path	Path blocked; payload may be lost on impact	Detection or planning failure on static obstacle	Steer around chair or stop	II	E	5
6	Forward navigation with person at moderate distance	Robot fails to avoid chair_6 or chair_71 on the path	Path blocked; person may be startled if approach is abrupt	Detection failure on static chairs with person presence in path	Reduce speed and steer around chairs; yield to person	II	D	4
7	Forward navigation at destination approach	Robot fails to avoid chair_6 on the left	Chair collision causing payload loss or uncontrolled motion toward person or table	Lateral detection failure on chair in tight clearance near table	Slow approach and stop; verify clearance before passing	II	E	5
8	Forward navigation with dynamic person at extreme close range	Robot fails to avoid collision with person_7 at close range	Collision with person causing serious injury	No stopping distance available for dynamic person in ground zone	Emergency stop; await person clearance	IV	A	1
9	Forward navigation at final approach with static person and table	Robot fails to avoid person_7 or table_103 at close range	Collision with table or person; possible trip	Insufficient clearance from static person and table at final approach	Stop and reposition; verify clearance before continuing	III	D	3

Table 22: Ground-truth FMECA entries for the nine evaluation scenes, structured fields.

Scene	Severity Rationale	Probability Rationale
1	Payload loss from collision; person far ahead and not likely injured	Chair_5 and box_9 are static and typically static; avoidable with detection
2	Payload loss from collision; persons far ahead and not likely injured	Chair_5 and box_9 are static and typically static; avoidable with detection
3	Payload loss from collision; person may be startled but not likely injured	Person_1 is static but typically dynamic; chair_3 and box_9 are static on the path
4	Person in front zone at 1.6 m; potential trip or collision causing injury requiring attention	Person_1 is dynamic and approaching in the front zone; collision likely without immediate stop
5	Payload loss from chair collision; person far ahead and not likely injured	Chair_6 is static and typically static; avoidable with detection
6	Payload loss from collision; person at moderate distance and not likely injured	Person_7 is static but typically dynamic in the ground zone; chairs are static
7	Chair collision may cause payload loss or uncontrolled motion toward person	Chair_6 is static and typically static; robot has time to detect and avoid
8	Person at extreme close range and approaching; very high injury potential	Person_7 is dynamic and approaching in the ground zone
9	Person at very close range; potential injury from trip or collision with robot	Person_7 is static but typically dynamic; table_103 is static and typically static

Table 23: Severity and probability rationales for the ground-truth FMECA entries.

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e.g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Additionally, I declare that Claude (Anthropic), specifically the Claude Opus 4.6 and Claude Opus 4.7 models, was used to assist with literature review, refinement of the written text, and code refactoring. All content produced with the assistance of the tool was reviewed, fact-checked, and edited by me. Further, I can attest that the content is true and follows the actual work I have done.



(Shashank Birur Manjunatha)
24th May 2026